

Lecture 01 : Recap Variable, sequence, and condition.

In last semester in course of programming fundamentals you understand the programming concept using C/C++ language.

1. Variable: "Balti"

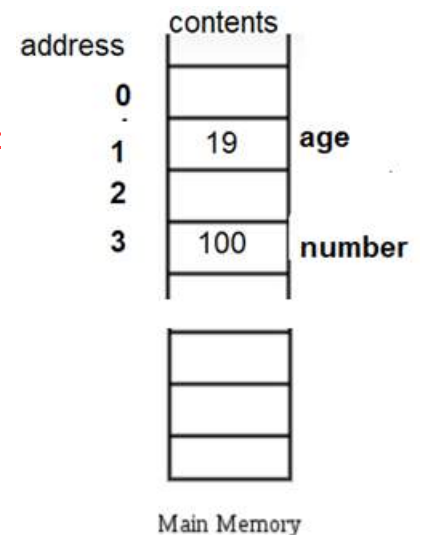
- The program (actual code) is running within the computer's memory (RAM/cache) under the control of the processor.

- Memory has parts:

- Address
- Contents

- A program (actual code) is comprised of two main components:

- a. Data
- b. Instructions or code



- For data, such as **age** data stored in memory:
 - Contents can be accessed using an address or label.
 - It is more convenient for programmers to access memory labels.
 - Labels, in this context, are also referred to as variables.
 - A variable implies that the contents can change from one statement to next
- A **variable/label** comprises:
 - I. datatype (int, float, double, etc) [sizes....]
 - II. name (age/num1): Variable name should be in lower/camel case and descriptive
 - III. value/ data. (using assignment operator/input function// operator)

Example:

Declare an int variable "age" and assign it the value 19:

```
int age // variable declaration statement  
age = 19; // variable assignment statement
```

OR

```
int age = 19; // variable definition or initialization statement
```

Other Operators:

- Arithmetic Operators We also have arithmetic operators: +, -, *, /, %
- Relational operators >, >=, ==, etc
- Logical Operators &&, ||
- Bitwise Operators &

2. **Program's Building Blocks:** There are 3 building block in every Computer programming language:

Sequence

Execution of steps in a fixed order.

Examples:

- Input age in years → display in days
- Radius → calculate diameter, circumference, area

Conditionals

Decision making based on conditions.

Examples:

- Toss a coin: if heads, do the task; else skip
- Decimal→binary: if negative, convert then take 2's complement

Loops

Repeat steps until a condition is met.

Examples:

- Continue filling tank until full
- Repeatedly divide a number by 2 until it becomes zero

Bohm and Jacopini's theorem (1966): Any computer program can be written using only three control structures: sequence, selection (condition), and iteration(loops)

All programming problems can be solved using one or more of the following fundamental elements:

I. Sequence: Execution of instructions in a fixed, specified order.

- Example 1 (Real-world): Input your age in years and display it in days.
- Example 2 (Programming): Take the radius of a circle as input, then calculate and display its diameter, circumference, and area.
- In such cases, the solution relies purely on a sequential flow of steps.

II. Conditionals (Decision Making): Execution of instructions depends on certain conditions being true or false.

- Example 1 (Real-world): Toss a coin; if it lands on heads, perform the task; otherwise, skip it.
- Example 2 (Programming): In decimal-to-binary conversion, if a number is negative, convert it to binary and then take its 2's complement.

III. Loops (Iteration): Repetition of steps until a specific condition is met.

- Example 1 (Real-world): Keep filling a tank until it reaches full capacity.
- Example 2 (Programming): Repeatedly divide a number by 2 until it becomes zero.

.....
,,,,,,,,,,,,,,,,,,,,,,,,,,,,,

I. Sequence: A direct execution of steps in a specified order.

- a. **Examples:** For a task of taking your age in years as input and displaying it in days.

```
#include <iostream>
using namespace std;

int main()
{
    int age; // Variable to store age in years

    cout << "Enter your age: ";
    cin >> age; // Take age input from the user (in years)

    age = 365 * age; // Convert age from years to days

    cout << "Age in days = " << age; // Display the calculated age in days

    return 0; // End of program
}
```

- ii. A program four digits integer from user and shows the digits on the screen separately i.e. if user enters 7531, it displays 1, 3, 5, 7 separately

- Use of / and % operators
- integer division truncate the decimal part

```
#include <iostream>
using namespace std;

int main()
{
    // declare variables
```

```

int number, digit;

// prompt the user for input
cout << "Please enter 4-digit number:";
cin >> number;

// get the first digit and display it on screen

digit = number % 10;
cout << "The digits are: ";

cout << digit << ",";
// get the remaining three digits number
number = number / 10;
// get the next d
digit = number % 10;
cout << digit << ",";
// get the remaining two digits number
number = number / 10;
// get the next digit and display it
digit = number % 10;
cout << digit << ",";
// get the remaining one digit number
number = number / 10;
// get the next digit and display it
digit = number % 10;
cout << digit;

return 0; // End of program
}

```

- a. A program takes the radius of a circle from the user and calculates the diameter, circumference and area of the circle and displays the result.

```

int main()
{
    // declare variables
    float radius, diameter, circumference, area;
    // prompt the user for radius of a circle
    cout << "Please enter the radius of the circle ";
    cin >> radius;
    // calculate the diameter, circumference and area of the circle
    // implementing formula i.e. diameter = 2*r, circumference = 2 pi r and area =
    pi r2
    diameter = radius * 2;
    circumference = 2 * 3.14 * radius; // 3.14 is the value of π (Pi)
    area = 3.14 * radius * radius;
    // display the results
    cout << "The diameter of the circle is : " << diameter << endl;
    cout << "The circumference of the circle is : " << circumference << endl;
    cout << "The area of the circle is : " << area << endl;

    return 0; // End of program
}

```

- b. In these scenarios, we rely solely on a sequence of steps to accomplish the task.

II. Conditionals

- c. Real world example: Toss a coin; if it lands heads, proceed with the task; otherwise, do not.
- d. Example: [If a number is negative, convert it to its binary representation and then take its 2's complement.]

- **Conditional statements**

- **Motivations**
- **Syntax and example1: ageMarks**
- **Relational operators**
 - **Handling all cases**
 - **Example2: age comparison**
 - **Example 3: shopkeeper**
- **Logical operator**

1.

2. Conditional statements

a. Motivation

- o Program to take 3 test score of a student and if avg > 95 display congrats message

- Syntax / standards for algorithm

1. condition

```

If (condition)
{
    Statement ;
}

```

Example 1:

If (averHeight is greater than 6) => instead of English like statement use relational operators

```

{
    cout<< "Ali can be a member of basketball team"
}

```

If (aliHeight > 6) // no space in variable name, lower camel case

```

{
    cout << "Ali can be a member of basketball team"
}

```

```
}
```

Example 2

if

```
If (avgMarks > 95) // no space in variable name, lower camel case
{
    cout << "Ali can be a member of basketball team"
}
```

Detail Exmple 2

Sample Execution:

Please enter the score of first quiz: 100
Please enter the score of second quiz: 98
Please enter the score of third quiz: 95
Averge Score: 97
Excellent

Solution:

```
#include <iostream>
using namespace std;

int main() {
    int quiz1, quiz2, quiz3, total, avgMarks;

    cout << "Please enter the score of first quiz: ";
    cin >> quiz1;

    cout << "Please enter the score of second quiz: ";
    cin >> quiz2;

    cout << "Please enter the score of third quiz: ";
    cin >> quiz3;

    total = quiz1 + quiz2 + quiz3;
    avgMarks = total / 3;

    if (avgMarks > 95)
    {
        cout << "Excellent";
    }

    return 0;
```

}

- Example explanation

- integer division truncate the decimal part

- Relational operator `>` , other operators are `<`, `>=` , `<=`, `!=` , `==` are the binary operators

	Algebraic	Pseudo/Actual C++ Code	Example	Meaning
Greater than	$>$	<code>></code>	<code>x > y</code>	x is greater than y
Equal to	$=$	<code>==</code>	<code>x == y</code>	x is equal to y
Less than	$<$	<code><</code>	<code>x < y</code>	x is less than y
Greater than or equal to	$\geq$	<code>>=</code>	<code>x >= y</code>	x is greater than or equal to y
Less than or equal to	$\leq$	<code><=</code>	<code>x <= y</code>	x is less than or equal to y
Not equal to	$\neq$	<code>!=</code>	<code>x != y</code>	x is not equal to y

- Difference between `=` and `==` operators

- Multiple if statement vs if else if

Modify the above algorithm to handle if avgMarks greater than 95 and less than 95 cases.

i. Only If statements

```
double avgMarks = 90;

if (avgMarks > 95)
{
    cout << "Congratulation, you Won the prize";
}
if (avgMarks < 95)
{
    cout << "Good Luck for the next time.";
}
if (avgMarks == 95)
{
    cout << "Very Close....Good Luck for the next time.";
}
```

ii. If else structure

```
double avgMarks = 90;

if (avgMarks > 95)
{
    cout << "Congratulation, you Won the prize";
}
else if (avgMarks < 95)
{
    cout << "Good Luck for the next time.";
}
else
{
    cout << "Very Close....Good Luck for the next time.";
}
```

NOTE:

- In multiple if All conditions are checked independently, while **nested if-else** Stops at the first true condition
- Nested if-else is usually cheaper (faster) than multiple if for mutually exclusive conditions.

Programming Error for not handling all possible cases:

Algorithmic error or oversight in an "if" case could result in the loss of many lives. On April 2, 1982, a war unfolded **between Argentine and British forces** on the **Falkland Islands**. Argentina procured weapons from Western countries like France. When Argentina fired a missile at the British forces, the **expensive British antimissile system** identified the incoming missile but did not take any action due to the **friendly relations with France, the missile's manufacturer**. The failure to check **the missile's trajectory resulted in significant loss of life**, highlighting a programming error in the **trajectory-checking aspect of the antimissile system**.

Example 2: AGE COMPARISON

i) Using two separate if statements

```
#include <iostream>

using namespace std;
int main()
{
    double ahmadAge, moosaAge;

    cout << "Enter the age of Ahmad: ";
    cin >> ahmadAge;

    cout << "Enter the age of Moosa: ";
    cin >> moosaAge;

    // First condition
    if (ahmadAge >= moosaAge) {
        cout << "Ahmad is older than Moosa or both are of the same age." << endl;
    }
    // Second condition
    if (ahmadAge < moosaAge) {
        cout << "Ahmad is younger than Moosa." << endl;
    }

    return 0;
}
```

ii) Using if-else statement

```
#include <iostream>
using namespace std;
int main() {
    double ahmadAge, moosaAge;

    cout << "Enter the age of Ahmad: ";
    cin >> ahmadAge;

    cout << "Enter the age of Moosa: ";
    cin >> moosaAge;

    if (ahmadAge >= moosaAge) {
        cout << "Ahmad is older than Moosa or both are of the same age." << endl;
    }
    else { // No need to write condition again, else will cover ahmadAge < moosaAge
        cout << "Ahmad is younger than Moosa." << endl;
    }
}
```

```
}  
  
return 0;}
```

Use of LOGICAL operator: && and || and ! not operator

Use of LOGICAL Operators

There are many occasions when we need to combine more than one condition in a decision. For example, a person can be selected for a basketball team **only if both conditions are true:**
if (age > 18 && height > 6)

cout << "You can be a basketball team player.";

Here, the message is displayed only when **both conditions** are fulfilled. This means that the **AND (&&) operator** forces both conditions to be true.

Similarly, we may say that a person can be admitted to the university if he has a **BCS degree OR a BSc degree**. In this case, it is clear that the person will be admitted if **any one** of the two conditions is satisfied. This shows the use of the **OR (||) operator**.

In conclusion, logical operators are used to combine conditions in decision-making. The most common logical operators are:

- && (Logical AND)
- || (Logical OR)

The result of these operators depends on the **truth values of their operands** and can be illustrated using **truth tables**. Truth tables show all possible combinations of operand values and the corresponding result of applying && or ||

Logical Negation Operator

Another logical operator is called logical negation, represented by the symbol **!**. This operator allows a programmer to reverse the truth value of a condition. It is a unary operator, meaning it works on a single operand.

The **! operator** is placed before a condition. If the original condition (without **!**) evaluates to false, the **! operator** converts it to true, and the associated statements are executed.

Example:

if (!(age > 18))

cout << "The age is less than or equal to 18";

In this case, the cout statement will be executed only when the condition (age > 18) is false, because the **! operator** reverses the result.

The behavior of the logical negation operator **!** can be explained using the following truth table:

Please enter the amount of the bill 6500
The discount at the rate 15% is Rupees
The payable amount is Rupees 5525

Homework

1. Sequential statements

Instructions:

- Attempt the following tasks exactly in the given order.
- Indent your code properly.
- Use meaningful variable names. Use the **camelCase** notation to name variables.
- Use meaningful prompt lines/labels for all input/output that is performed by your algorithms.

You are required to write C++ code for each of the following tasks:

Task # 1

A company has determined that its annual profit is typically 13 percent of total sales. Design an algorithm that asks the user to enter the amount of total sales, and then determines and displays the annual profit that will be made from that amount.

Task # 2

Design an algorithm that will ask the user to enter the amount of a purchase. The algorithm should then compute the federal and provincial sales tax. Assume that the federal sales tax is 6 percent and the provincial sales tax is 4 percent. The algorithm should display the amount of the purchase, the federal sales tax, the provincial sales tax, the total sales tax, and the total of the sale (which is the sum of the amount of purchase plus the total sales tax).

Task # 3

A car's mileage in kilometers per liter (KMPL) can be calculated with the following formula:

$$KMPL = \text{Kilometers driven} / \text{Liters of petrol used}$$

Design an algorithm that asks the user for the number of kilometers driven and the liters of petrol used. It should calculate the car's KMPL and display the result on the screen.

Task # 4

Design an algorithm that asks the user how many eggs he/she has bought and then tells the user how many dozen eggs he/she has and how many extra eggs are left over. A sample run of your algorithm is given below. Text shown in **red** is entered by the user.

Sample run:

How many Eggs you bought: 42 You bought 3 dozen and 6 extra Eggs
--

Hint: Use the division operator (/) and the modulus (%) operator.

2. Conditional

Task # 5

A shopkeeper announces a package for customers that he will give 10 % discount on all bills and if a bill amount is greater than 5000 then a discount of 15 % (15.0/100 make sure about integer division). Write an algorithm which takes amount of the bill from user and calculates the payable amount by applying the above discount criteria and display it on the screen.

Sample Execution:

Please enter the amount of the bill 6500
The discount at the rate 15 % is Rupees 975
The payable amount is Rupees 5525

Task # 6

Task: Determine the largest number among four integers entered by the user.

Sample Execution:

Please enter the first number: 35
Please enter the second number: 15
Please enter the third number: 69
Please enter the fourth number: 50
The largest number is 69

Lecture 02: Repetition Structures (Loops)

1. Introduction to Loops

In daily life, many events repeat:

- Days and nights occur in cycles.
- Four seasons return every year.
- A water tank fills until it reaches full capacity.

Similarly, in programming, tasks often need to be repeated. Loops (repetition structures) help automate such tasks.

2. While Loop

Example: Basic While Loop:

```
int number = 0;

while (number < 10)
{
    cout << "Welcome to Programming Fundamentals\n";
    number = number + 1;
}
```

Example: Sum of First 1000 Numbers:

```
int sum = 0, number = 1;

while (number <= 1000)
{
    sum = sum + number;
    number = number + 1;
}

cout << "The sum of the first 1000 numbers is = " << sum;
```

3. Sum of Even & Odd Numbers (Example)

```
int upperLimit, number, evenSum, oddSum;

cout << "Enter the upper limit: ";
cin >> upperLimit;

number = 1;
evenSum = 0;
```

```

oddSum = 0;

while (number <= upperLimit) {
    if (number % 2 == 0)
    {
        evenSum = evenSum + number;
    }
    else {
        oddSum = oddSum + number;
    }
    number = number + 1;
}

cout << "Sum of even numbers = " << evenSum << endl;
cout << "Sum of odd numbers = " << oddSum << endl;

```

4. Do-While Loop

The do-while loop executes at least once.

Syntax:

```

do {
    // statements
} while (condition);

```

Example (do-while):

```

int x = 1;

do {
    cout << x << endl;
} while (x < 0);

```

5. For Loop

Used when the number of repetitions is known.

Syntax:

```

for (initialization; condition; increment) {
    // statements
}

```

Example: Multiplication Table:

```

int number, multiplier;

cout << "Enter a number: ";
cin >> number;
cout << "Enter the multiplier: ";
cin >> multiplier;

for (int counter = 1; counter <= multiplier; counter++) {
    cout << number << " x " << counter << " = " << number * counter <<
endl;
}

```

6. Operators in Loops

Increment & Decrement:

```

x++; // x = x + 1
y = ++x; // pre-increment
y = x++; // post-increment

```

Compound Assignment:

```

x += 5; // x = x + 5
x -= 3; // x = x - 3

```

7. Factorial Program (Added)

Factorial of a non-negative integer n ($n!$) is the product of all positive integers $\leq n$.

Examples: $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Below are two C++ implementations: iterative and recursive. Note that standard 64-bit unsigned long long can hold factorials up to $20!$ safely. For larger values, consider big-integer libraries such as Boost.Multiprecision.

Iterative version (recommended for large n):

```

#include <iostream>
using namespace std;

int main() {
    unsigned int n;
    cout << "Enter a non-negative integer: ";
    cin >> n;

    unsigned long long fact = 1ULL;
    for (unsigned int i = 1; i <= n; ++i) {
        fact *= i;
    }
}

```

```
    }  
  
    cout << n << "! = " << fact << endl;  
    return 0;  
}
```

8. Practice Problems

1. Sum of squares of first N numbers.
3. Display multiplication table using for loop.
4. Menu-driven program using do-while loop.

Functions:

- Need and benefit
 - Type of functions
 - Structure of a function: function definition, function call, function prototype
 - Examples: square, raiseToPow, isEven, areaOfCircle, home work
 - Function calling mechanism, Call by value and call by reference.
-
- C is a function-oriented language. Every program can be written in the form of different functions.
 - Top-down designing mechanism is based on the principle of 'divide and conquer' i.e. we divide a big task into smaller tasks and accomplish them.
 - No one can perform all of life's tasks personally.

Example1:

Making a laboratory stool: (consists of seat and 3 legs)

- Make a seat
- Make legs => use the same prototype to make 2 more legs
- Assemble a seat and three legs



Building a Lab Stool (Easy Way!)

Making a stool can be tricky. Here's a shortcut:

1. **Build 3 Identical Legs:** Saves time and keeps things even!

2. **Make the Seat:** Focus on the comfy part!
3. **Put it Together:** Legs + Seat = Stool!

This is like a "top-down" plan, where we break a big job into smaller, easier ones.

Example 2:

Running an organization is working in a function-oriented fashion.

Communication between Chairman and PCO working is just like a communication of a between functions

Motivation for structure a program in the form of functions:

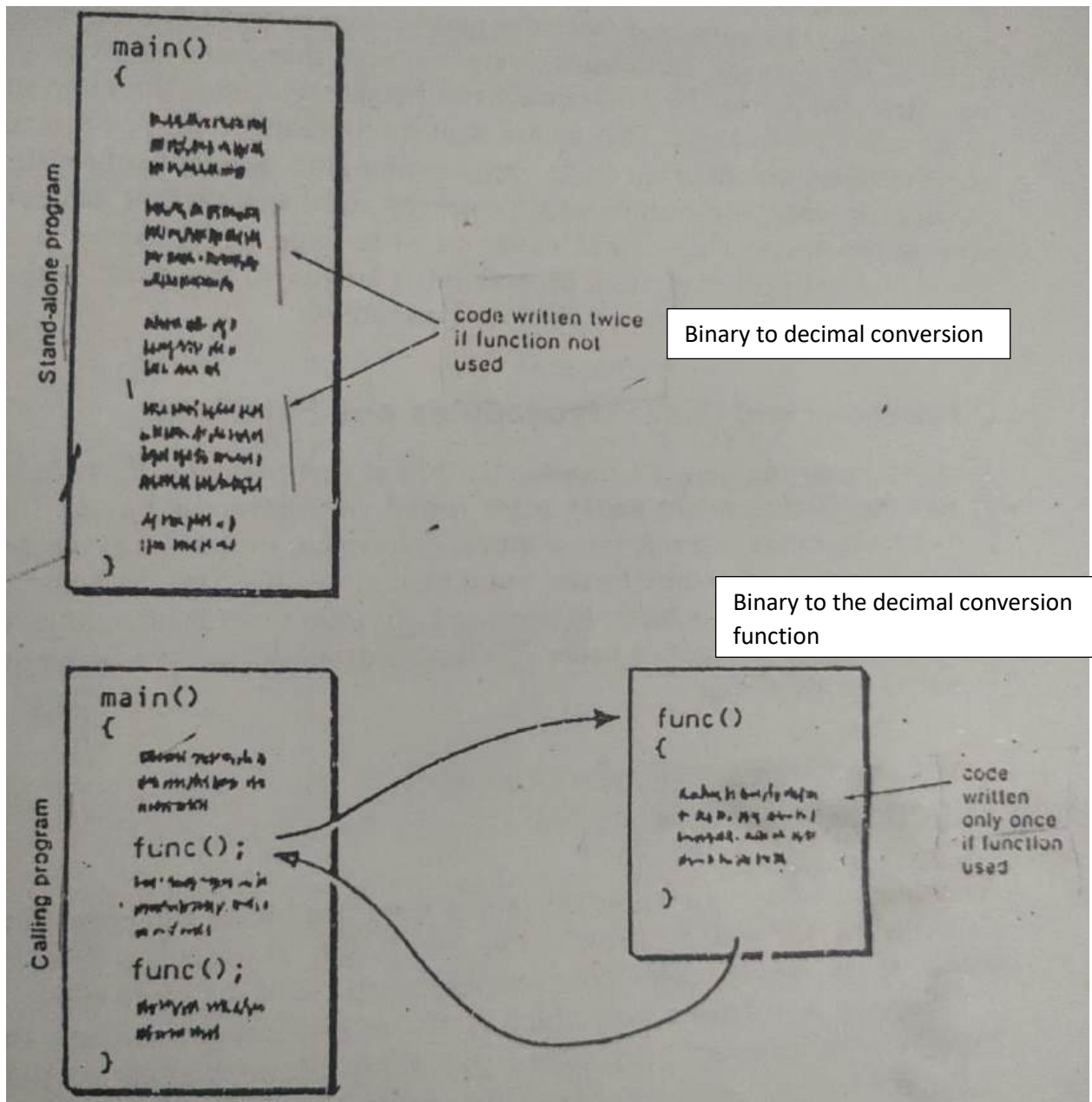


Figure 2.

Or

Figure 6-1 Modular programming

This program has one long, complex function containing all of the statements necessary to solve a problem.

[illegible]

In this program, the problem has been divided into smaller problems, each of which is handled by a separate function.

```
int main()
{
    statement;
    statement;
    statement;
}
```

main function

```
void function2()
{
    statement;
    statement;
    statement;
}
```

function 2

```
void function3()
{
    statement;
    statement;
    statement;
}
```

function 3

```
void function4()
{
    statement;
    statement;
    statement;
}
```

function 4

The benefit of making function:

1. Avoid unnecessary repetition of code => reusable see Figure 2
2. Make the program more readable, understandable, and easier to manage.
3. Independence: same variable name can be in different functions due to data hiding/ encapsulation

- ### 1. Structure of a function:

- a. Function definition // explain it with respect to main()
 - b. Function prototype
 - c. Function call
2. Function types:
- a. Datatype FunctionName (data type) // int square(int)
 - b. datatype FunctionName (void) // int getInput() from user
 - c. void FunctionName (data type)// void print(int) a specific value
 - d. void FunctionName (void) // print() a same message etc

Example 1 //calculate the integer power of a number(x^2)

```
int square(int n) // 1) function definition as we already define main()
{
    int result;
    result = n * n;
    return result;
}
```

```
main()
{
    result = square(number); // 2) function call
    result = 10 + square (5);
}
```

`int square(int);` // 3) function prototype, need when a function is defined in other file

// calculate the integer power of a number (x^2)

```
#include <iostream>
using namespace std;
int square(int);
```

```
int main()
{
    int number = 0;
    int result = 0;
    cout << "Please enter the number to calculate the square: ";
    cin >> number;
    result = square(number);
    cout << "The square of the " << number << " is " << result;
    return 0;
}
int square(int n)
{
    int result;
    result = n * n;

    return result;
}
```

SAMPLE Output:

Please enter the number to calculate the square: 23

The square of the 23 is 529

Function calling:

- Function call must be on the right-hand side of the assignment operators.
- A function call can be used with arithmetic operators if called function returns some number.

```
result = 2 * square(number);
result = 10 + square(number);
```

Example 2// calculate the integer power of a number (x^n)

Sample output:

Please enter the number: 3

Please enter the integer power that you want this number raise to: 3

3 raises to 3 is 27

reusability

```
#include <iostream>
```

```
using namespace std;
```

```
double raiseToPow(double, int); // prototype or function declaration
```

```
int main()
```

```
{
```

```
    double number = 0.0;
```

```
    int pow = 0;
```

```
    double result;
```

```
    cout << "Please enter the number: ";
```

```
    cin >> number;
```

```
    cout << "Please enter the integer power that you want this number rais to: ";
```

```
    cin >> pow;
```

```
    result = raiseToPow(number, pow);
```

```
    cout << number << " raise to " << pow << " is " << result;
```

```
    return 0;
```

```
}
```

```
double raiseToPow(double n, int p)
```

```
{
```

```
    double result = 1.0;
```

```
    for (int i = 1; i <= p; i++)
```

```
    {
```

```
        result = result * n;
```

```
    }
```

```
    return result;
```

```
}
```

Homework#1

// handle 0, positive, and negative cases:

```
double raiseToPow(double n, int p)
{
    double result = 1.0;
    if (p == 0)
    {
        result = 1.0;
    }
    else if (p < 0)
    {
        p = -1 * p;
        for (int i = 1; i <= p; i++)
        {
            result = result * n;
        }
        result = 1 / result;
    }
    else
    {
        for (int i = 1; i <= p; i++)
        {
            result = result * n;
        }
    }
    return result;
}
```

Sample output1:

Please enter the number: 2

Please enter the integer power that you want this number raise to: -2

2 raise to -2 is 0.25

Sample output2:

Please enter the number: 2

Please enter the integer power that you want this number rais to: 0

2 raise to 0 is 1

Sample output3:

Please enter the number: 3

Please enter the integer power that you want this number raise to: 3

3 raise to 3 is 27

// Reuse

Example 3 //calculate the area of a ring

Sample execution:

Please enter radius of outer circle: 5.5

Please enter radius of inner circle: 3.5

The area of ring having outer radius 5.5 and inner radius 3.5 is 56.5487

```
double aerofoiled(double);
int main()
{
    double rad1 = 0.0, rad2 = 0.0;
    double areaOfRing;
    cout << "Please enter radius of outer circle: ";
    cin >> rad1;

    cout << "Please enter radius of inner circle: ";
    cin >> rad2;

    areaOfRing = areaOfCircle(rad1) - areaOfCircle(rad2);
    cout << "The area of ring having outer radius " << rad1 << " and inner radius "
    << rad2 << " is " << areaOfRing;
    return 0;
}

double areaOfCircle(double r)
{
    double result = 1.0;

    return 3.1415926 * r * r;
}
```

Home work # 2

// put error checking

Sample output

Please enter radius of outer circle: 3

Please enter radius of inner circle: 5

Invalid inputs, as inner radius is larger than outer radius

Please enter again radius of outer circle: 5

Please enter radius of inner circle: 3

The area of ring having outer radius 5 and inner radius 3 is 50.2655

```

#include <iostream>
using namespace std;
double areaOfCircle(double);
int main()
{
    double rad1 = 0.0, rad2 = 0.0;
    double areaOfRing;
    cout << "Please enter radius of outer circle: ";
    cin >> rad1;

    cout << "Please enter radius of inner circle: ";
    cin >> rad2;
    while (rad2 > rad1)
    {
        cout << "Invalid inputs, as inner radius is larger than outer radius\n";
        cout << "Please enter again radius of outer circle: ";
        cin >> rad1;

        cout << "Please enter radius of inner circle: ";
        cin >> rad2;
    }

    areaOfRing = areaOfCircle(rad1) - areaOfCircle(rad2);
    cout << "The area of ring having outer radius " << rad1 << "and inner radius " <<
    rad2 << " is " << areaOfRing;
    return 0;
}

double areaOfCircle(double r)
{

    return (3.1415926 * r * r);
}

```

Example 4:// Is the number even or odd??

Sample execution:

Please enter a number: 8

The number is even

```
#include <iostream>
using namespace std;
int isEven(int num);
int main()
{
    int num;
    cout << "Please enter a number: ";
    cin >> num;
    if (isEven(num))
    {
        printf("The number is even ");
    }
    else
    {
        printf("The number is odd ");
    }

    return 0;
}

int isEven(int num)
{
    if (num % 2 == 0)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}
```

Homeworks:

1. Modify the raise to power function so that it can handle negative power of x, zero and positive power of x.
2. Modify the area of ring function put in error checking mechanism.

Tips

- We used functions for breaking complex problems into smaller pieces, which is a top-down structured approach.
- Each function should be a small module, self-contained. It should solve a well defined problem.
- Variable names and function names should be self-explanatory.
- Always comment the code.

Call by reference. The original variable is changed.

```
#include <iostream>
using namespace std;

void square(int *);

int main()
{
    int i = 10;
    cout << "In main(), the value of i is " << i << endl;
    square(&i);
    cout << "Back in main(), the value of i is " << i << endl;
    return 0;
}

void square(int *i)
{
    *i *= *i;
    printf("In square(int *i), the value of i is %d\n", *i);
}
```

OR

```
#include <iostream>
using namespace std;

void square(int &i);

int main()
{
    int i = 10;
    cout << "In main(), the value of i is " << i << endl;
    square(i);
    cout << "Back in main(), the value of i is " << i << endl;
}
```

```

        return 0;
    }

    void square(int &i)
    {
        i *= i;
        printf("In square(int), the value of i is %d\n", i);
    }

```

output

In main(), the value of i is 10

In square(int *i), the value of i is 100

Back in main(), the value of i is 100

Homework:

// sample functions that are called using reference by reference mechanism.

Task 1:

Write a function named **convertTime()** that has an integer parameter named seconds and

three integer reference parameters named hours, mins, and secs. The function is to convert the passed number of seconds into an equivalent number of hours, minutes, and seconds. Using the reference parameters, the function should alter the arguments in the calling function. Also, write a driver main function to test the **convertTime()** function.

Below is the prototype of a function that converts time from totalSeconds to hours, minutes, and seconds:

void convertTime(int totalSeconds, int* hours, int* min, int* sec);

Please provide the definition of the **convertTime()** function and its function call from main()

Task 2:

12. Star Search

A particular talent competition has five judges, each of whom awards a score between 0 and 10 to each performer. Fractional scores, such as 8.3, are allowed. A performer's final score is determined by dropping the highest and lowest score received, then averaging the three remaining scores. Write a program that uses this method to calculate a contestant's score. It should include the following functions:

- `void getJudgeData()` should ask the user for a judge's score, store it in a reference parameter variable, and validate it. This function should be called by `main` once for each of the five judges.
- `void calcScore()` should calculate and display the average of the three scores that remain after dropping the highest and lowest scores the performer received. This function should be called just once by `main` and should be passed the five scores.

The last two functions, described below, should be called by `calcScore`, which uses the returned information to determine which of the scores to drop.

- `int findLowest()` should find and return the lowest of the five scores passed to it.
- `int findHighest()` should find and return the highest of the five scores passed to it.

Input Validation: Do not accept judge scores lower than 0 or higher than 10.

You may consider the prototype of a function `void getJudgeData()` like
`void getJudgeData(double* score, int i);`

Solutions: // sample functions that are called using reference by reference mechanism.

Nested loop:

- Average test score
- Program asks for the no of students and the number of test scores per student.

Program Output with Example Input Shown in Bold

This program averages test scores.

For how many students do you have scores? **2**

How many test scores does each student have? **3**

Enter score 1 for student 1: **84**

Enter score 2 for student 1: **79**

Enter score 3 for student 1: **97**

The average score for student 1 is 86.7.

Enter score 1 for student 2: **92**

Enter score 2 for student 2: **88**

Enter score 3 for student 2: **94**

The average score for student 2 is 91.3.

```
int main()
{
    cout << "This program average the test score \n";
    int noOfStdts = 0;
    int noOfTests = 0;
    double score = 0.0;
    double total;
    double average = 0.0;
    cout << "For how many student do you have scores: ";
    cin >> noOfStdts;
    cout << "How many test scores does each student have: ";
    cin >> noOfTests;

    for (int i = 0; i < noOfStdts; i++)
    {
        total = 0;
        for (int j = 0; j < noOfTests; j++)
        {
            cout << "please enter score "<< j + 1 << " for student " << i + 1
            <<": ";

            cin >> score;
            total += score;
        }
        average = total / noOfTests;
        cout << "Average score of student " << i + 1 << " is " << average << endl;
    }
}
```

```

    return 0;
}

```

Homework 2:

3.32 (*Square of Asterisks*) Write a program that reads in the side of a square and then prints that square out of asterisks. Your program should work for squares of all side sizes between 1 and 20. For example, if your program reads a size of 4, it should print

```

****
****
****
****

```

Homework 3:

3.33 (*Hollow Square of Asterisks*) Modify the program you wrote in the preceding exercise so that it prints a hollow square. For example, if your program reads a size of 5, it should print

```

*****
*   *
*   *
*   *
*   *
*****

```

4.31 (*Diamond-Printing Program*) Write a program that prints the following diamond shape. Your printf statements may print either one asterisk (*) or one blank. Use nested for statements and minimize the number of printf statements.

```

  *
 ***
*****
*****
*****
*****
*****
  ***
   *

```

Homework 4: simulate clock complete

The output of the previous program segment follows:

```

00:00:00
00:00:01
00:00:02

```

(The program will count through each second of 24 hours.)

```

23:59:59

```

To make the simulated clock complete, another variable and loop can be added to count the hours.

```
cout << fixed << right;
cout.fill('0');
for (int hours = 0; hours < 24; hours++)
{
    for (int minutes = 0; minutes < 60; minutes++)
    {
        for (int seconds = 0; seconds < 60; seconds++)
        {
            cout << setw(2) << hours << ":";
            cout << setw(2) << minutes << ":";
            cout << setw(2) << seconds << endl;
        }
    }
}
```

The output of the previous program segment follows:

```
00:00:00
00:00:01
00:00:02
```

(The program will count through each second of 24 hours.)

```
23:59:59
```

The innermost loop will iterate 60 times for each iteration of the middle loop. The middle loop will iterate 60 times for each iteration of the outermost loop. When the outermost loop has iterated 24 times, the middle loop will have iterated 1,440 times and the innermost loop will have iterated 86,400 times!

The simulated clock example brings up a few points about nested loops:

- An inner loop goes through all of its iterations for each iteration of an outer loop.
- Inner loops complete their iterations faster than outer loops.
- To get the total number of iterations of a nested loop, multiply the number of iterations of all the loops.

Lifetime and scope/visibility of Global, Static, and local variables

```
int globalVar;
void showLocal();
void showStatic();

int main()
{
    cout << "GlobalVar:" << globalVar << endl;

    cout << "showLocal() local variable values " << endl;
    for (int i = 0; i < 5; i++)
    {
        showLocal();
    }
    cout << "showStatic() static variable values :\n";

    for (int i = 0; i < 5; i++)
    {
        showStatic();
    }

    return 0;
}

void showLocal()
{
    int localNum = 0; // local variable life and visibility only within function
    cout << localNum << endl;
    localNum++;
}

void showStatic()
{
    static int statNum = 0; // static local variable// life like global variable and
    // visibility like local variable
    cout << statNum << endl;
    statNum++;
}
```

Ouput:

GlobalVar: 0

showLocal() local variable values :

0

0

0

0

0

showStatic() static variable values :

0

1

2

3

4

Recursive function:

1. Elegant code as it matches with the recursive patterns
2. Whereas the Iterative code is efficient

$$5! = 5 * 4 * 3 * 2 * 1$$

$$5! = 5 * 4!$$

$$4! = 4 * 3! \quad //1. \text{ calling again and again}$$

$$3! = 3 * 2!$$

$$2! = 2 * 1!$$

$$1! = 1 * 0!$$

$$0! = 1 \quad //2. \text{ base case}$$

$n! = n * (n-1)!$ // is the general formula

```
long NonRecursivefactorial(int num)
{
    int fact = 1;
    for (int i = num; i > 0; i--)
    {
        fact = fact * i;
    }
    return fact;
}

long factorial(int num)
{
    if (num == 0)
        return 1;
    else
        return num * factorial(num - 1);
}

int main()
{
    cout << NonRecursivefactorial(5) << endl;
    cout << factorial(5) << endl;;
}
```

24

fact(4) temp=6
res=4x6=24

Tree
representation

6

fact(3) temp=2
res=3x2=6

2

fact(2) temp=1
res=2

1

fact(1) temp=1
res=1

1

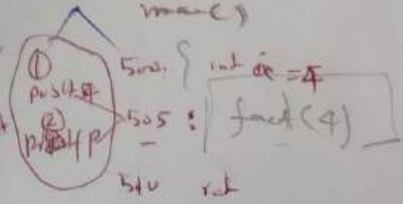
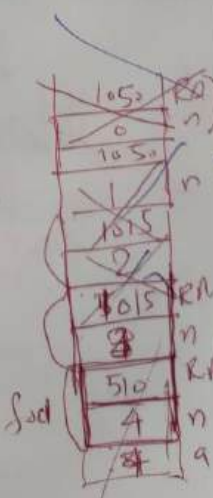
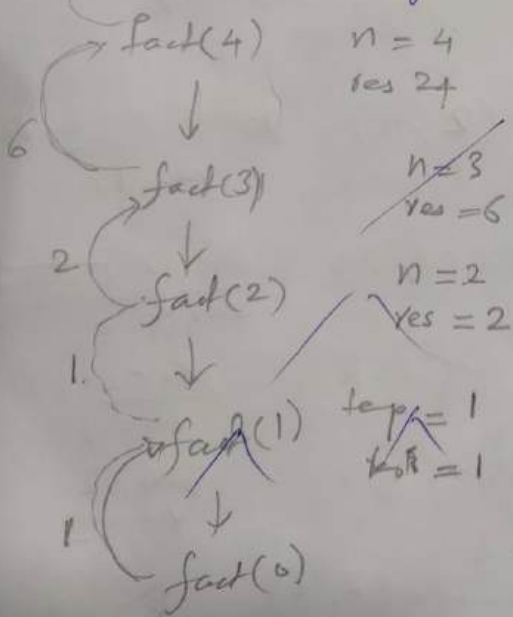
fact(0)

fact(0)

long fact(int n)

if (n==0)
return 1
else

return n * fact(n-1)

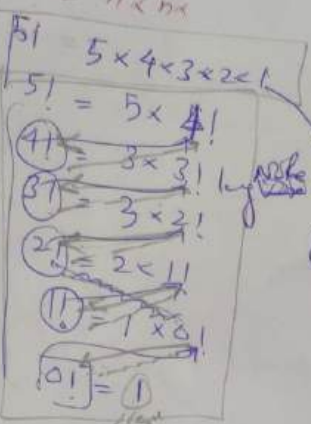


→ push pointer
→ push ~~fact~~ RA (IP)

①

① $n! = n \times n \times \dots$

factorial



③

$n \times (n-1)!$

include

②

```
long fact (int n)
{
    fact = 1;
    for (i = n; i > 0; i--)
    {
        fact = fact * i;
    }
    return fact;
}
```

reb. flr

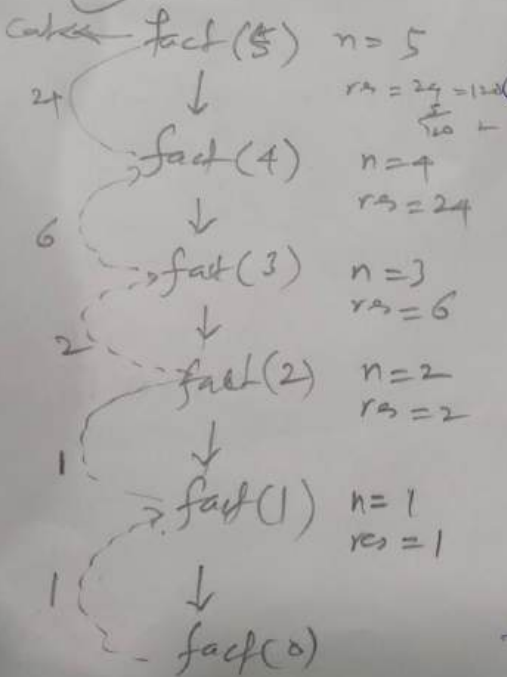
Efficient
no stack
overflow

fact(5)

long fact (int n)

```
if (n == 0) { return 1; }
return n * fact(n-1);
```

⑤



if (n == 0)

return 1;
 -ve
 elegant

else

```
return n * fact(n-1);
```

```
main() { int n = 5; fact(n); }
```

Homework2

- Write an implementation of **raiseToPow()** recursively

$$X^5 = X * X * X * X * X$$

$$X^5 = X * X^4$$

$$X^4 = X * X^3$$

$$X^3 = X * X^2$$

$$X^2 = X * X^1$$

$$X^1 = X * X^0$$

$$X^0 = 1$$

$$X^n = X * X^{n-1}$$

```
/*
double raisToPow(double n, int p)
{
    double res = 1.0;
    for(int i = p; i>0; i--)
    {
        res = res * n;
    }
    return res;
}
*/
//recursive implementation
double raisToPow(double n, int p)
{
    // HOME WORK 2... PROVIDE ITS IMPLEMENTATION
}
```

- Write the implementation of **void displayHello(int i)** that prints the **hello** message 10 times recursively.

```
// HOME WORK 2... PROVIDE ITS IMPLEMENTATION
```

```
void displayHello(int i)
{

}

int main()
{
    displayHello(1);
}
```

```

        return 0;
    }

```

Output sample:

```

1 Hello world
2 Hello world
3 Hello world
4 Hello world
5 Hello world
6 Hello world
7 Hello world
8 Hello world
9 Hello world
10 Hello world

```

HOMEWORK 3: Here is the non recursive implementation of Fibonacci series generation function. You need to provide its recursive implementation

```

int fibonacci(int n) {
    int a = 0, b = 1, c;
    if (n == 0)
        return a;
    for (int i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
    return b;
}

int main() {
    int n;
    cout << "Enter the number of terms for Fibonacci series: ";
    cin >> n;

    cout << "Fibonacci series up to " << n << " terms:" << endl;
    for (int i = 0; i < n; i++) {
        cout << fibonacci(i) << " ";
    }
    cout << endl;

    return 0;
}

```

Sample output:

```

Fibonacci series up to 10 terms:
0 1 1 2 3 5 8 13 21 34

```

NOTE: for all three task, you need to provide the trace the recursive call using both tree graph/tree structure and also using Stack

Today lecture: Arrays

Motivation of arrays

Initialization of Arrays

Sample Programs

- Average age of students
- Count the number of positive integers
- Sum of the square of array elements
- Copying
- Linear search

Use of #define SIZE 100

Use of rand() and srand(time(NULL))

Function with arrays

Home work

Motivation of arrays:

Let us consider an example about calculation of average age of 10 students. At first, we will declare 10 variables to store the age of each student and then sum up all the ages and divide this with 10 to get the average age. Suppose, we have 100 students instead of 10, we have to declare 100 variables i.e. one for each student's age. Is there any other way to deal with this problem? Arrays are possible solution to the problem.

Arrays:

Array is a special data-type. If we have a collection of data of same type as in the case of storage of ages of 100 students, arrays can be used. Arrays are data structure in which identical data types are stored. The concept of arrays is being explained further in the following parts of the lecture.

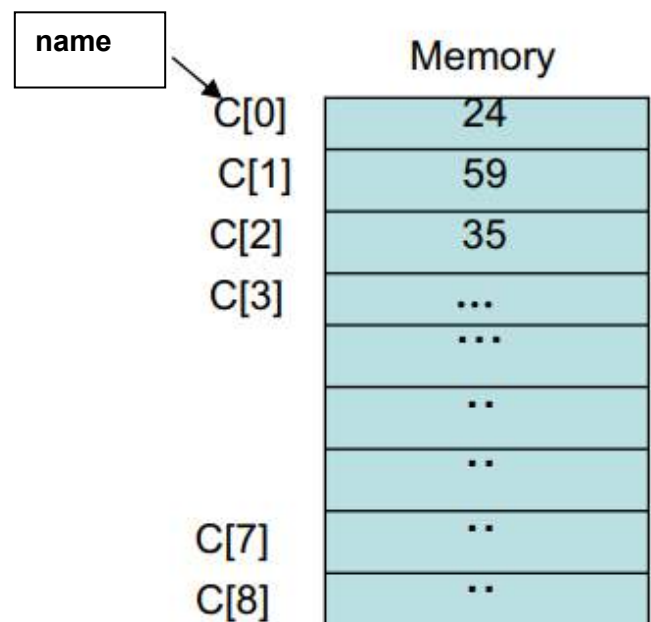
- **Arrays are like the data structure in which identical data types are stored.**
- **An array has a name, data type, and size**
- **They occupy the consecutive area of memory**
- **Can we save height and weight information in same array? (context should be same)**
-

Syntax:

```
datatype arrayName [size];
```

```
int age [10];
```

```
int C [10];
```



Usage of array

```
// initialization of array  
int age[10] = {0,0,0,0,0,0,0,0,0,0};  
OR
```

```
int age[10] = {0};
```

OR

```
for (int i = 0; i < 10; i++)  
{  
    age[i] = 0;  
}
```

Example 1: // find the average age of 10 students

```
int main()  
{  
    int age[10] = {0};  
  
    // initialization of array  
  
    for (int i = 0; i < 10; i++)  
    {  
        age[i] = 0;  
    }  
    // find the average age of 10 students  
    int sum = 0;  
    for (int i = 0; i < 10; i++)  
    {  
        cout << "Enter the age of student: " << i + 1;  
        cin >> age[i];  
        sum = sum + age[i];  
    }  
    cout << "Average age is << sum / 10;  
    return 0;  
}
```

Example 2: // Count the positive number read from the user until -1 is entered

Write a program that reads positive integers from the user and stores these ones in an array. Users can enter a maximum of 100 numbers. Stop taking input when a user enters -1.

```
int main()
{
    int a[100] = {0}, i = 0, input = 0;

    do
    {
        cout << "Entered the positive number (enter -1 for end of input):";
        cin >> input;
        if (input != -1)
        {
            a[i] = input;
            i++;
        }
        //i++;
    } while (input != -1 && i < 100);

    cout << "Total positive numbers are : " << i;
    return 0;
}
```

Example 3: // copy the arrays

Both arrays must have same datatype and size

```
int main()
{
    int a[100] = {0};
    int b[100];

    for (int i = 0; i < 100; i++)
    {
        b[i] = a[i];
    }

    cout << "Element at b[5] = " << b[5];
    return 0;
}
```

```
b[0] = a[0] ;
b[1] = a[1] ;
b[2] = a[2] ;
.....
.....
.....
b[9] = a[9] ;
```

Example 4:

// Take the sum of squares of 10 different numbers stored in an array.

```
int main()
{
    int a[10], sumOfSquare = 0;
    // initialize the array
    for (int i = 0; i < 10; i++)
    {
        a[i] = i; // take it from user
    }

    for (int i = 0; i < 10; i++)
    {
        sumOfSquare += a[i] * a[i];
    }

    cout << "The sum of the square of array elements are: " << sumOfSquare;
    return 0;
}
```

Example 4: use constant SIZE to make it easier to maintain the program

```
//#define SIZE 10
const int SIZE = 10;
int main()
{
    int arr[SIZE]
    for (i = 0; i < SIZE; i++)
    {
        arr[i] = i;
    }

    return 0;
}
```

Example 5: Linear search (Guess a number)

```
#define SIZE 100

int main()
{

    int arr[SIZE];
    int num=0, flag = 0, i = 0;

    //populate the array
    for (i = 0; i < SIZE; i++)
    {
        arr[i] = i
        //cin >>>arrp[i];
    }

    for (i = 0; i < SIZE; i++)
    {
        cout << arr[i];
    }

    //display the contents of populated array

    cout <<"\nPlease enter the positive number ";
    cin >> num;

    for (i = 0; i < SIZE; i++) // linear search
    {
        if (arr[i] == num)
        {
            flag = 1;
            break;
        }
    }
    if (flag == 1)
    {
        cout <<"Congratulations!, we found the integer at " << i;
    }
    else
    {
        cout << "Sorry number is not found!";
    }

    return 0;
}
```

Example 6: Use the random function

- The function is *rand()* and is in the standard library.
- To access this function, we need to include <cstdlib> library in our program.
- This function will return a random number.

The number can be between 0 and 32767.

```
#include <iostream>
#include<cstdlib> // for rand() and srand()
#include<ctime>    // for time(0)

#define SIZE 10

using namespace std;

int main()
{
    srand(time(0)); // act as seed for rand() to make it real random number generator,
    //every time different random numbers will be generated.

    int arr[SIZE];
    int num = 0, flag = 0, i = 0;

    for (i = 0; i < SIZE; i++)
    {
        arr[i] = rand(); // populate the array with numbers between 0-> 32767
        //a[i] = rand() % 100; // populate the array with numbers between 0-> 99
    }

    for (i = 0; i < SIZE; i++)
    {
        cout << arr[i] <<" ";
    }

    cout << "\nPlease enter the positive number ";
    cin >> num;

    for (i = 0; i < SIZE; i++) // linear search
    {
        if (arr[i] == num)
        {
            flag = 1;
            break;
        }
    }
    if (flag == 1)
    {
        cout << "Congratulations!, we found the integer at " << i;
    }
    else
    {
        cout << "Sorry number is not found!";
    }
}
```

```
return 0;}
```

What is the output of the following expression:

rand() % 6

1+ rand() % 6 => Use to make fair dice

rand() % 2

1+ rand() % 2 = > Use to make fair coin

Use to make fair dice, coin

- Make the cricket match simulation.

Tips

Initialize the array explicitly

Array index (subscript) starts from 0 and ends one less than the array size

To copy an array, the size and data type of both arrays should be same

Array subscript may be an integer or an integer expression

Example 7 : character array and value of \0

```
int main()
{
    char name[] = { 'a', 'b', 'c', '\0' };
    char name1[] = "abc";
    printf("%s", name);
    printf("\n%s", name1);
    printf("\nprint character array using loop \n" );
    int i = 0;
    while (name1[i] != '\0')
    {
        printf("%c", name1[i]);
        i++;
    }
}
```

```
        return 0;
    }
```

Example 11: function and array

When an array name is passed to a function it is called by a reference mechanism, whereas simple array elements whenever are passed to a function it is implemented as call by value.

```
void fun1(int[], int);
int main()
{
    int array[10];
    fun1(array, 10);
    for (int i = 0; i < 10; i++)
    {
        cout << array[i];
    }
    return 0;
}
void fun1(int a[], int s)
{
    for (int i = 0; i < s; i++)
    {
        a[i] = i;
    }
}
```

Homework: define the functions that perform the following operation.

- Reverse the array elements from arrayA and store them in arrayB
- Comparing the two arrays (identical or not)
- Sort the array elements in ascending and descending order

Motivation of arrays:

Let us consider an example about calculation of average age of 10 students. At first, we will declare 10 variables to store the age of each student and then sum up all the ages and divide this with 10 to get the average age. Suppose, we have 100 students instead of 10, we have to declare 100 variables i.e. one for each student's age. Is there any other way to deal with this problem? Arrays are possible solution to the problem.

Arrays:

Array is a special data-type. If we have a collection of data of same type as in the case of storage of ages of 100 students, arrays can be used. Arrays are data structure in which identical data types are stored. The concept of arrays is being explained further in the following parts of the lecture.

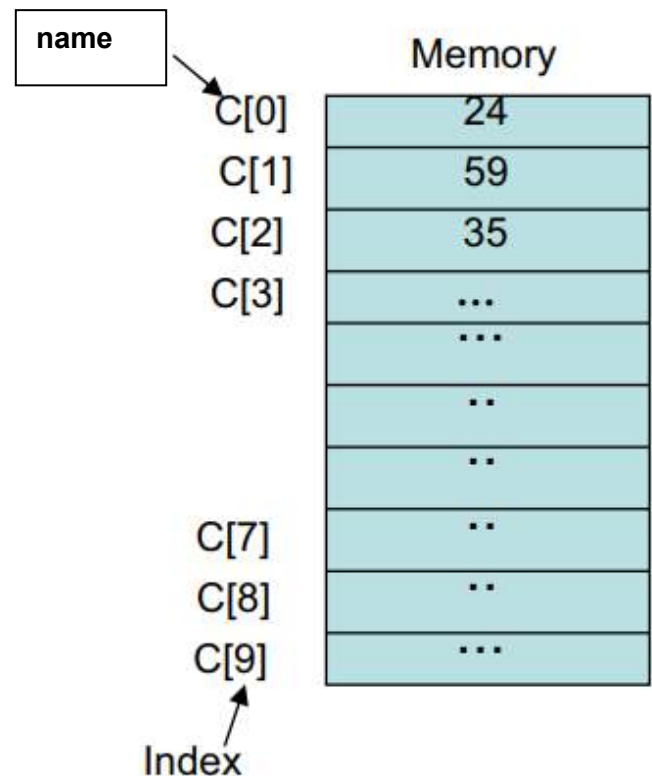
- **Arrays are like the data structure in which identical data types are stored.**
- **An array has a name, data type, and size**
- **They occupy the consecutive area of memory**
- **Can we save height and weight information in same array? (context should be same)**
-

Syntax:

```
datatype arrayName [size];  
int age [10];  
int C [10];
```

Usage of array

```
// initialization of array  
int age[10] = {0,0,0,0,0,0,0,0,0,0};  
OR
```



```
int age[10] = {0};
```

OR

```
for (int i = 0; i < 10; i++)  
{  
    age[i] = 0;  
}
```

Example 1: // find the average age of 10 students

```
int main()  
{  
    int age[10] = {0};  
  
    // initialization of array  
  
    for (int i = 0; i < 10; i++)  
    {  
        age[i] = 0;  
    }  
    // find the average age of 10 students  
    int sum = 0;  
    for (int i = 0; i < 10; i++)  
    {  
        cout << "Enter the age of student: " << i + 1;  
        cin >> age[i];  
        sum = sum + age[i];  
    }  
    cout << "Average age is << sum / 10;  
    return 0;  
}
```

Example 2: // Count the positive number read from the user until -1 is entered

Write a program that reads positive integers from the user and stores these ones in an array. Users can enter a maximum of 100 numbers. Stop taking input when a user enters -1.

```

int main()
{
    int a[100] = {0}, i = 0, input = 0;

    do
    {
        cout << "Entered the positive number (enter -1 for end of input):";
        cin >> input;
        if (input != -1)
        {
            a[i] = input;
            i++;
        }
        //i++;
    } while (input != -1 && i < 100);

    cout<<"Total positive numbers are : " << i;
    return 0;
}

```

Example 3: // copy the arrays

Both arrays must have same datatype and size

```

int main()
{
    int a[100] = {0};
    int b[100];

    for (int i = 0; i < 100; i++)
    {
        b[i] = a[i];
    }

    cout << "Element at b[5] = " << b[5];
    return 0;
}

```

```

b[0] = a[0] ;
b[1] = a[1] ;
b[2] = a[2] ;
.....
.....
.....
b[9] = a[9] ;

```

Example 4:

// Take the sum of squares of 10 different numbers stored in an array.

```

int main()
{
    int a[10], sumOfSquare = 0;
    // initialize the array

```

```

    for (int i = 0; i < 10; i++)
    {
        a[i] = i; // take it from user
    }

    for (int i = 0; i < 10; i++)
    {
        sumOfSquare += a[i] * a[i];
    }

    cout <<"The sum of the square of array elements are: " << sumOfSquare;
    return 0;
}

```

Example 4: use constant SIZE to make it easier to maintain the program

```

// #define SIZE 10
const int SIZE = 10;
int main()
{
    int arr[SIZE]
    for (i = 0; i < SIZE; i++)
    {
        arr[i] = i;
    }

    return 0;
}

```

Example 5: Linear search (Guess a number)

```

#define SIZE 100

int main()
{
    int arr[SIZE];
}

```

```

    int num=0, flag = 0, i = 0;

//populate the array
    for (i = 0; i < SIZE; i++)
    {
        arr[i] = i
        //cin >>>arrp[i];
    }

    for (i = 0; i < SIZE; i++)
    {
        cout << arr[i];
    }

//display the contents of populated array

    cout <<"\nPlease enter the positive number ";
    cin >> num;

    for (i = 0; i < SIZE; i++) // linear search
    {
        if (arr[i] == num)
        {
            flag = 1;
            break;
        }
    }
    if (flag == 1)
    {
        cout <<"Congratulations!, we found the integer at " << i;
    }
    else
    {
        cout << "Sorry number is not found!";
    }

    return 0;
}

```

Example 6: Use the random function

- The function is **rand()** and is in the standard library.
- To access this function, we need to include <cstdlib> library in our program.
- This function will return a random number.
The number can be **between 0 and 32767**.

```

#include <iostream>
#include<cstdlib> // for rand() and srand()
#include<ctime>    // for time(0)

```

```

#define SIZE 10

using namespace std;

int main()
{
    srand(time(0)); // act as seed for rand() to make it real random number generator,
    //every time different random numbers will be generated.

    int arr[SIZE];
    int num = 0, flag = 0, i = 0;

    for (i = 0; i < SIZE; i++)
    {
        arr[i] = rand(); // populate the array with numbers between 0-> 32767
        //a[i] = rand() % 100; // populate the array with numbers between 0-> 99
    }

    for (i = 0; i < SIZE; i++)
    {
        cout << arr[i] << " ";
    }

    cout << "\nPlease enter the positive number ";
    cin >> num;

    for (i = 0; i < SIZE; i++) // linear search
    {
        if (arr[i] == num)
        {
            flag = 1;
            break;
        }
    }

    if (flag == 1)
    {
        cout << "Congratulations!, we found the integer at " << i;
    }
    else
    {
        cout << "Sorry number is not found!";
    }

    return 0;}

```

What is the output of the following expression:

rand() % 6

1+ rand() % 6 => Use to make fair dice

rand() % 2

1+ rand() % 2 => Use to make fair coin

Use to make fair dice, coin

- Make the cricket match simulation.

Tips

Initialize the array explicitly

Array index (subscript) starts from 0 and ends one less than the array size

To copy an array, the size and data type of both arrays should be same

Array subscript may be an integer or an integer expression

Homework 1:

- Reverse the array elements from arrayA and store them in arrayB
- Comparing the two arrays (identical or not)
- Sort the array elements in ascending and descending order

;;;;;;;;;;;;;;;;;

Example 10: function and array

When an array name is passed to a function it is called by a reference mechanism, whereas simple array elements whenever are passed to a function it is implemented as call by value.

```
void getInput(int[], int);  
int main()  
{
```

```

    int array[10];
    getInput(array, 10);
    for (int i = 0; i < 10; i++)
    {
        cout << array[i] << " ";
    }
    return 0;
}
void getInput(int a[], int s)
{
    for (int i = 0; i < s; i++)
    {
        a[i] = i;
    }
}

```

Tips

Initialize the array explicitly

Array index (subscript) starts from 0 and ends one less than the array size

To copy an array, the size and data type of both arrays should be same

Array subscript may be an integer or an integer expression

Example 11: function and array

When an array name is passed to a function it is called by a reference mechanism, whereas simple array elements whenever are passed to a function it is implemented as call by value.

```

void fun1(int[], int);
int main()
{
    int array[10];
    fun1(array, 10);
    for (int i = 0; i < 10; i++)
    {
        cout << array[i];
    }
    return 0;
}
void fun1(int a[], int s)
{
    for (int i = 0; i < s; i++)
    {
        a[i] = i;
    }
}

```

```
}
```

Homework2:

- Void sortArray(int arr, int size);
- void mergeArray(int m[], int sm, int a1[], int s1, int a2[], int s2);
- int findSmallest(int a[], int size);
- int isSorted(int a[], int size);

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
```

Lecture 06

Character Arrays

To store and enter a name in a program, we typically use a character array, for example:

```
char name[100];
```

Character arrays have a special property — they behave differently from other arrays. Unlike an integer or float array, a character array (used as a string) automatically places a special marker called the null character ('\0') at the end of the text.

This null character indicates where the string ends. For instance, if your name is only 15 characters long but the array size is 100, the computer won't treat all 100 elements as a part of the name. Instead, it will stop at the null character, making it easier to manipulate and display strings.

The size of character array is 1 larger than the number of character you want to store in it.

Initialization of character arrays

```
char name[20] = "ahmad";
```

```
char name[20] = { 'a','h','m','a','d','\0' }
```

```
include<iostream>
```

```
using namespace std;
```

```
void input(char src1[], int size)
```

```
{
    int i = 0;
    printf("Please enter your name: ");
    char ch = getchar();
    while (ch != 10)
    {
        src1[i] = ch;
        ch = getchar();
        i++;
    } // cin.getline(src), read complete name.
    src1[i] = '\0';
}
```

```
void display(char src1[], int size)
```

```
{
    int i = 0;
    while (src1[i] != '\0')
    {
        //printf("%c", src1[i]);
        cout << src1[i];
        i++;
    }
}
```

```
}
```

```
int main()
```

```
{
    const int SIZE = 100;
    char name[SIZE];
    input(name, SIZE);

    cout << "Inside the main()";
    cout << name;
    cout << "calling display " << endl;

    display(name, SIZE);
    return 0;
}
```

Example 7 : character array and value of \0

```
int main()
{
    char name[] = { 'a', 'b', 'c', '\0' };
    char name1[] = "abc";
    printf("%s", name);
    printf("\n%s", name1);
    cin.getline(name1);
    printf("\nprint charachter array using loop \n" );
    int i = 0;
    while (name1[i] != '\0')
    {
        //printf("%c", name1[i]);
        cout << name1[i];
        i++;
    }

    return 0;
}
```

Example 7 : Comparison of arrays (Note in case of character arrays upper and lower case array of same characters are different ; ali != ALI)

```
int main()
{
    int main()
    {
        char name[] = { 'a', 'b', 'c', '\0' };
        char name1[] = "abc";
        int i = 0;
        int flag = 1;
        while (name1[i] != '\0')
        {
            if (name[i] != name1[i])
            {
                flag = 0;
                break;
            }
            i++;
        }
        if (flag == 1)
        {
            printf("Both arrays are equal");
        }
        else
        {
            printf("Both arrays are not equal ");
        }
    }
}
```

```
    return 0;  
}
```

Example 7: Display the characters array in reverse order

Example 8: determine the length of the character array

Homework 3: could be Lab tasks:

1. Print the name using a loop and without a loop
2. Write a function that compares two-character array
3. Write a function that Copies a character array into another character array
4. Find a character from a character array.
5. Write a function that reverses character arrays.

```
void reverse ( char [], int );
```

```
void reverse ( char ch [], int arraySize ) { // The body of the function. }
```

6. void toUpper(char ch[], int size);
7. void toLower(char ch[], int size);
8. int merge(char m[], char ch1,char ch2[],int m,int c);

Multidimensional arrays:

Multidimensional arrays:

- Vectors manipulation is done using a one-dimensional array
- Matrix in mathematics is represented by 2D arrays in C
 - example images etc
- Syntax : use two index to access the 2d array
 - **int a[row][cols];**
 - memory view

- Nested loop is used to access every element using the following syntax
 - `a[i][j]`
- Function and 2 arrays:
 - for the 2d array mentioned the cols size is important for compiler
 - In **C / C++**, when you pass a **2D array** to a function, the compiler needs to know how many **columns** each row has (the "stride") so it can correctly compute the memory address of an element.
 - Memory for a 2D array is stored **row by row** (row-major order).
- function must know `colsSize` — otherwise, it cannot jump from the start of one row to the start of the next.

```
int main()
{
    int mat[2][3]; // do all task as we did with matrix
    // using of nested loop for 2D array/ matrix
    // how to process the matrix

    for (int row = 0; row < 2; row++)
    {
        for (int cols = 0; cols < 3; cols++)
        {
            printf("please enter the value of row %d and cols %d: ", row, cols);
            scanf("%d", &mat[row][cols]);
        }
    }

    printf("\n Entered array is: \n");
    for (int row = 0; row < 2; row++)
    {
        for (int cols = 0; cols < 3; cols++)
        {
            printf("%d ", mat[row][cols]);
        }
        printf("\n");
    }

    return 0;
}

void flippedCol(int a[][COLS])
{

```

```

printf( "Flipped cols/revese cols are \n");
for (int row = 0; row < ROWS; row++)
{
    for (int col = COLS - 1; col >= 0; col--)
    {
        printf("%d \t", a[row][col]) ;
    }
    printf("\n");
}

void flippedRow(int a[][COLS])
{
    printf( "Flipped row/revese rows are \n");
    for (int row = ROWS - 1; row >= 0; row--)
    {
        for (int col = 0; col < COLS; col++)
        {
            printf("%d \t", a[row][col]);
        }
        printf("\n");
    }
}

```

write a program to input (read in) a matrix and print it in reverse column order, the

```

#define ROWSIZE 3
#define COLSIZE 3

```

HOMEWORK 04:

```

void input(int a[][COLSIZE]);
void flippedRow(int a[][COLSIZE]);
void flippedCol(int a[][COLSIZE]);
void display(int a[][COLSIZE]);

```

```
int isIdentity(int a[][COLSIZE]);
```

```
void transpose(int a[][COLSIZE]);
```

HINT:

- a. For transposing the matrix swap it with the upper triangle with lower triangle

HOME WORK 5:

```
#define NUM_STUDENTS 3
```

```
#define NUM_SCORES 5
```

```
int sum2D(int a[][NUM_SCORES]);
```

```
int avgStd(int a[][NUM_SCORES]); // Avg marks of a student
```

```
int avgClass(int a[][NUM_SCORES]); // Avg marks of class per  
quiz
```

```
int main()
```

```
{
```

```
    int arr[NUM_STUDENTS][NUM_SCORES] = { {10,11,12,9,20},  
                                             {12,12,11,19,2},  
                                             {9,11,13,9,12} };
```

```
}
```

Pointers

A pointer is a special type of variable that stores a *memory address* rather than a direct value. In other words, instead of holding data itself, it points to the location in memory where the data is stored.

Real-world analogy:

1. Identifying Ahmad's house:
 - By name → *Ahmad's house*
 - By address → *Street #10, House #5*
2. Giving a prize in an auditorium:
 - By name → *Moosa*
 - By seat number → *Seat #A-5*

In C programming:

- A value in memory can be accessed either by the variable name or its memory address.
- Example:
- `int a;`
- `a = 10;` // Assigning by variable name
- Trying to assign directly by address does **not** work:
- `&a = 10;` // Invalid
- `0x100 = 10;` // Invalid

This shows why pointers are needed—they provide a valid way to manipulate data through its memory address

We have already seen the pointer usage in the function calling by reference mechanism

Declaration of pointer variable:

Syntax:

```
datatype * nameOfPointerVariable;
```

```
int *ptr; // ptr is a pointer to integer
```

```
int * ptr1, *ptr2, *ptr2, x; etc;
```

Access of memory location using pointer notation

```
#include <iostream>
using namespace std;
int main()
{
    int a = 10;

    int *aPtr = NULL; // pointer point to nothing,
                       //NULL marco define in <cstddef>/
                       //<cstdlib> like a global constant
    int *aPtr1 = nullptr; // modern way to initialized pointer
    aPtr = &a;             // set aPtr to the address of a
    *aPtr = 100; // value at this address = > *(100) if x address is 100
                  // value whatever ptr point to

    cout << "a: " << a << endl; // 00BBFDE4

    cout << "*aPtr:" << *aPtr << endl;;

    cout << "&a: " << &a << endl; // 00BBFDE4
    cout << "aPtr: " << aPtr << endl; //00BBFDE4

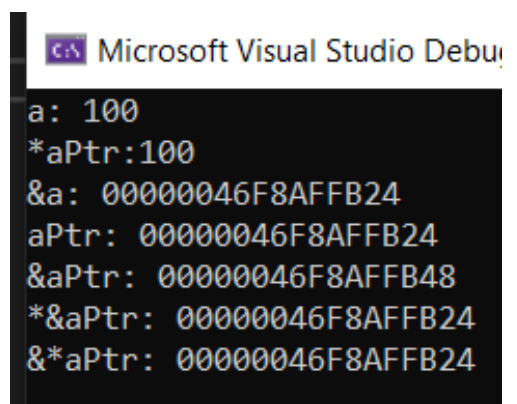
    cout << "&aPtr: " << &aPtr << endl; // 00BBFDD8

    cout << "**&aPtr: " << *(&aPtr) << endl; 00BBFDE4
    cout << "&*aPtr: " << &(*aPtr) << endl;; 00BBFDE4

    return 0;
}
```

Output:

```
a: 10
*aPtr: 10
&a: 00BBFDE4
aPtr: 00BBFDE4
&aPtr: 00BBFDD8
*&aPtr: 00BBFDE4
&*aPtr: 00BBFDE4
```



2. Arithmetic operation with pointer notations:

```
int z = *aptr + 2;
z = *ptr * 2;
z = *ptr / 2;
ptr++; ??? // will discuss it
```

3. Call by reference calling mechanism: need pointer notation

```
void func(int* x)
{
    cin >> x; // enter value 20
}
```

```
int main()
{

    int a = 0;
    func(&a);
    cout << a; // output is 20
}
```

- Pointer variable can appears at left and right side of assignment operator.
- Swapping the array elements are needed in the following tasks:
 - Sorting (Insertion/ bubble sort)
 - reverse array elements
 - transpose of array

```
;;;;;;;;;
```

Example 8: Sort an array

```
#include<ctime> // for time(0)
#include<cstdlib> // for rand() and srand()
#define SIZE 10

void getInput(int[], int);
int main()
{
```

```

srand(time(0));
int array[SIZE];

getInput(array, 10);

cout << "\nBefore sorting array: ";

for (int i = 0; i < 10; i++)
{
    cout << array[i] << " ";
}

// Selection Sort:
int temp;
for (int i = 0; i < 10; i++)
{
    for (int j = i + 1; j < 10; j++)
    {
        if (array[i] > array[j])
        {
            temp = array[i];
            array[i] = array[j];
            array[j] = temp;
        }
    }
}

cout << "\nAfter applying Selection Sort: ";
cout << "\nArray in ascending order: ";

for (int i = 0; i < 10; i++)
{
    cout << array[i] << " ";
}
return 0;
}

```

```

;;;;;;;;;;

```

Example 8: Sort an array

```

#include<iostream>
using namespace std;
#include<ctime> // for time(0)
#include<cstdlib> // for rand() and srand()
#define SIZE 10
void swap(int* x, int* y) // call by reference
{
    int temp = 0;
    temp = *x;
    *x = *y;
    *y = temp;
}

void getInput(int arr[], int)
{
    for (int i = 0; i < 10; i++)

```

```

        {
            arr[i] = rand() % 100;
        }
    }
int main()
{
    srand(time(0));
    int array[SIZE];

    getInput(array, 10);

    cout << "\nBefore sorinting array: ";

    for (int i = 0; i < 10; i++)
    {
        cout << array[i] << " ";
    }

    // Selection Sort:
    int temp;
    for (int i = 0; i < 10; i++)
    {
        for (int j = i + 1; j < 10; j++)
        {
            if (array[i] > array[j])
            {
                swap(&array[i], &array[j]); // call by reference
            }
        }
    }

    cout << "\nAfter applying Selection Sort: ";
    cout << "\nArray in asending order: ";

    for (int i = 0; i < 10; i++)
    {
        cout << array[i] << " ";
    }
    return 0;
}

```

Output:

Before sorinting array: 61 91 41 61 78 69 56 30 38 40

After applying Selection Sort:

Array in asending order: 30 38 40 41 56 61 61 69 78 91

```

;;;;;;;;;;;;;

```

```

void swap(int x, int y) // call by value
{
    int temp;
    temp = x;
    x = y;
}

```

```

        y = temp;
    }

```

Call by value mechanism does not work for swapping the elements (for sorting, reversing, or transposing operations)

```

void swap(int *x, int *y) // call by reference
{
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}

```

Homework:

`void toUpperCase(char *ch);`

function take the address of a character as input and convert it to upper case. HINT: 97-32 = 65

4. Array manipulation

```

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
int main()
{
    // array name is constant pointer
    //6.// variable pointer can pointer to any other locations
        int y[10] = { 0,1,2,3,4,5,6,7,8,9 };

    int* yptr, * yptr1;
    yptr1 = y;
    yptr = y; // yptr is pointer variable
    //yptr = x; // as yptr is pointer variable, so we may assigned a new
address to it
    //y = x; // cant do so, as => int *const y = &x[0]
    // y = &x[0];

    // pointer arithmetic
    cout << y[3] << endl;
    yptr++;
    yptr++;
    yptr++;
    cout << *yptr << endl;
    yptr++;
    cout << *yptr << endl;
    // increament a pointer to illegal memory and OS may not allowed to do so
    // may crash the program// could be undefined behavior.
}

```

```

// pointer arithmetic
*yptr += 3; // add 3 in contents
yptr += 3; // add three integers
cout << *yptr << endl;;

// addition of two pointer
//yptr = yptr + yptr1; /// not allowed
// subtraction of two pointers
cout << yptr - yptr1 << endl;; // distance between to pointers

//comparision of ponters
cout << (yptr > yptr1) << endl;
cout << (*yptr > *yptr1) << endl;
// make a list what we can do with pointers, like assign adress, arithmetic

```

etc

```

// Notation for array []/ *
cout << y[5] << " " << *(y + 5) << endl;
cout << yptr1[5] << " " << *(yptr1 + 5) << endl;

yptr = &y[0];
yptr1 = &y[4];

cout << yptr1 - yptr << endl;

```

5. Home work

use of call by reference and array manipulation using pointer notation: Provide the definition of the following function using **using pointer notation**

- a. **void myStrCopy (char *dest, char * src);**
- b. **ToUpperCase(char *ch);**
- c. **toLowerCase (char *ch)**

Recap

- A pointer is a special type of variable that stores a *memory address* rather than a direct value
- **Declaration of pointer variable:**
- **Access of memory location using pointer notation**
- **Arithmetic operation with pointer notations:** except `ptr++;`
- **CALL BY REFERENCE:**
 - `void swap(int *x, int *y)`
 - `void toUpperCase(char *ch);`

Today lecture

1. Issue in call by reference
2. Use of const keyword: const variable, arrays, and pointers
3. relationship of arrays and pointers
 - constant pointer vs variable pointer
 - pointer arithmetic
 - notation difference `*`() vs `[]`
 - comparison of pointers
 - addition and subtraction of operator
 - passing array and using array notation
4. home work

1. Issue in Call by Reference:

- Advantage: Call by reference is memory-efficient, especially when working with arrays, since no extra copy of the data is created.
- Drawback: The calling function can directly modify the original data. This may be undesirable in situations where the source data must remain unchanged — for example, when copying the contents of an array without altering the original.

• Solution:

```
void myCopy(char dst[], const char src[])
{
    int i = 0;
    while (src[i] != '\0') // while (src[i] != '\0')
        //; // The const keyword helps programmers
        // prevent accidental modifications.
    {
        dst[i] = src[i];
        i++;
    }
    // The const keyword helps programmers prevent accidental
    // modifications.
}
```

2. Use of const keyword:

```
const int SIZE = 10;
SIZE = 11; // error
```

```
const int *ptr = &x;
*ptr = 10; // error so it is useful for safety of source array
           // ptr able to point to other integer variables
           // ptr can read the value of x
           // ptr not able to write on x
```

```
int *const ptr = &x;
ptr = &y // array name is a constant pointer
        // error as pointer is constant pointer to x
        // it can write on x
```

```
const int *const ptr = &x;
           // this pointer
           // it always point to x
           // it don't able point any other int va
           // it is just use to read the value of
```

```
const int SIZE = 10;
1.
int x = 100;
int y = 150;
const int *ptr = &x;
*ptr = 10;
ptr = &y;
*ptr = 10;
2.
int *const ptr = &x;
ptr = &y;
*ptr = 10;
As with array name
3.
const int * const ptr
```

3. The relationship between Arrays and pointers

- Example of pointer variable vs constant pointer example:
- Array name is constant pointer, so it doesn't point to any other variable
- Whereas a variable pointer can point to any other locations

Demo:

```
int main()
{

    int x[10] = { 10,11,12,13,14,15,16,17,18,19 };
    int y[10] = { 0,1,2,3,4,5,6,7,8,9 };

    // constant pointer
    cout << *x; // 10
    x++; // not allowed
    x = y; // not allowed as it is the constant pointer to integer

    y = x; // Error as int *const y = &y[0]
    y = &x[0]; // Error as int *const y = &x[0]

    // pointer variable
    int *yptr, *yptr1;
    yptr1 = y; // or yptr1 = &y[0]
    yptr = y; // yptr is pointer variable
    yptr = x; // as yptr is pointer variable, so we may assigned a
new address to it

    cout << *yptr; // 10
    yptr++; // allowed
```

// array accessing notations

```
cout << "y[0]: " << y[0];  
cout << "*(yptr + 0): " << *(yptr + 0));  
cout << "*yptr: " << *yptr ;
```

// increasement in the pointer

```
yptr++;  
yptr++;  
yptr++; //be careful as increment may cause a pointer to  
point an illegal memory and OS not allowed to do so
```

```
cout << "*yptr: " << *yptr;  
yptr++;  
cout << "*yptr: " << *yptr;  
cout << "++(*yptr) = " << ++(*yptr);  
cout << "(*yptr)++ = ", << (*yptr)++;
```

// print array element using pointer variable

```
for (int i = 0; i < SIZE; i++)  
{  
    cout << *yptr << " ";  
    yptr++; // point to next integer  
}  
  
//  
cout << "Display the array in reverse order " << endl;  
  
yptr--;  
for (int i = 0; i < SIZE; i++)  
{  
    cout << *yptr << " ";  
    yptr--; // point to next integer}  
  
}
```

// Pointer arithmetic

```
*yptr += 3; // add 3 in contents of whatever yptr point to  
yptr += 3; // add three in the pointer address
```

```
cout<<"*yptr: ", *yptr;  
cout<<"yptr: ", yptr;
```

```
yptr -= 3;  
cout <<"y[0]: " << y[0] ;  
cout <<"*(yptr):" << *(yptr);
```

// addition of two pointer

```
yptr = yptr + yptr1; // not allowed as we may go outof memeory  
address
```

// subtraction of two pointers

```
cout<<"yptr - yptr1: " << yptr - yptr1; // distance between to  
pointers  
cout << "yptr1 - yptr: " << yptr1 - yptr; // distance between to  
pointers
```

//comparision of pointers

```
cout<<"yptr > yptr1: " << yptr > yptr1;  
cout<<"*yptr > *yptr1: " << *yptr > *yptr1;
```

// Home work is to make a list what we can do with pointers,
like assign address, arithmetic etc

// Notation for array manipulation with [] and *()

```
cout << "Notation for array with [] and *()\n";
cout << "y[5] and *(y + 5)= " << y[5] << *(y + 5);
cout << "yptr1[5] and *(yptr1 + 5) = " << yptr1[5] <<
*(yptr1 + 5);
//
```

Homework for the following function use of a pointers: 1) call by reference 2) now do it with array manipulation

```
void myStrCopy(char* dst, const char* src)
{
    while (*src != '\0')
    {
        *dst++ = *src++;
    }
    *dst = '\0';
}

int main()
{
    char str1[30] = "pakistan";
    char str2[30];

    myStrCopy(str2, str1);
    cout<<"After calling the myStrCopy(str2, str1) str2= " << str2;

    return 0;
}
```

HomeWork:

Task1: Write a **void printAddressValue(const char *ch)** function that out the address and the value of a character array using pointer notation.

Task2: Write **void copyIntArray(int *dest, const int *src, int size)** function that copies an array of integers from one array to another using pointer notation.

Task3: Write the implementation of **void uppercase(char *)** function that converts the character array of lowercase characters to uppercase using pointer notation.

Task4: Define an int array called **t[]** of size 20 and initialize it randomly. Send the address of the first element of the array to a function called **ReadThemAll()**. Also, pass 2 integers to **ReadThemAll()**. The function will assign the array address to a pointer called **p** and read int numbers in all the slots between these two numbers. The **ReadThemAll()** function should not

use brackets, [] but only the asterisk operator, *. When the function is done, have the main() print out the entire array.

Task5: Write function **void addArrays(const int *a1, const int *a2, int *sum, int size)** which adds two arrays and stores their sum in the third array. Do all the array processes with the help of pointer notation.

2D array:

Task 6: Write a program that creates a two-dimensional integer array named **X** of 3 by 3, and initialize it by user input, and write the following functions.

1. Write a function to print array as **printArray(X)**;
2. Write a function that calculates the average of all the elements in the integer array named **x**.
3. Write a function that computes the square root of the sum of the squares of all the positive elements in an array named **X**.
4. Write a function that takes array **X** and an integer **num** and returns true if **num** exists in **X**, false otherwise.

Relation of arrays and pointers: constant pointer vs variable pointer

```
int main()
{
    int x[10] = { 10,11,12,13,14,15,16,17,18,19 };
    int y[10] = { 0,1,2,3,4,5,6,7,8,9 };
```

// constant pointer

```
x++; // not allowed
x = y; // not allowed as it is the constant pointer to integer
```

// pointer variable

```
int *yptr, *yptr1;

yptr = x; // as yptr is pointer variable, so we may assigned a new address to it
yptr1 = y;      // or yptr1 = &y[0]

yptr = y; // yptr is pointer variable

cout << *yptr; // 10

yptr++; // allowed
```

//2// Notation for array manipulation with [] and *()

```
cout << "Notation for array with [] and *()\n";
cout << "y[0]: " << y[0];
cout << "*(y+0)" << *(y+0);
cout << "yptr[0]: " << yptr[0];
cout << "*(yptr + 0): " << *(yptr + 0);
cout << "*yptr: " << *yptr ;
cout << "y[5] and *(y + 5)= " << y[5] << *(y + 5);
cout << "yptr[5] and *(yptr+5)=" << yptr[5] << *(yptr+5);
```

//3. Pointer arithmetic

```
*yptr += 3; // add 3 in contents of whatever yptr point to yptr += 3; // add
three in the pointer address

yptr -= 3;
```

//4. addition of two pointer

`yptr = yptr + yptr1; // not allowed as we may go out of memory address`

//5. subtraction of two pointers

`cout<<"yptr - yptr1: " << yptr - yptr1; // distance between two pointers`

`cout << "yptr1 - yptr: " << yptr1 - yptr; // distance between two pointers`

//6. comparison of pointers

`cout<<"yptr > yptr1: " << yptr > yptr1; cout<<"*yptr > *yptr1: " << *yptr > *yptr1;`

// **Home work** is to make a list what we can do with pointers, like assign address, arithmetic etc

1. Dynamic memory allocation
2. Return pointer from a function
3. Passing dynamic arrays to a function
4. Issues:
 - a) Memory leakage
 - b) Dangling pointer

Example: Dynamic 1D Array Creation and Use

```
#include <iostream>
using namespace std;

int main() {
    int n;

    // Step 1: Ask the user for the size of the array
    cout << "Enter the size of the array: ";
    cin >> n;

    // Step 2: Dynamically allocate memory for the array
    int *arr = new int[n]; // creates an array of 'n' integers on the heap

    // Step 3: Take input for array elements
    cout << "Enter " << n << " elements:" << endl;
    for (int i = 0; i < n; i++) {
        cout << "Element " << i + 1 << ": ";
        cin >> arr[i]; // store values in dynamically created array
    }

    // Step 4: Display the array elements
    cout << "\nYou entered: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    // Step 5: Deallocate memory (very important!)
    delete[] arr; // frees the dynamically allocated memory
    arr = nullptr; // avoids dangling pointer

    cout << "\n\nMemory successfully released!" << endl;

    return 0;
}
```

1. Dynamic memory allocation, Need/ motivation

```
int main()
{
    double *sale, sum = 0.0, avg = 0.0;
    int numDays;
    cout << "Enter the number of days for sale average calculation: ";
    cin >> numDays;

    sale = new double[numDays];

    for (int i = 0; i < numDays; i++)
    {
        cout << "Enter the sale of day: " << i + 1 << ": ";
        cin >> *(sale + i); // or sale[i]
    }
    for (int i = 0; i < numDays; i++)
    {
        sum = sum + sale[i]; // or sum = sum + *(sale + i)
    }
    avg = sum / numDays;
    cout << "Average sale is " << avg;

    delete[] sale;
    sale = nullptr;
}
```

Example: Returning a Dynamically Created 1D Array from a Function

```
#include <iostream>
using namespace std;
// Function to dynamically create and return an array
int* createArray(int size)
{
    // Step 1: Dynamically allocate memory
    int* arr = new int[size];

    // Step 2: Fill the array with values
    cout << "Enter " << size << " elements:" << endl;
    for (int i = 0; i < size; i++)
    {
        cout << "Element " << i + 1 << ": ";
        cin >> arr[i];
    }
    // Step 3: Return the pointer (memory remains valid outside the function)
    return arr;
}
// Function to display array elements
void displayArray(int* arr, int size)
{
    cout << "\nArray elements are: ";
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}
int main() {
    int n;

    // Step 1: Ask user for size
    cout << "Enter array size: ";
    cin >> n;

    // Step 2: Call function to create array
    int* myArray = createArray(n);

    // Step 3: Display array
    displayArray(myArray, n);

    // Step 4: Free allocated memory
    delete[] myArray;
    myArray = nullptr;
    cout << "\nMemory successfully released!" << endl;
    return 0;
}
```

```
}
```

Another example:

1. Returnning pointer from a function:

```
int* getRandomNo(int num)
{
    int* p = new int[num];
    for (int i = 0; i < 10; i++)
    {
        *(p + i) = rand() % 10;
    }
    return p;
}
int main()
{
    srand(time(0));

    int* b = getRandomNo(10);
    for (int i = 0; i < 10; i++)
    {
        cout << *(b + i) << endl;
    }

    delete[] b; // should delete, otherwise memory Leakage occurs
    b = NULL;
    return 0;
}
```



Memory Leakage

◆ Meaning

A memory leak happens when:

You allocate memory dynamically using new, but forget to delete it later.

That memory remains reserved in RAM (Heap) even after your program no longer needs it — it's "lost" because there's no pointer referencing it anymore.

```
#include <iostream>
using namespace std;
int main()
{
    int *arr = new int[5]; // dynamically allocate array of 5 integers

    for (int i = 0; i < 5; ++i)
        arr[i] = i + 1;

    arr = new int[10]; // ⚠ previous memory (5 ints) is lost
}
```

```

// We allocated new memory but never deleted the old one!
// -> Memory leak occurs here

delete[] arr; // only deletes the second (10-int) allocation
return 0;
}

```

Correct Version (no leak) – use new and delete in pairs

```

int *arr = new int[5];
for (int i = 0; i < 5; ++i) arr[i] = i + 1;

delete[] arr; // free the first allocation before reusing pointer
arr = new int[10]; // safe reuse
delete[] arr; // clean up again

```

Dangling Pointer

Meaning

A **dangling pointer** occurs when:

You delete or free dynamically allocated memory, but the pointer still holds the address of that now-freed memory.

If you try to use that pointer afterward, it leads to **undefined behavior** (crashes, garbage values, etc.).

```

#include <iostream>
using namespace std;

int main() {
    int *arr = new int[5];
    for (int i = 0; i < 5; ++i)
        arr[i] = i + 1;

    delete[] arr; // memory freed
    cout << arr[0]; // ⚠️ dangling pointer access! memory is freed!
    return 0;
}

```

Correct Fix

After deleting, always nullify the pointer:

```

delete[] arr;
arr = nullptr; // mark it as invalid pointer

```

Now, if someone tries to use it:

```
if (arr != nullptr)
    cout << arr[0];
else
    cout << "Pointer is null, cannot access memory.";
```

Always check (arr != nullptr) before accessing any dynamically allocated memory.

In modern, safe C++ programming, it's **always recommended** to check whether a pointer is nullptr **before accessing it**, especially when the pointer might:

- not be initialized yet,
- have been deleted, or
- conditionally assigned to memory.

HomeWork:

- 1 Write a program that dynamically allocates an array, fills it with values, then **intentionally leaks memory**.
- 2 Modify it to properly release memory.
- 3 Then intentionally create a **dangling pointer**, observe the issue, and fix it.

////////////////////

Example: Dynamically Creating and Returning a 2D Array in C++

```
#include <iostream>
using namespace std;

// Function to dynamically create a 2D array
int** create2DArray(int rows, int cols) {
    // Step 1: Allocate memory for row pointers
    int** arr = new int*[rows];

    // Step 2: Allocate memory for each row
    for (int i = 0; i < rows; i++) {
        arr[i] = new int[cols];
    }

    // Step 3: Fill array with user input
    cout << "Enter " << rows * cols << " elements for the matrix:\n";
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            cout << "Element [" << i << "][" << j << "]: ";
            cin >> arr[i][j];
        }
    }

    return arr; // Return pointer to pointer
}

// Function to display a 2D array
void display2DArray(int** arr, int rows, int cols) {
    cout << "\nMatrix elements are:\n";
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            cout << arr[i][j] << "\t";
        }
        cout << endl;
    }
}
```

```
// Function to delete the dynamically created 2D array
```

```
void delete2DArray(int** arr, int rows) {  
    for (int i = 0; i < rows; i++) {  
        delete[] arr[i]; // delete each row  
    }  
    delete[] arr; // delete array of pointers  
    arr = nullptr; // avoid dangling pointer  
}
```

```
int main() {
```

```
    int rows, cols;
```

```
    cout << "Enter number of rows: ";
```

```
    cin >> rows;
```

```
    cout << "Enter number of columns: ";
```

```
    cin >> cols;
```

```
    // Step 1: Create 2D array
```

```
    int** matrix = create2DArray(rows, cols);
```

```
    // Step 2: Display 2D array
```

```
    display2DArray(matrix, rows, cols);
```

```
    // Step 3: Free memory
```

```
    delete2DArray(matrix, rows);
```

```
    cout << "\nMemory successfully released!" << endl;
```

```
    return 0;
```

```
}
```

For example, to create a 2D array of size rows × cols:

int rows = 5, cols = 5;

```
// Step 1: Create an array of int* (each will point to a row)
```

```
int **arr = new int*[rows];
```

```
// Step 2: For each row, allocate an array of int
for (int i = 0; i < rows; ++i) {
    arr[i] = new int[cols];
}
```

Now arr behaves like a normal 2D array:

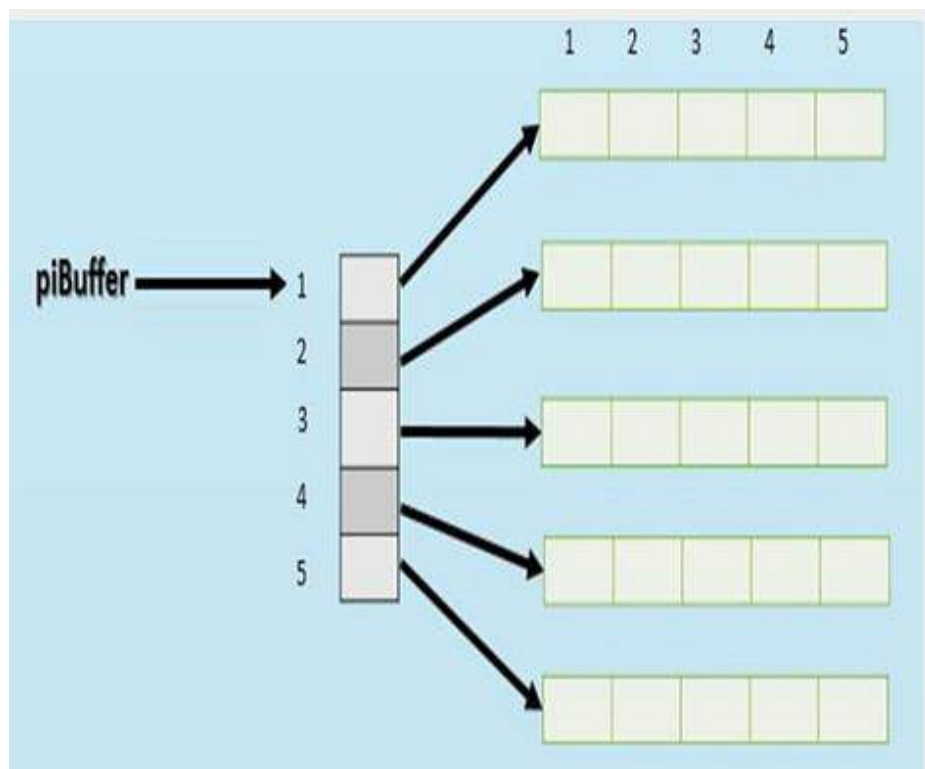
```
arr[0][0] = 10;
```

```
arr[1][2] = 25;
```

🔧 Deleting (Freeing Memory)

Always free memory in the **reverse order** of allocation:

```
for (int i = 0; i < rows; ++i) {
    delete[] arr[i]; // delete each row
}
delete[] arr;        // delete the array of pointers
```



```
using namespace std;
```

```

int main() {
    int rows = 3, cols = 4;

    // Dynamic allocation
    int **arr = new int*[rows];
    for (int i = 0; i < rows; ++i)
        arr[i] = new int[cols];

    // Assign values
    for (int i = 0; i < rows; ++i)
        for (int j = 0; j < cols; ++j)
            arr[i][j] = i * cols + j + 1;

    // Display values
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j)
            cout << arr[i][j] << " ";
        cout << endl;
    }

    // Deallocate memory
    for (int i = 0; i < rows; ++i)
        delete[] arr[i];
    delete[] arr;

    return 0;
}

```

Output:

```

83 86 77 15
93 35 86 92
49 21 62 27

```

Example: Function to Display a Dynamic 2D Array (printMatrix (arr, rows, cols);)

```

#include <iostream>
using namespace std;

```

```
// Function to print 2D array
```

```
void printMatrix(int **arr, int rows, int cols) {
```

```
    cout << "\nMatrix:\n";
```

```
    for (int i = 0; i < rows; ++i) {
```

```
        for (int j = 0; j < cols; ++j)
```

```
            cout << arr[i][j] << " ";
```

```
        cout << endl;
```

```
    }
```

```
}
```

```
int main() {
```

```
    int rows = 3, cols = 4;
```

```
    // Dynamic allocation
```

```
    int **arr = new int*[rows];
```

```
    for (int i = 0; i < rows; ++i)
```

```
        arr[i] = new int[cols];
```

```
    // Fill array
```

```
    for (int i = 0; i < rows; ++i)
```

```
        for (int j = 0; j < cols; ++j)
```

```
            arr[i][j] = i * cols + j + 1;
```

```
    // Function call
```

```
    printMatrix(arr, rows, cols);
```

```
    // Free memory
```

```
    for (int i = 0; i < rows; ++i)
```

```
        delete[] arr[i];
```

```
    delete[] arr;
```

```
    return 0;
```

```
}
```

❑ Implementation a function that **calculate the sum of all elements**.

❑ Implementation a function that **find the maximum element in the matrix**.

❏ Implementation a function `void inputMatrix(int **arr, int rows, int cols);` to handle input separately.

Example: Returning a 2D Dynamic Array

```
#include <iostream>
```

```
using namespace std;
```

```
// Function to create and return a 2D dynamic array
```

```
int** createMatrix(int rows, int cols) {
```

```
    int **arr = new int*[rows];
```

```
    for (int i = 0; i < rows; ++i) {
```

```
        arr[i] = new int[cols];
```

```
        for (int j = 0; j < cols; ++j)
```

```
            arr[i][j] = i * cols + j + 1; // fill with numbers
```

```
    }
```

```
    return arr; // returning pointer to pointer
```

```
}
```

```
void printMatrix(int **arr, int rows, int cols) {
```

```
    for (int i = 0; i < rows; ++i) {
```

```
        for (int j = 0; j < cols; ++j)
```

```
            cout << arr[i][j] << " ";
```

```
        cout << endl;
```

```
    }
```

```
}
```

```
int main() {
```

```
    int rows = 3, cols = 4;
```

```
    int **matrix = createMatrix(rows, cols);
```

```
    cout << "Matrix:\n";
```

```
    printMatrix(matrix, rows, cols);
```

```
    // Free memory
```

```
    for (int i = 0; i < rows; ++i)
```

```
        delete[] matrix[i];
```

```
    delete[] matrix;
```

```
    return 0;
```

```
}
```

Program 2: Function That Returns a 2D Array of a Multiplication Table

Objective:

Create a function that returns a 2D array containing a multiplication table up to $n \times n$.

```
#include <iostream>
```

```
using namespace std;
```

```
int** createMultiplicationTable(int n) {  
    int **table = new int*[n];  
    for (int i = 0; i < n; ++i)  
        table[i] = new int[n];  
  
    for (int i = 0; i < n; ++i)  
        for (int j = 0; j < n; ++j)  
            table[i][j] = (i + 1) * (j + 1);  
  
    return table;  
}
```

```
void displayTable(int **table, int n) {  
    for (int i = 0; i < n; ++i) {  
        for (int j = 0; j < n; ++j)  
            cout << table[i][j] << "t";  
        cout << endl;  
    }  
}
```

```
int main() {  
    int n;  
    cout << "Enter table size: ";  
    cin >> n;  
  
    int **table = createMultiplicationTable(n);  
  
    cout << "\nMultiplication Table:\n";  
    displayTable(table, n);  
}
```

```
for (int i = 0; i < n; ++i)
    delete[] table[i];
delete[] table;

return 0;
```

Today lecture:

- 2D dynamic memory allocation
- File handling

For example, to create a 2D array of size rows × cols:

```
int rows = 5, cols = 5;
```

```
// Step 1: Create an array of int* (each will point to a row)
```

```
int **arr = new int*[rows];
```

```
// Step 2: For each row, allocate an array of int
```

```
for (int i = 0; i < rows; ++i) {
```

```
    arr[i] = new int[cols];
```

Now arr behaves like a normal 2D array:

```
arr[0][0] = 10;
```

```
arr[1][2] = 25;
```

Deleting (Freeing Memory)

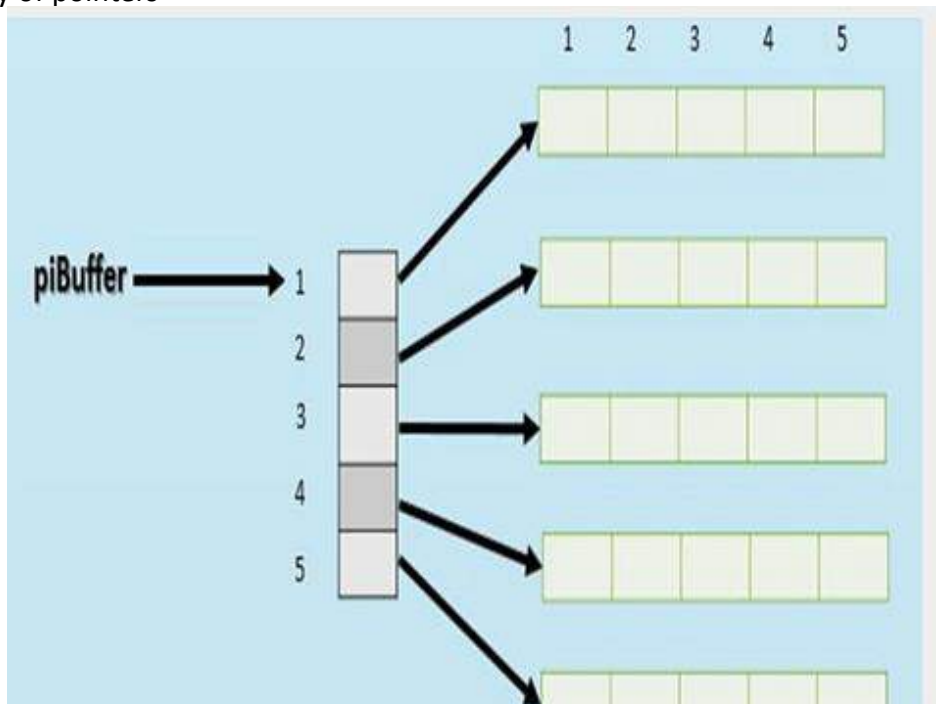
Always free memory in the **reverse order** of allocation:

```
for (int i = 0; i < rows; ++i) {
```

```
    delete[] arr[i]; // delete each row
```

```
}
```

```
delete[] arr; // delete the array of pointers
```



Complete Example

```
#include <iostream>
using namespace std;

int main() {
    int rows = 3, cols = 4;

    // Dynamic allocation
    int **arr = new int*[rows];
    for (int i = 0; i < rows; ++i)
        arr[i] = new int[cols];

    // Assign values
    for (int i = 0; i < rows; ++i)
        for (int j = 0; j < cols; ++j)
            arr[i][j] = i * cols + j + 1;

    // Display values
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j)
            cout << arr[i][j] << " ";
        cout << endl;
    }

    // Deallocate memory
    for (int i = 0; i < rows; ++i)
        delete[] arr[i];
    delete[] arr;

    return 0;
}
```

Output:

```
83 86 77 15
93 35 86 92
49 21 62 27
```

```
}
```

Example: Function to Display a Dynamic 2D Array (printMatrix (arr, rows, cols);)

```
#include <iostream>
```

```
using namespace std;
```

```
// Function to print 2D array
```

```
void printMatrix(int **arr, int rows, int cols) {
```

```
    cout << "\nMatrix:\n";
```

```
    for (int i = 0; i < rows; ++i) {
```

```
        for (int j = 0; j < cols; ++j)
```

```
            cout << arr[i][j] << " ";
```

```
        cout << endl;
```

```
    }
```

```
}
```

```
int main() {
```

```
    int rows = 3, cols = 4;
```

```
    // Dynamic allocation
```

```
    int **arr = new int*[rows];
```

```
    for (int i = 0; i < rows; ++i)
```

```
        arr[i] = new int[cols];
```

```
    // Fill array
```

```
    for (int i = 0; i < rows; ++i)
```

```
        for (int j = 0; j < cols; ++j)
```

```
            arr[i][j] = i * cols + j + 1;
```

```
    // Function call
```

```
    printMatrix(arr, rows, cols);
```

```
    // Free memory
```

```
    for (int i = 0; i < rows; ++i)
```

```
        delete[] arr[i];
```

```

delete[] arr;

return 0;
}

```

Home works:

1. Implementation a function that **calculate the sum of all elements**.
2. Implementation a function that **find the maximum element in the matrix**.
3. Implementation a function **void inputMatrix(int **arr, int rows, int cols);** to handle input separately.

Example: Returning a 2D Dynamic Array

```

#include <iostream>
using namespace std;

```

```

// Function to create and return a 2D dynamic array

```

```

int** createMatrix(int rows, int cols) {
    int **arr = new int*[rows];

    for (int i = 0; i < rows; ++i) {
        arr[i] = new int[cols];
        for (int j = 0; j < cols; ++j)
            arr[i][j] = i * cols + j + 1; // fill with numbers
    }

    return arr; // returning pointer to pointer
}

```

```

void printMatrix(int **arr, int rows, int cols) {

```

```

    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j)
            cout << arr[i][j] << " ";
        cout << endl;
    }
}

```

```

int main() {
    int rows = 3, cols = 4;

```

```

int **matrix = createMatrix(rows, cols);

cout << "Matrix:\n";
printMatrix(matrix, rows, cols);

// Free memory
for (int i = 0; i < rows; ++i)
    delete[] matrix[i];
delete[] matrix;

return 0;
}

```

Program 2: Function That Returns a 2D Array of a Multiplication Table

Objective:

Create a function that returns a 2D array containing a multiplication table up to $n \times n$.

```
#include <iostream>
```

```
using namespace std;
```

```

int** createMultiplicationTable(int n) {
    int **table = new int*[n];
    for (int i = 0; i < n; ++i)
        table[i] = new int[n];

    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            table[i][j] = (i + 1) * (j + 1);

    return table;
}

```

```

void displayTable(int **table, int n) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j)
            cout << table[i][j] << "\t";
        cout << endl;
    }
}

```

```

int main() {
    int n;
    cout << "Enter table size: ";
    cin >> n;

    int **table = createMultiplicationTable(n);

    cout << "\nMultiplication Table:\n";
    displayTable(table, n);

    for (int i = 0; i < n; ++i)
        delete[] table[i];
    delete[] table;

    return 0;
}

```

//////////

Another Sample code with functions:

Example: Dynamically Creating and Returning a 2D Array in C++

```

#include <iostream>
using namespace std;
// Function to dynamically create a 2D array
int** create2DArray(int rows, int cols) {
    // Step 1: Allocate memory for row pointers
    int** arr = new int*[rows];

    // Step 2: Allocate memory for each row
    for (int i = 0; i < rows; i++) {
        arr[i] = new int[cols];
    }

    // Step 3: Fill array with user input
    cout << "Enter " << rows * cols << " elements for the matrix:\n";
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            cout << "Element [" << i << "][" << j << "]: ";

```

```

        cin >> arr[i][j];
    }
}

return arr; // Return pointer to pointer
}

// Function to display a 2D array
void display2DArray(int** arr, int rows, int cols) {
    cout << "\nMatrix elements are:\n";
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            cout << arr[i][j] << "t";
        }
        cout << endl;
    }
}

// Function to delete the dynamically created 2D array
void delete2DArray(int** arr, int rows) {
    for (int i = 0; i < rows; i++) {
        delete[] arr[i]; // delete each row
    }
    delete[] arr; // delete array of pointers
    arr = nullptr; // avoid dangling pointer
}

int main() {
    int rows, cols;

    cout << "Enter number of rows: ";
    cin >> rows;
    cout << "Enter number of columns: ";
    cin >> cols;

    // Step 1: Create 2D array
    int** matrix = create2DArray(rows, cols);

```

```

// Step 2: Display 2D array
display2DArray(matrix, rows, cols);

// Step 3: Free memory
delete2DArray(matrix, rows);

cout << "\nMemory successfully released!" << endl;
return 0;
}

```

```

;;;;;;;;;;;;;

```

FileHandling:

5.11 Using Files for Data Storage:

When a program needs to save data for later use, it writes the data in a file. The data can then be read from the file later.

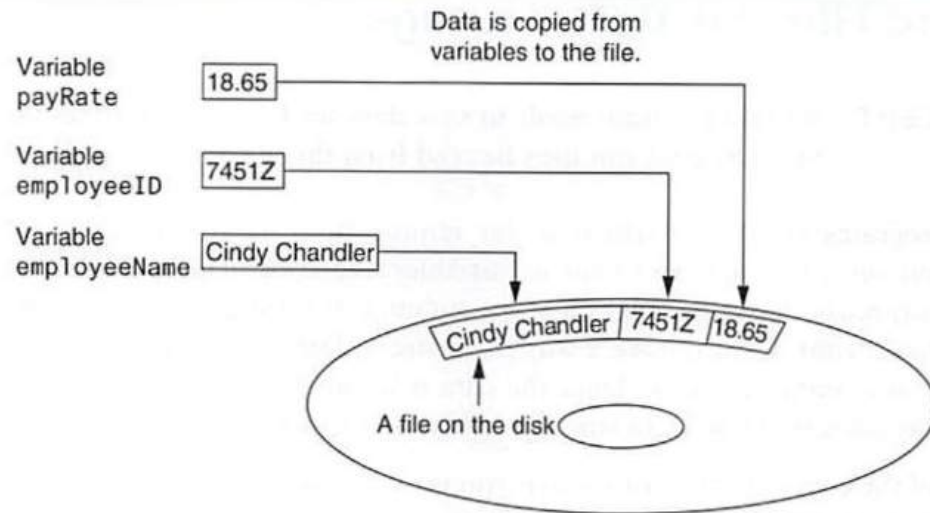
Examples:

- Payroll programs keep employee data in the files.
- Inventory programs keep data about the product in files.
- The accounting system keep data about a company's financial operations in files.

1. File writing and reading:

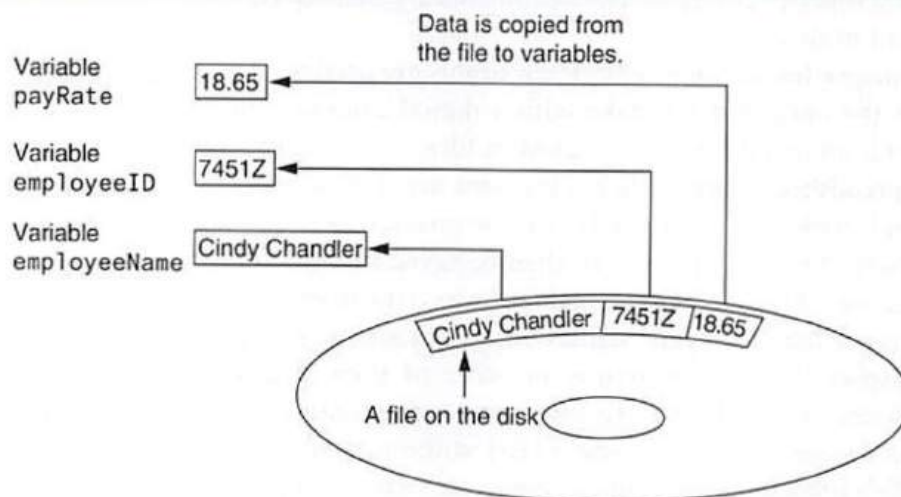
Programmer usually refer to the process of saving data in a file as writing data to the file. When a piece of data is written to a file, it is copied from a variable in RAM to the file. This is illustrated in Figure 5.12. an output ifle is a file to which data is written. It is called an output file because the program stores output in it.

Figure 5-12 Writing data to a file



The process of retrieving data from a file is known as reading data from the file. When a piece of data is read from a file, it is copied from the file into a variable in RAM, Figure 5.13 illustrates this process. An input is a file from which data is read. It is called an input file because the program gets input from the file.

Figure 5-13 Reading data from a file



This section discusses ways to create program that write data to files and read data from the files.

2. STEPS for reading or writing operation:

When a file is used by a program, three steps must be taken.

1. Open the file : opening a file create a connection between the file and the program. Opening an output file usually creates the file on the disk, and allow the program to write data to it. Opening an input file allows the program to read data from the file.
2. Process the file: data is either written to the file- if file is an output file) or read from the file (if it is input file).
3. Close the file- after the program is finished using the file, the file must be closed. Closing a file disconnects the file from the program.

3. TYPES OF FILES:

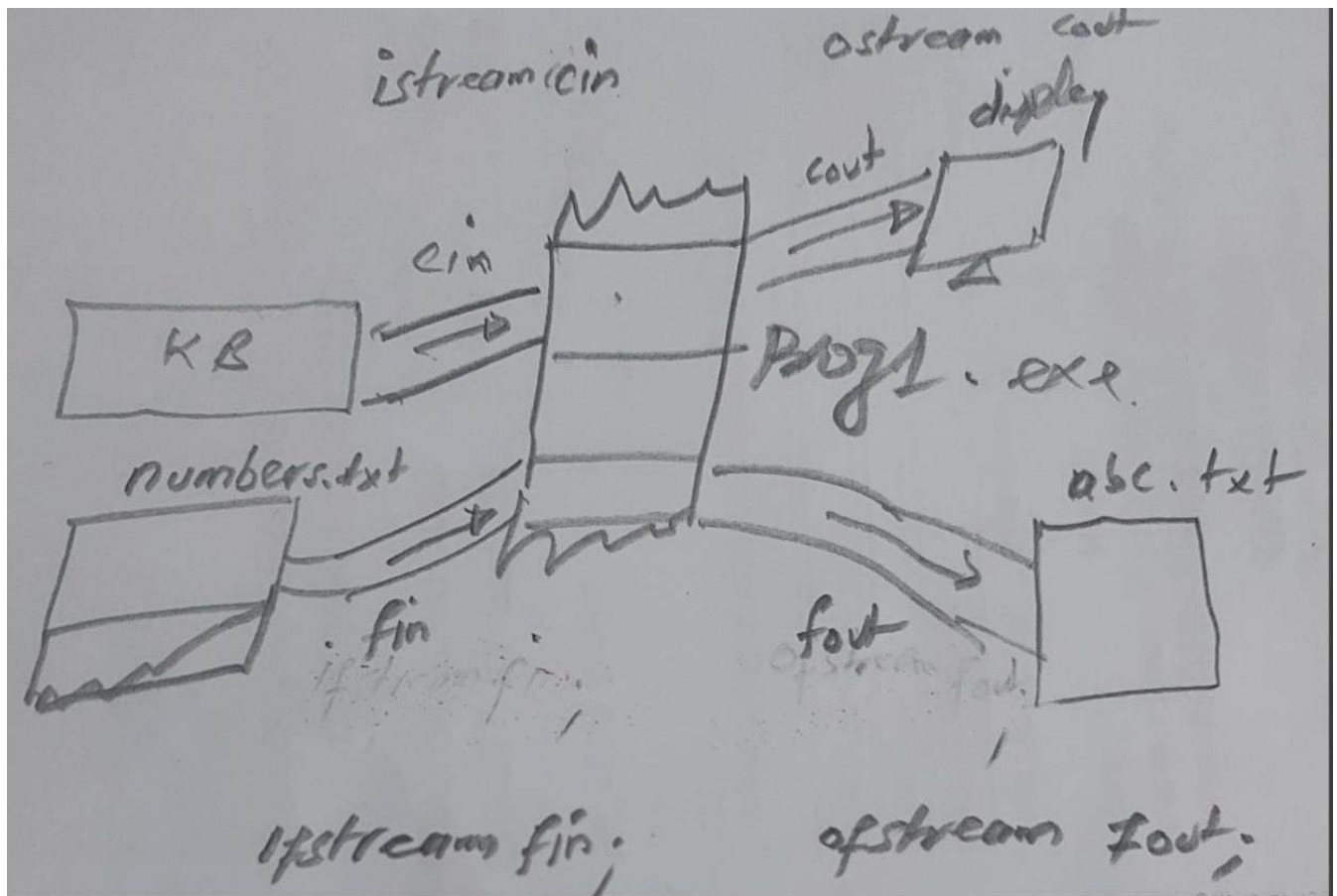
1. TEXT FILES: contains the data that has been encoded as text, using a scheme such as ASCII or Unicode. We can view in notepad. (DISCUSS IN THIS CHAPTER)
2. BINARY FILES: contains data that has not be converted to text. You can not view the contents of a binary file with a text editor. (DISCUSS IN CHAPTER 12)

4. FILE ACCESS METHODS:

Two methods to access data stored in a file: **sequential access** (cassets type player DISCUSS IN THIS CHAPTER)and **direct access/random-accesss** (CD Player or MP3 player works- DISCUSS IN CHAPTER 12).

5. SETTING UP A PROGRAM FOR FILE INPUT/OUTPUT

`#include <fstream>` # controls all the declaration necessary for file operations. It defines the data types of ofstream, ifstream, and fstream



6. Writing to a file

Prog1.cpp (string data)

```

/*This program write data to a file*/
#include<iostream>
#include <fstream>
using namespace std;
int main()
{
    // Open file
    ofstream outputFile;
    //outputFile.open("H:\\PUCIT\\PF_2024_Spring\\ClassLecture_PF\\demoFile5.txt");
    outputFile.open("demoFile5.txt");

    // write some message to the file
    if (outputFile) //or if (!outputFile.fail())
    {
        cout << "Now writting data to the file" << endl;
        outputFile << "Welcome to programmingFundamental course";
        outputFile << "I love C progrmming";
        cout << "File writting process is done! " << endl;
    }
}

```

```

    }
    else
    {
        cout << "There is some error in file writting";
    }
    //close the file
    outputFile.close();
    return 0;
}

;;;;;;;;;;;;;;

```

Prog2.cpp (writing numeric data)

```

/*This program writes user input to a file*/
#include<iostream>
#include <fstream>
using namespace std;
int main()
{
    // Open file
    ofstream outputFile;
    int num1, num2, num3;
    //outputFile.open("H:\\PUCIT\\PF_2024_Spring\\ClassLecture_PF\\demoFile5.txt");
    outputFile.open("numbers.txt");

    // get three numbers from the user
    cout << "Enter a number: ";
    cin >> num1;
    cout << "Enter an other number : ";
    cin >> num2;
    cout << "Once more time , Enter a number: ";
    cin >> num3;

    // write some message to the file
    if (!outputFile.fail())
    {
        cout << "\nNow writting data to the file" << endl;

        outputFile << num1 << endl;
        outputFile << num2 << endl;
        outputFile << num3 << endl;

        cout << "Done!";
    }
    else
    {
        cout << "There is some error in file writting";
    }
    //close the file
    outputFile.close();
    return 0;
}

;;;;;;;;;;;;;;

```

Prog3.CPP (Write friends names to a file)

```
/*This program writes user input to a file*/
#include<iostream>
#include <fstream>
using namespace std;
int main()
{
    // Open file
    ofstream outputFile;
    string name1, name2, name3;
    //outputFile.open("H:\\PUCIT\\PF_2024_Spring\\ClassLecture_PF\\demoFile5.txt");
    outputFile.open("friends.txt");

    // get name of three friends from the user
    cout << "Enter a name of friend 1: ";
    cin >> name1;
    cout << "Enter a name of friend 2: ";
    cin >> name2;
    cout << "Enter a name of friend 3: ";
    cin >> name3;

    // write some message to the file
    if (!outputFile.fail())
    {
        cout << "Now writting data to the file" << endl;

        outputFile << name1 << endl;
        outputFile << name2 << endl;
        outputFile << name3 << endl;

        cout << "Done!";
    }
    else
    {
        cout << "There is some error in file writting";
    }
    //close the file
    outputFile.close();
    return 0;
}

;;;;;;;;;;;;;
```

7. READING FROM FILE:

Prog4.cpp (read name of friends and display them)

```
/*This program read three names from a file*/
#include<iostream>
#include <fstream>
using namespace std;
int main()
{
    // Open file
    ifstream fin;
    string name1, name2, name3;

    fin.open("friends.txt");

    // write some message to the file
    if (!fin.fail())
    {
        cout << "\nReading data from the file" << endl;

        // read the three numbers from the file
        fin >> name1; // extract data from file
        cout << name1 << endl;;
        fin >> name2;
        cout << name2 << endl;;
        fin >> name3;
        cout << name3 << endl;;

    }
    else
    {
        cout << "There is some error in file writting";
    }
    //close the file
    fin.close();
    return 0;
}
```

8. The read position:

Figure 5-19 Initial read position

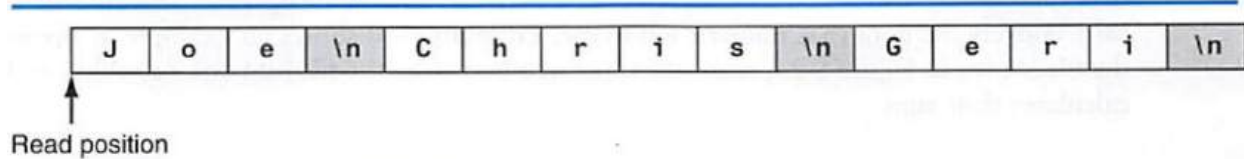


Figure 5-20 The next read position

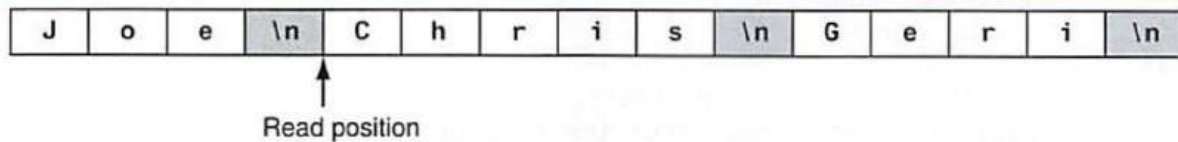
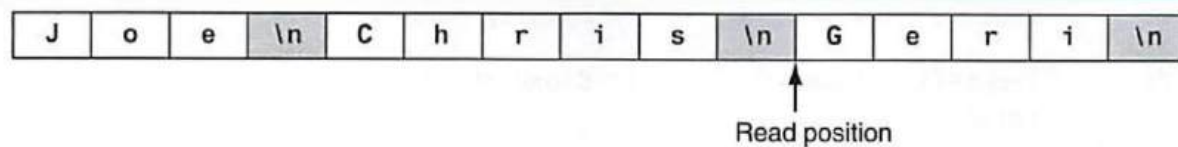


Figure 5-21 The next read position



Prog5.CPP (reading data and compute their sum)

```
/*This program read three numbers from a file*/  
// FIND sum, product, average, sort them, each number can be reverse, and check for  
palindrome etc
```

```
#include<iostream>  
#include <fstream>  
using namespace std;  
int main()  
{  
    // Open file  
    ifstream fin;  
    int num1, num2, num3, sum;  
  
    fin.open("numbers.txt");  
  
    // write some message to the file  
    if (!fin.fail())  
    {  
        cout << "\nNow reading data from the file" << endl;  
  
        // read(Extract data from file) the three numbers from the file  
        fin >> num1; // >> automatically convert it numeric data type  
        fin >> num2;  
        fin >> num3;
```

```

        cout << "Done!\n";

        sum = num1 + num2 + num3;
        cout << "sum of " << num1 << " " << num2 << " " << num3 << " = " << sum;
    }
    else
    {
        cout << "There is some error in file writting";
    }
    //close the file
    fin.close();
    return 0;
}

```

9. USING LOOPS TO PROCESS FILES.

Prog6.CPP

```

/*This program write data to file using loop*/
#include<iostream>
#include <fstream>
using namespace std;
int main()
{
    // Open file
    ofstream fout;
    int numOfDays;
    double sales;

    // get number of days
    cout << "How many days do you have sales: ";
    cin >> numOfDays;

    fout.open("Sales.txt");

    // Get the sales for each day and write it to the file
    if (!fout.fail())
    {
        // get the sales for each day and write it to the file

        for (int i = 0; i < numOfDays; i++)
        {
            cout << "Enter the sales for the day "<< i+1 << ": ";
            cin >> sales;

            // write the sales to files;
            fout << sales << endl;
        }
    }
    else
    {
        cout << "There is some error in file writting";
    }
}

```

```
//close the file
fout.close();
return 0;}
```

10. Detection the End of the file with >> read data and return true or false

Prog7.CPP

```
/*This program read data to file using loop*/
#include<iostream>
#include <fstream>
using namespace std;
int main()
{
    // Open file
    ifstream fin;
    double sales;

    fin.open("Sales.txt");

    // Get the sales for each day and write it to the file
    if (!fin.fail()) // if (!fin)
    {
        // get the sales for each day and write it to the file

        while (fin >> sales)
        {

            cout << sales << endl;

        }

    }
    else
    {
        cout << "There is some error in file writting";
    }
    //close the file
    fin.close();
    return 0;
}
```

Course outline Topics:

1. Object orientation vs. function orientation
2. Fundamentals of object orientation (Information hiding, encapsulation, separation of interface from implementations, abstraction, inheritance, generalization ,specialization , Multiple Inheritance , aggregation, compositions and polymorphism)
3. Introductions to Objects and Classes
4. Constructors and Destructor
5. Accessor Functions & this Pointer
6. Constant data members & Static Variables
7. Constant data members & Static Variables & Array of Objects
8. New Operator & Getter and Setter
9. Friend Functions and friend classes
10. Operator overloading

After midterm exam

11. Composition & Aggregation
12. Inheritance
13. Access Specifiers
14. Copy Constructor & Assignment Operator in inheritance
15. Overriding
16. Types of Inheritance (public, private, protected with exmples)
17. Polymorphism
18. Abstract & Concrete Classes
19. Multiple Inheritance
20. Templates classes and functions

////////////////////

“In Programming Fundamentals, we learned how to solve problems using *functions* and *data structures/variables/arrays*.

We wrote programs that worked — but as they grew larger (100’s functions, many arrays etc), they became harder to manage, debug, and extend.

Today, we begin something that changes the way we *think* about programming — not just *what* we code, but ***how* we organize our code**.

This is **Object-Oriented Programming (OOP)**.”

The agenda of today's lecture is:

[Acknowledgment: Contents taken from Sir Abdul Mateen Handouts.]

1. OOP Definition
2. Motivation for OOP
3. Maintenance Difficulties in Procedural Programming
4. How OOP Address Maintenance Issues
5. Summary
6. Practice Problems

1. Definition OOP

- Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects", which represent real-world entities.
- Each object contains data (attributes) and functions (methods) that operate on the data.
- The goal of OOP is to **model real-world problems more naturally**, **promote code reusability**, and **improve program structure and maintenance**.

1. **Real-world modeling** → **Abstraction + Encapsulation** (class)
2. **Reusability** → **Inheritance** (Inheritance allows new classes to reuse, modify, or extend the behavior of existing ones.)
3. **Maintenance** → **Encapsulation + Polymorphism**
4. **Encapsulation** localizes code changes — internal modifications don’t affect other parts of the program.
5. **Polymorphism** makes systems **easier to update or expand because new classes can integrate seamlessly under a common interface.** [Add new types without changing existing logic” means you can create new subclasses (new behaviors) and the old code (that uses the base class interface) will still work perfectly — no rewrites, no breakage.]
6.]

1.1. OOP Four Principles

- 1) Encapsulation – bundling data and methods together, restricting direct access to data
- 2) Abstraction –
Designing: Not consider irrelevant details while design class) [add more details form Sir Shafiqur Rehman notes]
Implementation : Hiding complex details and exposing only essential features
- 3) Inheritance – allowing new classes to reuse and extend existing ones
- 4) Polymorphism – enabling a single interface to represent different behaviors

2. Motivation for OOP

In procedural programming (like C or traditional structured code), the program is divided into functions (Fine for small size)

OOP was introduced to overcome these challenges and make programs easier to manage, extend, and maintain

Key motivations include:

- Real-world modeling: systems are composed of interacting entities, not just procedures
- Reusability: classes can be reused or extended, avoiding duplication
- Maintenance: code changes become localized and safer
- Scalability: larger systems can be built in a modular way

3. Maintenance Difficulties in Procedural Programming

- Procedural programming faces severe maintenance issues because of the separation between data and functions.
- Functions often access global or shared data, making it hard to track dependencies.
- A single change in one function can unexpectedly affect others

3.1. Example 1 – Tight Coupling through Global Data

3.2. Example 2 – Difficult to Extend and Debug

3.3. Lack of Modularity

Problems with Procedure/Function-Oriented Programming (PF)

3.2 Example 2 – Difficult to Extend and Debug: Difficult to maintain programs --which explains another major weakness of procedural programming that led to the rise of Object-Oriented Programming (OOP).

```
#include <iostream>
#include<cstring>
using namespace std;
char studentName [100][50];
int studentAge[100];
int totalStudents = 0;
void addStudent(char *name, int age)
{
    strcpy (studentName[totatStudent], name);
    studentAge[totalStudents] = age;
    totalStudents++;
}
void showAllStudents() {
    for (int i = 0; i < totalStudents; i++)
        cout << studentName[i] << studentAge[i] << endl;
}
```

Your **manager** says:

"Add functionality to calculate the average age and also store each student's GPA."

To do that, you now need to:

- Add a new global array float gpa[100];
- Modify addStudent() to also handle GPA
- Update showAllStudents() to print GPA
- Add a new function to calculate the average

⚠ Every change touches multiple parts of the program.

If you **miss updating one array**, or misalign indices, **bugs start appearing** (e.g., mismatched names and GPAs).

1. Each class manages its own internal state and exposes only controlled interfaces

```
class Student {
private:
    string name;
    int age;
    float gpa;
public:
    Student(string n, int a, float g)
    {
        name = n;
        age = a;
        gpa = g;
    }

    void showData() {
        cout << name << " - " << age << " - " << gpa << endl;
    }

    int getAge() { return age; }
};
```

You extend behavior by **adding new methods** or subclasses, not by modifying existing code.

- Procedural code is often tightly coupled — changes in one part may affect many others.
- Type conversions (e.g., short → long) and dependencies make updates risky.
- As programs grow large, debugging and enhancement become harder.

3.1. Example 1. Tight Coupling via Global Data /Unrestricted access to global data/ [data is easily accessible to every function]

- In procedural programming, functions have direct access to global data. [we make the local variable than problem visualization become even complex]
- This makes the program prone to errors because any function can modify shared data unexpectedly.
- It becomes difficult to track where and why a value changes.

Tight Coupling via Global Data

```
#include <iostream>
using namespace std;

// Global data
int studentAge;

void setStudentData(string name, int age)
{
    studentAge = age; // ✗ No validation here
}

OR // make a local variable and pass to each function call by
reference it still very messy and error prone
void setStudentData(int &age)
{
    age = 21; // ✗ No validation here
}

void showStudent()
{
    cout << "Age: " << studentAge << endl;
}

int main() {
    setStudentData("Ali", 21);
    showStudent();

    studentAge = -5; // ✗ Direct access - invalid data possible
    showStudent(); // Output: Age = -5 (Invalid)
}
```

From Tight Coupling to Encapsulation

In OOP, data and related functions live together (inside a class).

That's what makes it robust and secure.

OOP combines data and functions into one unit called a class. This means: - Data is **encapsulated**** and protected from unauthorized access**

```
#include <iostream>
using namespace std;

class Student {
private:
    int age;

public:
    void setData( int a) {
        if (a > 0 && a < 120) { // ✓ Validation enforced
            age = a;
        } else {
            cout << "Invalid age entered!\n";
        }
    }

    void showData() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};

int main() {
    Student s1;
    s1.setData("Ali", 21);
    s1.showData();

    // s1.age = -5; ✗ Not allowed - private member (compiler error)
    // by OOP design default restrict you to change the variable values.
    s1.setData("Sara", -5); // Validation prevents invalid data
    s1.showData();
}
```

Which function is changing the global data is difficult to trace,

3.3. Lack of Modularity

Lack of Modularity -Handling new data types is difficult

```
float totalSalary = 0;
void addSalary(float s)
{
    totalSalary += s;
}
void showSalary()
{
    cout << "Total Salary: " << totalSalary;
}
```

2. Now imagine we change the data structure

```
class Employee {
public:
    float totalSalary;
};
```

You must rewrite them like this:

```
void addSalary(Employee &e, float s) {
    e.totalSalary += s;
}

void showSalary(Employee e) {
    cout << "Total Salary: " << e.totalSalary;
}
```

data and its related functions are **bundled together** inside a **class**.

Changes are localized — modifying one class does not affect others.

```
class Employee {
```

```
private:
```

```
    float totalSalary;
```

```
public:
```

```
    void addSalary(float s) { totalSalary += s; }
```

```
    void showSalary() { cout << "Total Salary: " << totalSalary; }
```

```
};
```

1. Data structure changes are internal to the class.
2. The interface (addSalary, showSalary) remains the same.
3. Other parts of the code don't break when the internal variable changes.

Changes are localized — modifying one class does not affect others.

1. OOP combines data and functions into one unit called a class. This means: - Data is ****encapsulated**** and protected from unauthorized access
2. Each class manages its own internal state and exposes only controlled interfaces
3. Changes are localized — modifying one class does not affect others.

Sample Code:

```
//#pragma warning(disable : 4996)
#define _CRT_SECURE_NO_WARNINGS
#include <cstring>
#include<iostream>

Class Student
{
    char name[100];
    int age;
public:
    void setData(const char *n, int a)
    {
        if (n != nullptr)
            strcpy(name, n);

        if (a > 0)
        {
            age = a;
        }
    }
    char* getName()
    {
        return name;
    }

    void displayData()
    {
        cout << name << " " << age;
    }
};

int main() {

    Student s;
    s.setData("ali", 19);
    cout << s.getName();
    s.displayData();
    return 0;
}
```

5. Summary

- Procedural programming separates data and functions, leading to complex dependencies
- Maintenance becomes difficult because of global data, weak data protection, and lack of modular design
- Object-Oriented Programming integrates data and behavior in a single class, making software easier to maintain, extend, and debug
- OOP models real-world entities and encourages reusable, modular code structures

6. Home work

- 1. Create a class Rectangle with the attributes: length (float), width (float). Implement methods to:
 - - get the length and width of the rectangle
 - - set the length and width of the rectangle
 - - calculate the area of the rectangle
 - - calculate the perimeter of the rectangle
- 2. Create a class Students with the attributes: name (char *), roll no (char *), no of subjects (int), grades (array of int). Implement methods to:
 - - get the student's name, roll no, grades
 - - add a grade to the student's record
 - - calculate the average grade

3. complete the implemenatoin of the following class

```
class BankAccount
{ int balance;
public:
    BankAccount()...
    void deposit(int)...
    void withdraw(int)...
    void showBalance()...
};
```

```
int main() {
BankAccount acc;
acc.deposit(1000);
acc.withdraw(500);
acc.showBalance();
return 0;
}
```

Last classes

1. Mechanism for user defined types in C++ are
 - ▶ Structures
 - ▶ Classes
2. Built-in types are like int, float and double
3. User defined type can be
 - ▶ Student in student management system
 - ▶ Circle in a drawing software

```
class Student // encapsulation
{
    char *name;
    int rollNo;
    double cgpa;

public: // abstraction --- interface
    void setRollNo(int r)
    {
        if (a >= 0)
        {
            rollNo = r;
        }
        else
        {
            rollNo = 0;
        }
    }

    int getRollNo()
    {
        return rollNo;
    }

    void displayData()
    {
        cout << name << " " << rollNo << " " << cgpa << endl;
    }
};

int main() {
```

```

    Student s1;
    s1.RollNo = -20; // not allowed

    s1.setRollNo(20);

    return 0;
}

```

Today lecture:

1. Accessor Functions: getters and setters
2. Constructors
 - i. Constructor overloading
 - ii. Constructor Default argument
3. Destructor

Constructors

A constructor is a special member function:

- a non-parameterized constructor exists by default in every class
- automatically called when an object is created
- has the same name as the class
- has no return type, not even void
- we can't call a constructor other than object creation

```

// constructors

Student() // default constructor
{
    name = nullptr;

    rollNo = 0;

    cgpa = 0.0;
}

```

Constructor Overloading

A constructor can be overloaded:

- we can write one constructor to overwrite the default constructor
- we may write multiple constructors with different:
 - * parameter count
 - * parameter type
- Compiler matches the parameters and bind the closest match

```
Student(const char *n) // outsider just take course
```

```
{
    if (n != nullptr)
    {
        int len = strlen(n);
        name = new char[len + 1];
        strcpy(name, n);
    }
    else
    {
        name = nullptr;
    }
    rollNo = 0;
    cgpa = 0.0;
}
```

```
Student(const char* n, int a) //first semester student at University
```

```
{  
    if (n != nullptr)  
    {  
        int len = strlen(n);  
        name = new char[len + 1];  
        strcpy(name, n);  
    }  
    else  
    {  
        name = nullptr;  
    }  
    if (a >= 0)  
    {  
        rollNo = a;  
    }  
    else  
    {  
        rollNo = 0;  
    }  
    cgpa = 0.0;  
}
```

```
Student(const char* n, int a, double c) //migrated student from other University
```

```
{
    if (n != nullptr)
    {
        int len = strlen(n);
        name = new char[len + 1];
        strcpy(name, n);
    }
    else
    {
        name = nullptr;
    }
    if (a >= 0)
    {
        rollNo = a;
    }
    else
    {
        rollNo = 0;
    }
    if (c > 0 && c <= 4.0)
    {
        cgpa = c;
    }
    else
    {
        cgpa = 0.0;
    }
}
```

```
Student(const char* n = nullptr, int a = 0, double c = 0.0) //default argument
constructor
```

```
{
    if (n != nullptr)
    {
        int len = strlen(n);
        name = new char[len + 1];
        strcpy(name, n);
    }
    else
    {
        name = nullptr;
    }
    if (a >= 0)
    {
        rollNo = a;
    }
    else
    {
        rollNo = 0;
    }
    if (c > 0 && c <= 4.0)
    {

```

```

        cgpa = c;
    }
    else
    {
        cgpa = 0.0;
    }
}

```

Destructor

used to destroy objects and free resources when they go out of scope

- has the same name as the class, preceded by a tilde (~).
- takes no arguments and has no return type
- called automatically when an object's lifetime ends
- only one destructor is allowed per class (cannot be overloaded)
- in reverse order of object creation
- a default destructor exist in each class

```

~Student() // destructor
{
    if (name != nullptr)
    {
        cout << " ~Student() " << name << " is called!!" << endl;

        delete[] name;
        name = nullptr;
    }
    else
    {
        cout << "~Student () is called " << endl;
    }
}

```

```
#define _CRT_SECURE_NO_WARNINGS // as strcpy()etc function don't perform bounds
```

```
#checking
```

```
#include <cstring>
```

```
#include<iostream>
```

```
using namespace std;
```

```
class Student
```

```
{
```

```
    char *name;
```

```
    int rollNo;
```

```
    double cgpa;
```

```
public:
```

```
    /*
```

```
    // constructors overloading
```

```
    Student() // default constructor
```

```
    {
```

```
        name = nullptr;
```

```
        rollNo = 0;
```

```
        cgpa = 0.0;
```

```
    }
```

```
    Student(const char *n) // outsider just take course
```

```
    {
```

```
        if (n != nullptr)
```

```
        {
```

```
            int len = strlen(n);
```

```
            name = new char[len + 1];
```

```
            strcpy(name, n);
```

```
        }
```

```
    } else
```

```
    {
```

```
        name = nullptr;
```

```
    }
```

```
    rollNo = 0;
```

```
    cgpa = 0.0;
```

```
    }
```

```
    Student(const char* n, int a) //first semester student at University
```

```
    {
```

```
        if (n != nullptr)
```

```
        {
```

```
            int len = strlen(n);
```

```
            name = new char[len + 1];
```

```
            strcpy(name, n);
```

```
        }
```

```
    } else
```

```
    {
```

```
        name = nullptr;
```

```
    }
```

```
    if (a >= 0)
```

```
    {
```

```
        rollNo = a;
```

```
    }
```

```

        else
        {
            rollNo = 0;
        }
        cgpa = 0.0;
    }
    Student(const char* n, int a, double c) //migrated student from other
University
    {
        if (n != nullptr)
        {
            int len = strlen(n);
            name = new char[len + 1];
            strcpy(name, n);
        }
        else
        {
            name = nullptr;
        }
        if (a >= 0)
        {
            rollNo = a;
        }
        else
        {
            rollNo = 0;
        }
        if (c > 0 && c <= 4.0)
        {
            cgpa = c;
        }
        else
        {
            cgpa = 0.0;
        }
    }
    */

    Student(const char* n = nullptr, int a = 0, double c = 0.0) //default argument
constructor
    {
        if (n != nullptr)
        {
            int len = strlen(n);
            name = new char[len + 1];
            strcpy(name, n);
        }
        else
        {
            name = nullptr;
        }
        if (a >= 0)
        {
            rollNo = a;
        }
    }

```

```

        else
        {
            rollNo = 0;
        }
        if (c > 0 && c <= 4.0)
        {
            cgpa = c;
        }
        else
        {
            cgpa = 0.0;
        }
    }
}

```

```

int getRollNo()
{
    return rollNo;
}
void setRollno(int a)
{
    if (a >= 0)
    {
        rollNo = a;
    }
    else
    {
        rollNo = 0;
    }
}
void displayData()
{
    cout << name << " " << rollNo << " " << cgpa << endl;
}

~Student() // destructor
{
    if (name != nullptr)
    {
        cout << " ~Student() " << name << " is called!!" << endl;

        delete[] name;
        name = nullptr;
    }
    else
    {
        cout << "~Student () is called " << endl;
    }
}
}

```

```
};
```

```
int main() {
```

```
    Student s1;
```

```
    Student s2("Ahmad");
```

```
    Student s3("Ali", 1);
```

```
    Student s4("Moosa", 20, 3.7);
```

```
    s4.displayData();
```

```
    return 0;
```

```
}
```

Last Lecture lecture:

- Member function: Accessor Functions: getters and setters
- Constructors
 - Constructor overloading
 - Constructor Default argument
- Destructor

1. Constructors [for a valid state]

A constructor is a special member function:

- a non-parameterized constructor exists by default in every class
- automatically called when an object is created
- has the same name as the class
- has no return type, not even void
- we can't call a constructor other than object creation

```
#include <iostream>
#include <cstring>
using namespace std;

class Student {
    char *name;
    int rollNo;
    double cgpa;
public:
    // Simple version of Constructor with default arguments
    Student(const char *n = "Unknown", int r = 0, double c = 0.0) {
        name = new char[strlen(n) + 1];
        strcpy(name, n);
        rollNo = r;
        cgpa = c;
    }

    // Display function
    void display() {
        cout << "Name: " << name << endl;
        cout << "Roll No: " << rollNo << endl;
        cout << "CGPA: " << cgpa << endl;
    }

    // Destructor to free memory
    ~Student() {
        delete[] name; name= nullptr;
    }
};

int main() {
```

```

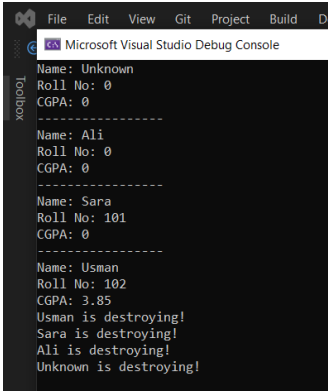
Student s1;                                // uses all default arguments
Student s2("Ali");                         // custom name, default rollNo & cgpa
Student s3("Sara", 101);                   // custom name & rollNo
Student s4("Usman", 102, 3.85);             // all custom values

s1.display();
cout << "-----\n";
s2.display();
cout << "-----\n";
s3.display();
cout << "-----\n";
s4.display();

return 0;
}

```

Output:



```

Name: Unknown
Roll No: 0
CGPA: 0
-----
Name: Ali
Roll No: 0
CGPA: 0
-----
Name: Sara
Roll No: 101
CGPA: 0
-----
Name: Usman
Roll No: 102
CGPA: 3.85
Usman is destroying!
Sara is destroying!
Ali is destroying!
Unknown is destroying!

```

Better Version of constructor with default arguments:

Constructor with default arguments(instead of writing overloaded constructor):

```

Student(const char* n = "Unknown", int a = 0, double c = 0.0)
{
    if (n != nullptr) {
        name = new char[strlen(n) + 1];
        strcpy(name, n);
    } else {
        name = new char[8];
        strcpy(name, "Unknown");
    }
    rollNo = (a >= 0) ? a : 0;
    cgpa = (c >= 0 && c <= 4.0) ? c : 0.0;
}

}

```

2. Destructor

Since Student class uses dynamic memory allocation (via `new char[...]` for name), you must define a destructor to free that memory and avoid memory leaks.

Destructor is used to destroy objects and free resources when they go out of scope

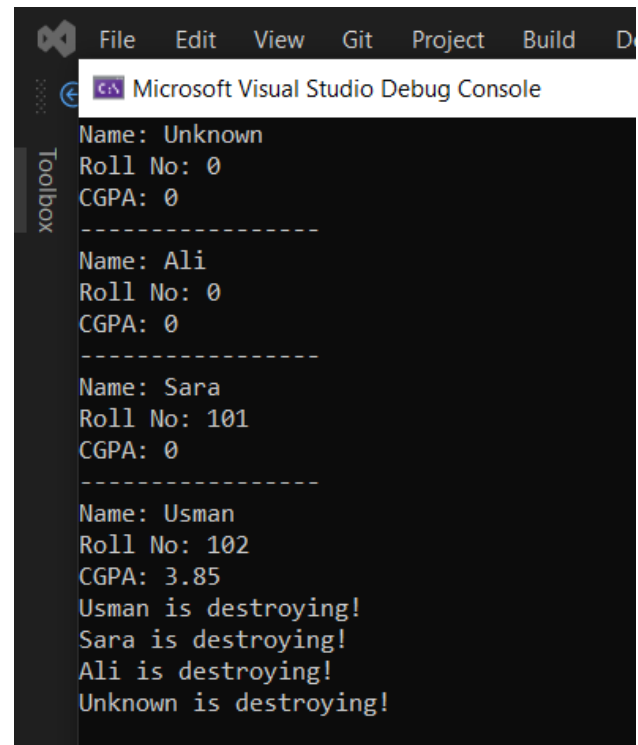
- has the same name as the class, preceded by a tilde (~).
- takes no arguments and has no return type
- called automatically when an object's lifetime ends
- only one destructor is allowed per class (cannot be overloaded)
- in reverse order of object creation
- a default destructor exists in each class

```
~Student() {  
    if (name) { // shorthand for (name != nullptr)  
        cout << "Destructor called for " << name << endl;  
        delete[] name;  
        name = nullptr;  
    }  
}
```

Today lecture :

- Sequence of execution:
- Dynamic creates object and access it through ->operator
- Reference variable
 - o Function call by reference
 - o Vs call by address
- Copy constructor

1. Sequence of execution:



```
Microsoft Visual Studio Debug Console

Name: Unknown
Roll No: 0
CGPA: 0
-----
Name: Ali
Roll No: 0
CGPA: 0
-----
Name: Sara
Roll No: 101
CGPA: 0
-----
Name: Usman
Roll No: 102
CGPA: 3.85
Usman is destroying!
Sara is destroying!
Ali is destroying!
Unknown is destroying!
```

2. Dynamically allocation of object and accessing throught ->

- Use of dot and -> for accessing member function/ data members of a class

```
Student *sp = new Student("Muhammad Moosa", 201, 3.5);

sp->setCGPA(3.8);
sp->displayData(); // (* sp).displayData();
```

3. Reference variable

- i. In C++, a **reference variable** is an **alias (another name)** for an existing variable. It does not create a new variable in memory — instead, it provides a new name to access the same memory location.

```
int x = 10, y = 20;
// Reference
int &r = x;
r = y; // assigns y's value to x (x = 20)

// Equivalent pointer behavior
int *const p = &x;
*p = y; // assigns y's value to x (x = 20)
```

// In short:

A C++ reference variable is **conceptually similar** to a const pointer (`int *const p = &x;`), but with cleaner syntax and automatic dereferencing handled by the compiler.

- ii. Function call mechanism with reference variable (Mostly used)

Example 1: Using a Reference Parameter	Equivalent Pointer Version
<pre>#include <iostream> using namespace std; // Function that uses a reference parameter void increment(int &num) { num = num + 1; // modifies the original variable directly } int main() { int x = 10; cout << "Before function call: " << x << endl; increment(x); // passes x by reference cout << "After function call: " << x << endl; return 0; }</pre> Output: Before function call: 10 After function call: 11	<pre>#include <iostream> using namespace std; void increment(int *num) { *num = *num + 1; // explicitly dereference the pointer } int main() { int x = 10; cout << "Before function call: " << x << endl; increment(&x); // explicitly pass the address of x cout << "After function call: " << x << endl; return 0; }</pre>

4. Copy constructor

- The Problem: Shallow Copy = Double Deletion + Dangling Pointer
- When the function ends, the destructor of the copy object runs first and deletes the shared memory.
- Then the original object's destructor also tries to delete that same memory → ❌
Runtime Error / Crash (double free).

```
#include <iostream>
#include <cstring>
using namespace std;

class Student {
    char *name;
    int rollNo;
    double cgpa;

public:
    // Constructor
    Student(const char *n, int r, double c) {
        name = new char[strlen(n) + 1];
        strcpy(name, n);
        rollNo = r;
        cgpa = c;
        cout << "Constructor called for " << name << endl;
    }

    // ⚠ No copy constructor defined here!
    // Compiler will generate a shallow one automatically.

    void display() const {
        cout << "Name: " << name << ", Roll No: " << rollNo << ", CGPA: " << cgpa <<
endl;
    }

    // Destructor
    ~Student() {
        cout << "Destructor called for " << name << endl;
        delete[] name; // Both copies try to delete the same memory!
    }
};

// Function using call by value
void showStudent(Student s) {
    cout << "Inside showStudent() function:\n";
    s.display();
}

int main() {
    Student s1("Ali", 101, 3.7);
    cout << "\nCalling showStudent(s1)...\n";
    showStudent(s1); // ❌ Shallow copy made here
```

Constructor called for Ali

Calling showStudent(s1)...

Inside showStudent() function:

Name: Ali, Roll No: 101, CGPA: 3.7

Destructor called for Ali <-- temporary object
destroyed here (frees memory)

Back in main()

Destructor called for Ali <-- original also deletes s
memory ❌

*** Error: double free or corruption detected ***

```

    cout << "\nBack in main()\n";
    s1.display(); // May crash here

    return 0;
}

```

Solution:

```

Student(const Student &other) {
    cout << "Copy constructor called!" << endl;

    // Safe deep copy of name
    if (other.name != nullptr) {
        name = new char[strlen(other.name) + 1];
        strcpy(name, other.name);
    } else {
        name = nullptr;
    }

    //Direct copy (assuming proper validation in constructors)
    rollNo = other.rollNo;
    cgpa   = other.cgpa;
}

// show the demo with the following function:
// Function using call by value
void showStudent(Student s)
{
    cout << "Inside showStudent() function:\n";
    s.display();
}

Student createStudent()
{
    Student s("Ahmad", 102, 3.8);
    return s; // Return by value
}

```

```

int main() {

    Student s4("Moosa", 20, 3.7);

    Student s5 = s4;

    s4.displayData();
    fun(s5);
    s5.displayData();

    Student s6 = createStudent();

    s6.displayData();

    return 0;}

```

Home work:

Problem Statement: String Class Implementation

In this task, you are required to **design and implement a user-defined data type (ADT)** named **String**, which mimics the basic functionality of C++ string handling. The purpose of this exercise is to help you understand **dynamic memory allocation, constructors, destructors, and operator overloading** in Object-Oriented Programming (OOP).

Specifications:

1. The class should contain a private data member:
 - `char *buff;` — a dynamically allocated character array used to store the string.
2. Implement the following member functions:
 - **Default Constructor:** Initializes the string as an empty string (or NULL pointer).
 - **Parameterized Constructor:** Accepts a C-style string and dynamically allocates memory to store it.
 - **Copy Constructor:** Creates a new object as a deep copy of another String object.
 - **Destructor:** Releases any dynamically allocated memory to avoid memory leaks.
 - **set() Function:** Allows setting or modifying the value of the string dynamically.
 - **display() Function:** Displays the current string on the console.

Objectives:

- To understand the concept of **dynamic memory allocation** in classes.
- To learn how to implement a **deep copy using a copy constructor**.
- To explore **constructors and destructors** in managing object lifecycles.

Expected Output Example:

Enter first string: Hello

Enter second string: World

String 1: Hello

String 2: World

After copying using copy constructor:

String 3 (copied from String 1): Hello

Copy Constructor:

```
#define _CRT_SECURE_NO_WARNINGS // as strcpy()etc function don't perform bounds
checking
#include <iostream>
#include <cstring>
using namespace std;

class Student {
    char* name;
    int rollNo;
    double cgpa;

public:
    Student(const char* n = "Unknown", int r = 0, double c = 0.0) {
        name = new char[strlen(n) + 1];
        strcpy(name, n);
        rollNo = r;
        cgpa = c;
    }
    Student(const Student& other) {
        cout << "Copy constructor called for " << other.name << endl;
        name = new char[strlen(other.name) + 1];
        strcpy(name, other.name);
        rollNo = other.rollNo;
        cgpa = other.cgpa;
    }
    ~Student()
    {
        cout << "Destructor called for " << name << endl;
        delete[] name;
    }
    // Display function
    void display() {
        cout << "Name: " << name << endl;
        cout << "Roll No: " << rollNo << endl;
        cout << "CGPA: " << cgpa << endl;
    }
};

// show the demo with the following function:
void displayData(Student s)
{
    cout << "Inside void fun(Student s) ";
    s.display();
}
```

```

Student createStudent()
{
    Student s("Ahmad", 102, 3.8);
    return s; // Return by value
}

int main() {

    Student s1("Usman", 102, 3.85);

    //Student s2 = s1;
    // displayData(s2);
    Student s3= createStudent(); //

    return 0;
}

```

What is this Pointer?

In C++, the this pointer is an implicit (hidden) pointer that is automatically passed to all non-static member functions of a class.

It points to the object that invoked the member function.

In simple words:

Inside any non-static member function, this represents the address of the current object.

Key Points

1. this is **automatically available** inside all non-static member functions.
2. this is **a pointer** (so you use -> to access members).
3. this is **not available** in static member functions.
4. It helps when:
 - The this pointer allows a single shared copy of each member function to **correctly access the data of the calling object**, even when multiple objects exist.
 - You need to **return the current object** (used in operator overloading or function chaining).
 - You need to **distinguish between data members and parameters** with the same name.

```

Student s1, s2;
s1.setRollNo(101); // setRollNo(&s1, 101);
s2.setRollNo(202);

```

```

s1.setRollNo(101); //setRollNo(&s1, 101);
s2.setRollNo(202); //setRollNo(&s2, 202);

```

Sample code:

```
#include <iostream>
using namespace std;
class Student {
private:
    int rollNo;

public:
    void setRollNo(int r) {
        this->rollNo = r;
    }

    void show() {
        cout << "Inside void show()" << endl;
        cout << "Object at: " << this << " has Roll No: " << rollNo << endl;
    }
};

int main() {
    Student s1, s2,s3,s4;
    cout << "Inside main";
    cout << "sizeof(s2)" << sizeof(s2) << endl;;
    cout << " &s1 " << &s1 << endl;
    cout << " &s2 " << &s2 << endl;
    cout << " &s2 -&s1" << &s2 - &s1 << endl;
    cout << " &s3 -&s2" << &s3 - &s2 << endl;
    cout << " &s4 -&s3" << &s4 - &s3 << endl;
    s1.setRollNo(101);
    s2.setRollNo(202);

    s1.show();
    s2.show();

    return 0;
}
```

Today lecture

What is a Multi-File C++ Program?

1. A **multi-file program** means splitting your code into **multiple source files** (usually .h and .cpp files) instead of writing everything inside one main.cpp.
2. **Example structure:**

Student.h → Declarations (blueprint)

Student.cpp → Definitions (implementation)

main.cpp → Uses the class

3. Benefits of Using Multi-File C++ Programs

A. Modularity (Organized Code)

Each class or module is in its own pair of files — .h for interface and .cpp for implementation.

Example:

If you have classes Student, Teacher, and Course, each can live in its own files.

This keeps the code base neat and easy to navigate.

B. Team Collaboration

In a large software team, multiple developers can work on different modules simultaneously:

- One works on Student.cpp
- Another works on Teacher.cpp
- Another works on main.cpp

No one overwrites each other's code — this supports **parallel development**.

C. Separation of Interface (.h) and Implementation (.cpp)

.h (header) = Interface — tells “what” the class or function does.

.cpp = Implementation — shows “how” it does it.

```
char name[50]; // character array for storing name
```

```
update it
```

```
char *name
```

still the interface to interact with name is setName("Ahmad")

This separation improves readability and makes your program easier to understand and maintain.

4. Encapsulation & Information Hiding

The user (e.g., main.cpp) only includes the **header file** and doesn't see the implementation details inside .cpp.

Users can use the class without needing to know how it works internally — similar to how you use a library like <iostream>.

5. Easier Debugging and Maintenance

Each module (e.g., Student.cpp) can be tested and debugged **independently**, without affecting other parts of the project.

If something breaks, you know exactly which file to fix.

Multi-File C++ Program:

Project5

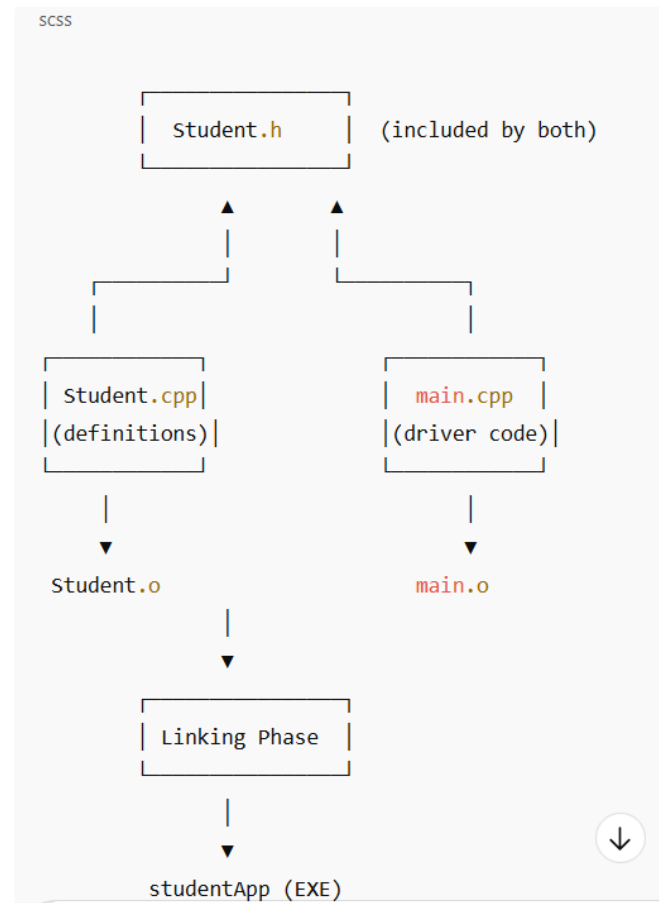
Header Files

└─ Student.h

Source Files

└─ Student.cpp

└─ main.cpp



Include **only the header file in main.cpp**; including the .cpp file causes duplicate definitions and linker errors.

Including Files in a Multi-File C++ Program

- In a multi-file program, **only the header file** (e.g., Student.h) should be included in main.cpp.
- The **.cpp implementation file** (e.g., Student.cpp) should **never** be included directly in main.cpp.

Reason:

When Student.cpp is compiled, it already contains the function definitions of the Student class. If you also include Student.cpp in main.cpp, the compiler will insert the same definitions into main.cpp as well — causing **duplicate definitions** of the class and its member functions.

Result:

The **linker** detects these duplicates and throws an error such as:

#pragma once helps to avoid the multiple inclusion of the same header within that one file's compilation,

Student.h

#pragma once //Prevent multiple inclusions of the same header file in a single compilation unit — a situation known as a “multiple inclusion” or “redefinition” error.

// compiler directive that tells the C++ compiler: “Include this header file only once per compilation — even if it’s included multiple times in different parts of the program.”

#define _CRT_SECURE_NO_WARNINGS // as strcpy()etc function don’t perform boundschecking

#include <iostream>

#include <cstring>

using namespace std;

class Student {

private:

int rollNo;

char name[50];

public:

Student(int r = 0, const char* n = "Unknown");

void display() const; // const function declaration

};

Student.cpp

#include "Student.h"

Student::Student(int r, const char* n) {

rollNo = r;

strcpy(name, n);

}

```
void Student::display() const { // const in definition

    cout << "Roll No: " << rollNo << ", Name: " << name << endl;

}
```

Driver.cpp

```
#include "Student.h"
```

```
int main() {

    const Student s1(101, "Ali");

    s1.display(); // works because display() is const

    return 0;

}
```

Need of Pragma once

```
// Student.h
class Student {};
```

```
// Account.h
#include "Student.h"
class Account {};
```

```
// main.cpp
#include "Student.h"
#include "Account.h"
int main() { ... }
```

When the compiler compiles main.cpp, it performs **preprocessing** — it copies and pastes the contents of all included headers into one giant file before actual compilation.

```
// (expanded version during preprocessing)
class Student { ... }; // from first include
class Student { ... }; // again from Account.h include ⚠️ DUPLICATE
class Account { ... };
int main() { ... }
```

// Now you get a **redefinition error** — because Student appears twice in the *same compilation unit*.

Solution is #pragma once fixes it

If you add #pragma once at the top of Student.h:

```
#pragma once
class Student { ... };
```

```
// after preprocessing
class Student { ... }; // from the first include only ✅
class Account { ... };
int main() { ... }
```

1. Use of this

```
Student s1;  
Student s2;;  
s2 = s1.setName("ali");  
Student s2 = s1.setName("ali");  
Student s2 = s1.setName("ali").setRollNo(2);
```

2. const key

a. const member function

- Compiler helps us to avoid our own mistake
- `bool isRollNo(int r)`
{
 If (rollNo == r){ return true;} // no error but it is logical error

}
- Solution:
- `bool isRollNo(int r) const // const Student * const this;`
{
 If (rollNo == r){ return true;}
}

b. return the const

```
const char * getName() const // to facility information hiding  
{ return name}
```

```
main()  
{  
    Student s1;  
    const char *n=s1.getName(); // main only read the name  
  
}
```

Note:

- a) const key word with the function make the this as a constant pointer:
`const Student *const this;`
- b) Constant function can't call the non constant member function

3. const data members

i. member initialization list

b. const objects

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
using namespace std;

class Student
{
    char* name;
    const int rollNo;
public:
    Student(const char* n = NULL, int r = 0);
    ~Student();
    bool isRollNo(int) const;
    int getRollNo() const;
    void display()const;
};

Student::Student(const char* n, int r) :rollNo(r)
{
    if (n != NULL)
    {
        int len = strlen(n);
        this->name = new char[len + 1];
        strcpy(this->name, n);
    }
    //rollNo = r; // ERROR
    cout << name << " constructor " << endl;
}

Student::~~Student()
{
}
```

```

        cout << name << " Destructor " << endl;
        if (name)
        {
            delete[] name;
            name = NULL;
        }
    }
    void Student::display() const
    {
        cout << name << " " << rollNo << endl;
    }
    //int Student::getRollNo(const Student* const this) const // compiler created code

    int Student::getRollNo() const
    {
        return rollNo;
    }
    //bool Student::isRollNo(int r, const Student *const this) const // compiler created code
    bool Student::isRollNo(int r) const
    {
        //display();
        if (this->rollNo == r) // rollNo is read only
            return true;
        else
            return false;
    }

    int main()
    {
        Student s1("moosa", 1);
        s1.display();
        const Student s2("ahmad", 2);
        s2.display();
    }

```

4.

const Objects

const objects can access only const member functions so chances of change of state of const objects once they are created are eliminated.

We make getRollNo function constant so that we can access it using constant objects,

```
#include <cstdlib>
#include <iostream>
using namespace std;
class Student
{
    int rollNo;
public:
    int getRollNo()
    {
        return rollNo;
    }
    void display()const
    {
        Cout<< rollNo;
    }

};

int main()
{const Student aStudent(5);
int a = aStudent.getRollNo(); //error
aStudent.display() ; // ok
```

Constant member functions

Make all functions that don't change the state of the object constant This will enable constant objects to access more member functions

Today lecture:

- static variable
- static data member
- static member function

1. Static variable is only one time initialized.

```
void fun1(int i)
{
    static int svar= i;
    cout << svar << endl;
    svar = 10;
}
```

```
int main()
{
    fun1(1);
    fun1(2);
    return 0;
}
```

2. Static data member :

- A variable that part of a class, yet is not part of an object of that class, is called static data member
- They are shared to all instance of class
- They don't belong to any particular instance of a class

Student s1, s2, s3; // static variable got memory once, even though we create 3 objects
// they got the space in memory at the time of definition outside the class

A. Mechanics of static data member:

- Key word static is used to make a data member static
- Static member is declared inside the class
- But they are defined outside the class

Syntax:

a. declaration

class className

{

Static DataType variableName; // declaration

};

b. definition/ initialization only once at time of definition in a file scope

dataType className :: variableName = 0; // definition or initialization

private data member cannot be accessed outside the class except static variable

Question is it violation of information hiding

Answer: No, it is definition or initialization and it happen only once

Default value: if we don't assign any value at the time of initialization, compiler put the default value of the variable

```
using namespace std;
class Student
{
    int rollNo;
    static int noOfStudent;

public:
};

// only static private data member can be accessible
// outside the class for initialization
int Student::noOfStudent = 0; //
```

is equivalent to

```
int Student::noOfStudent; //
```

How to access the static variable:

- i. access like the normal variable
- ii. access using scope resolution operator

```
int main()
{
    Student s1;
    cout << s1.noOfStudent;
    cout << Student::noOfStudent; // belong to class
}
```

Life of static data members:

- i. They are created even when on object is created
- ii. They remain in memory even when all object of a class are destroyed.

B. USE/ motivation of static variable

1. They can be used to store the information that is required by all objects like global vairables
2. Global variable within the class is called static variable as they are accessible by all objects.

Example: Modify the class student such that one can know the no of student created in a system.

```
class Student
{
    int rollNo;
```

```

public:
    static int noOfStudent;

    static int getNoOfStudent()
    {
        //rollNo = 100;
        return noOfStudent;
    }
    Student(int i = 0)
    {
        cout<< "No of constructed objects " << ++noOfStudent;
        cout << "\nDefault constructor " << endl;
    }
    ~Student()
    {
        //noOfStudent--;
        cout << "Destructor " << noOfStudent--<< endl;
    }
    void display() const
    {
        cout << "display " << endl;
    }
};

int Student::noOfStudent; //

int main()
{
    Student s1;
    cout << s1.noOfStudent;
    cout << Student::noOfStudent;
}

```

Problem: no of student is accessible outside the class

It is bad design to kept public the local variables

Solution :

static member function :

- i. to access the private static variable
- ii. They can be accessed using class name
- iii. They belong to class not objects
- iv. Accessing mechanism is same as the static data members

Note: they can not be accessed the non static data member of class : as this pointer is not passed to static member function

```

static int getNoOfStudent()

```

```

{
    //rollNo = 100; // they can not
    return noOfStudent;
}

```

Example of using static member functions:

```

class Student
{
    int rollNo;
    static int noOfStudent;
public:
    static int getNoOfStudent()
    {
        //rollNo = 100; // they can not
        return noOfStudent;
    }
    Student(int i = 0)
    {
        cout<< "No of constructed objects " << ++noOfStudent;
        cout << "\nDefault constructor " << endl;
    }
    ~Student()
    {
        //noOfStudent--;
        cout << "Destructor " << noOfStudent--<< endl;
    }
    void display() const
    {
        cout << "display " << endl;
    }
};

int Student::noOfStudent; //

int main()
{
    Student s1;
    cout << s1.getNoOfStudent();
    cout << Student::getNoOfStudent();
}

```

Array of objects:

```

Student s1[2]; // only possible of default constructor is defined in class either by
compiler or programmer
// if only parameterized constructor is define the there is an error

// solution // call explicitly the specific constructor

```

```
Student s1[2] = {Student(1), Student(1)}; // only able to make few no of objects
```

or

```
Student s1, * ptr;  
ptr = new Student (1);
```

```
ptr ->getName()
```

// but for larger array we are restricted to call the default constructor

```
Student s1[100]
```

Or

```
Ptr = new Student [100]
```

```
Cout <<prt[0] - > getName()
```

Assignment

Objective

Implement the following classes in C++ using concepts of constructors, destructors, static data members, dynamic memory allocation, and const correctness.

Task 1: Class String

Implement a class that manages a dynamic character string using a pointer.

```
class String {
```

```
private:
```

```
    char* ptr; // Pointer to dynamically allocated character array
```

```
public:
```

```
    String();           // Default constructor
```

```
    String(const String&); // Copy constructor
```

```
    void setString(const char*); // Setter function
```

```
    const char* getString() const; // Getter function returning const pointer
```

```
    ~String();           // Destructor
```

```
};
```

Requirements

1. Use dynamic memory allocation for handling the string.

2. Implement deep copy behavior in the copy constructor.
3. Properly deallocate memory in the destructor to prevent memory leaks.
4. Demonstrate the use of const with the getter function.

Sample Driver Program

```
#include <iostream>
#include <cstring>
#include "String.h"
using namespace std;

int main() {
    String s1;
    s1.setString("Hello World!");
    cout << "s1: " << s1.getString() << endl;

    String s2 = s1; // Copy constructor
    cout << "s2 (copied from s1): " << s2.getString() << endl;

    s1.setString("New String");
    cout << "After modifying s1: " << endl;
    cout << "s1: " << s1.getString() << endl;
    cout << "s2 (unchanged): " << s2.getString() << endl;

    return 0;
}
```

Task 2: Class Date

Implement a class that represents a calendar date and supports various date operations.

```
class Date {
private:
    int day, month, year;
    static Date defaultDate; // Static member representing a default date
```

public:

void setDefaultDate(int aDay, int aMonth, int aYear);

void setDay(int aDay);

int getDay() const;

void addDay(int x);

void setMonth(int aMonth);

int getMonth() const;

void addMonth(int x);

void setYear(int aYear);

int getYear() const;

void addYear(int x);

bool leapYear(int x) const;

Date(int aDay, int aMonth, int aYear); // Parameterized constructor

void setDate(int aDay, int aMonth, int aYear);

~Date(); // Destructor

};

Requirements

1. Implement static member defaultDate and initialize it appropriately.
2. Validate the range of day, month, and year values.
3. Implement helper functions such as addDay(), addMonth(), and addYear().
4. Include a function leapYear(int) to determine leap years.

Sample Driver Program

```
#include <iostream>
```

```
#include "Date.h"
```

```
using namespace std;
```

```
int main() {
```

```
    Date d1(15, 8, 2024);
```

```
cout << "Initial Date: " << d1.getDay() << "/" << d1.getMonth() << "/" << d1.getYear() << endl;

d1.addDay(5);
cout << "After adding 5 days: " << d1.getDay() << "/" << d1.getMonth() << "/" << d1.getYear() << endl;

d1.addMonth(2);
cout << "After adding 2 months: " << d1.getDay() << "/" << d1.getMonth() << "/" << d1.getYear() << endl;

d1.addYear(1);
cout << "After adding 1 year: " << d1.getDay() << "/" << d1.getMonth() << "/" << d1.getYear() << endl;

cout << "Leap Year (2024): " << (d1.leapYear(2024) ? "Yes" : "No") << endl;

return 0;
}
```

Submission Guidelines

- Submit the following files:
 - String.h and String.cpp
 - Date.h and Date.cpp
 - Driver file (main.cpp)
- Ensure your code is properly indented and commented.
- Bonus marks for handling invalid inputs gracefully.

Composition

- code reusability
- code simplicity
- ease to understand

//1. Implement student class

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <cstring>
using namespace std;
class Student
{
    char* name;
    //String name;
    int rollNo;
public:

    Student(const char* n = NULL, int r = 0)//: name(n)
    {
        cout << "Student default parameterized constructor" << endl;
        rollNo = r;
        setName(n);
        //name.setBuff(n);
    }
    Student(const Student& s)//:name(s.name)
    {
        cout << "Copy constructor is called " << endl;
        rollNo = s.rollNo;
        setName(s.getName());
        //name.setBuff(s.name.getBuff());
    }
    /*
    Student abc() // demo of *this
    {
        cout << "Demo of calling of copy constructors" << endl;
        return *this;
    }
    */
    void setName(const char* n)
    {
        if (name != NULL) // handle memory leakage
        {
            delete[] name;
            name = NULL;
        }
        if (n != NULL) // handle dangling pointer issue - deep copy
        {
            name = new char[strlen(n) + 1];
            strcpy(name, n);
        }
        //name.setBuff(n);
    }
    const char* getName()const
```

```

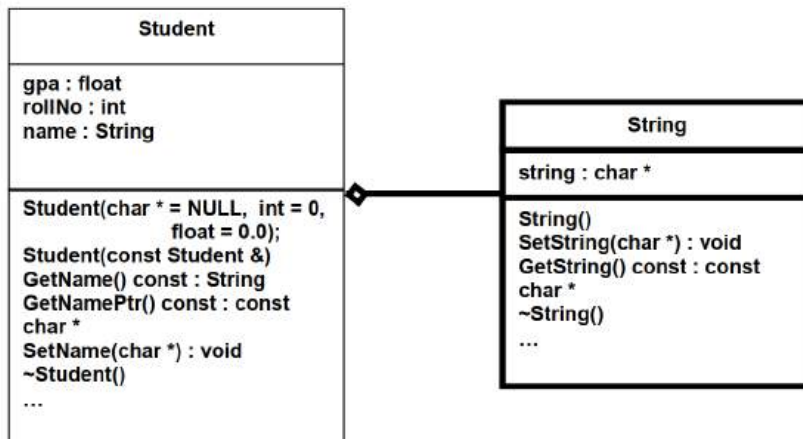
    {
        return name;
        //return name.getBuff();
    }
    ~Student()
    {
        cout << "Student destructor" << endl;
        if (name != NULL) // to handle memory leakage
        {
            delete[] name;
            name = NULL;
        }
    }
    void display()
    {
        cout << "Stduent data : "<< name<< " " << rollNo << endl;
        //cout << "Stduent data : " << name.getBuff() << " " << rollNo << endl;
    }
};

int main()
{
    Student s1("Moosa", 1);
    s1.display();
    s1.setName("Muhammad Moosa");
    s1.display();

    // block to demo copy constructor
    {
        Student s2 = s1;
        s2.display();
    }
    s1.display();
    return 0;
}
2.

```

UML notation for OOP modeling



```
;;;;;;;;;;
```

```
//2. Implement the String class
```

```
#define _CRT_SECURE_NO_WARNINGS
```

```
#include <iostream>
```

```
using namespace std;
```

```
class String
```

```
{
```

```
    char* buff;
```

```
public:
```

```
    String(const char* b= NULL)
```

```
    {
```

```
        cout << " String default argument constructor" << endl;
```

```
        if (b != NULL) //if (strlen(b)> 0)
```

```
        {
```

```
            setBuff(b);
```

```
        }
```

```
        else
```

```
        {
```

```
            cout << "Empty string" << endl;;
```

```
        }
```

```
    }
```

```
    String(const String &s) // copy constructor
```

```
    {
```

```
        cout << "String copy constructor" << endl;
```

```
        setBuff(s.buff);
```

```
    }
```

```
    void setBuff(const char * b)
```

```
    {
```

```
        if (buff != NULL) //to handle memory leakage
```

```
        {
```

```
            delete[] buff;
```

```
            buff = NULL;
```

```
        }
```

```
        if (b != NULL) // to handle dangling pointer issue
```

```
        {
```

```
            buff = new char[strlen(b) + 1];
```

```
            strcpy(buff, b);
```

```
        }
```

```
    }
```

```
    const char *getBuff()const
```

```
    {
```

```
        return buff;
```

```
    }
```

```
    ~String ()
```

```
    {
```

```
        cout << "String class Destructor " << endl;
```

```
        if (buff != NULL)
```

```
        {
```

```
            delete[] buff;
```

```
            buff = NULL;
```

```
        }
```

```
    }
```

```
};
//
;;;;;Now the updated the student class by composing the name as object of String class
;;;;;; use the setting and getters....
```

```
class Student
{
    //char* name;
    String name;
    int rollNo;
public:

    Student (const char *n = NULL, int r = 0)//: name(n)
    {
        cout << "Student default paramterized constructor" << endl;
        rollNo = r;
        //setName(n);
        name.setBuff(n);
    }
    Student (const Student & s)
    {
        cout << "Copy constructor is called " << endl;
        rollNo = s.rollNo;
        //setName(s.getName());
        name.setBuff(s.name.getBuff());
    }
    /*
    Student abc() // demo of *this
    {
        cout << "Demo of calling of copy constructors" << endl;
        return *this;
    }
    */
    void setName (const char * n)
    {
        /*
        if (name != NULL) // handle memory leakage
        {
            delete[] name;
            name = NULL;
        }
        if (n != NULL) // handle dangling pointer issue
        {
            name = new char[strlen(n) + 1];
            strcpy(name, n);
        }
        */
        name.setBuff(n);
    }

    const char * getName()const
    {
        //return name;
        return name.getBuff();
    }
    ~Student()
    {
        cout << "Student destructor" << endl;
    }
}
```

```

        /*if (name != NULL) // to handle memory leakage
        {
            delete[] name;
            name = NULL;
        }*/
    }
    void display()
    {
        //cout << "Stduent data :"<< name<< " " << rollNo << endl;
        cout << "Stduent data :"<< name.getBuff()<< " " << rollNo << endl;
    }

};

int main()
{
    Student s1("Moosa", 1);
    s1.display();
    s1.setName("Muhammad Moosa");
    s1.display();

    // block to demo copy constructor
    {
        Student s2 = s1;
        s2.display();
    }
    s1.display();
    return 0;
}

```

//3 Improve the idea of composition by using member initialization list

- It reduce the function calling overhead:

```

class Student
{
    String name;
public:
    Student(const char* p = NULL, int y = 2001, int m = 1, int d = 1) : name(p)
    {
        cout << "Student(const char* p = NULL, int y = 0, int m = 0, int d =0)" <<
endl;
        // No need to call the setters and getters as they are overhead
        //name.setPtr(p);
    }
    ~Student()
    {
        cout << "~Student()" << endl;
    }
    Student(const Student& obj): name(obj.name)
    {
        cout << "Student(const Student & obj)" << endl;
    }
    void display() const

```

```

    {
        cout << "Student::display()" << endl;
        name.display();
    }
};

int main()
{
    Student s1("ali,", 2005, 04, 06);

    //Student s2 = s1;
    s1.display();
}

```

// 4. final version

```

#pragma warning(disable : 4996)
#include <iostream>
using namespace std;
class String
{
private:
    char* ptr;
    int size;
public:
    String(const char* p = NULL)
    {
        cout << "String(const char* p = NULL)" << endl;
        if (p != NULL)
        {
            ptr = new char[strlen(p) + 1];
            strcpy(ptr, p);
        }
        else
        {
            ptr = NULL;
            size = 0;
        }
    }

    String(const String& obj)
    {
        cout << "String (const String & obj)" << endl;
        if (obj.ptr != NULL)
        {
            ptr = new char[strlen(obj.ptr) + 1];
            strcpy(ptr, obj.ptr);
        }
    }
    ~String()
    {
        cout << "~String()" << endl;
        if (ptr != NULL)
        {
            delete[] ptr;
            ptr = NULL;
        }
    }
}

```

```

}
void setPtr(const char* p)
{
    cout << "void setPtr(const char* p)" << endl;
    if (ptr != NULL) // just like destructor code
    {
        delete[] ptr;
        ptr = NULL;
    }
    if (p != NULL) // just like default parameterized constructor's code
    {
        ptr = new char[strlen(p) + 1];
        strcpy(ptr, p);
    }
}

const char* getPtr()const
{
    return ptr;
}

String abc(String s)
{
    cout << "String abc()" << endl;
    return *this;
}
void display()const
{
    cout << "String ::void display()" << endl;
    cout << ptr << endl;
}
};

```

```

class Date
{
    int day, month, year;
public:
    Date(int d = 1, int m = 1, int y = 2001)
    {
        cout << "Date(int d = 1, int m = 1, int y = 2001)" << endl;
        day = d;
        month = m;
        year = y;
    }
    Date(const Date& obj)
    {
        cout << "Date(const Data & d)" << endl;
        day = obj.day;
        month = obj.month;
        year = obj.year;
    }
    ~Date()
    {
        cout << "~Date()" << endl;
    }
}

```

```

void setDate(int y , int m , int d )
{
    cout << "setDate(int d , int m, int y )" << endl;
    day = d;
    month = m;
    year = y;
}
void display() const
{
    cout << "Date::display()" << endl;
    cout << day << " " << month << " " << year << endl;
}

};

class Student
{
    String name;
    Date dob;
public:
    Student(const char* p = NULL, int y = 2001, int m = 1, int d = 1) : name(p),
    dob(y, m, d)
    {
        cout << "Student(const char* p = NULL, int y = 0, int m = 0, int d =0)" <<
endl;
        // No need to call the setters and getters as they are overhead
        //name.setPtr(p);
        //dob.setDate(y, m, d);
    }
    ~Student()
    {
        cout << "~Student()" << endl;
    }
    Student(const Student& obj): name(obj.name), dob(obj.dob)
    {
        cout << "Student(const Student & obj)" << endl;
    }
    void display() const
    {
        cout << "Student::display()" << endl;
        name.display();
        dob.display();
    }
};

int main()
{
    Student s1("ali,", 2005, 04, 06);

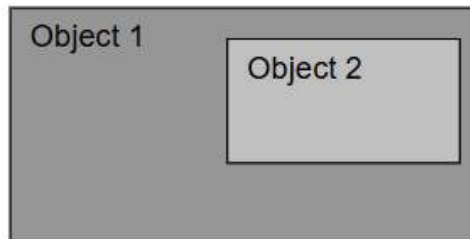
    //Student s2 = s1;
    s1.display();
    //String *s = new String;
    //String s1, s2;
    //s1.abc(s2);
    //cout << "Helllloo..." << endl;
    //String s1 ("ahmad");
    //String s2 = s1;
    //s2.setPtr("Moosa");
}

```

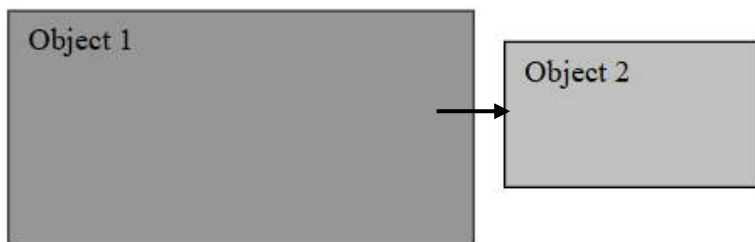
```
    //cout << s2.getPtr() << endl;

    //delete s;
    //s = NULL;
    return 0;
}
```

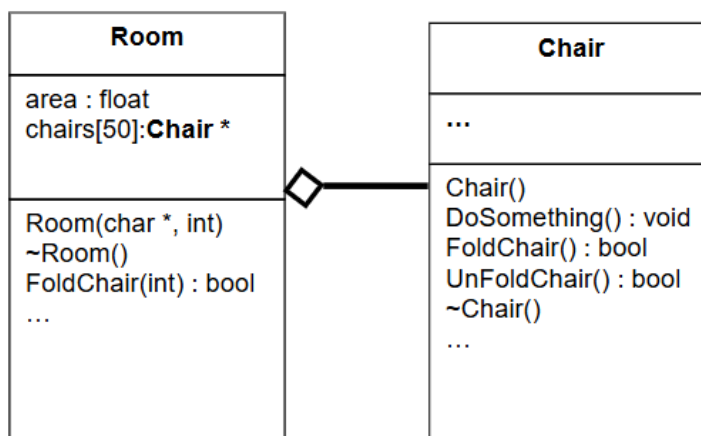
Aggregation in C++ (Simple Explanation)



Composition



Aggregation



Aggregation is a “has-a” relationship between classes where:

- One class contains a reference or pointer to another class
- The contained object is created outside and passed in
- The lifetime of the part (contained object) is independent of the whole

➡ If the container object is destroyed, the contained object still exists.

This is different from composition, where the container *owns* the object and controls its lifetime.

Real-life Example

- A Student *has-a* Course, but the Course exists independently.
- A student joining or leaving does not delete the course.

☐ Student aggregates Course by storing a pointer to a Course object

☐ The Course object is created outside Student → *true aggregation*

Complete Example (Aggregation + char array)

```
#define _CRT_SECURE_NO_WARNINGS
#pragma warning(disable : 4996)
#include <iostream>
using namespace std;
class Course {
private:
    char courseName[50];

public:
    Course(const char* name) {
        strcpy(courseName, name);
    }

    void showCourse() const {
        cout << "Course: " << courseName << endl;
    }
    ~Course()
    {
        cout << "Destructor of course" << endl;
    }
};

class Student {
private:
    char name[50];
    Course* course;    // Aggregation (pointer to external object)

public:
    // Constructor receives a pointer to Course (not created inside Student)
    Student(const char* n, Course* c) : course(c) {
        strcpy(name, n);
    }
};
```

```

    }

    void showStudent() const {
        cout << "Student: " << name << endl;
        course->showCourse();
    }
    ~Student()
    {
        cout << "Destructor of Student" << endl;;
    }
};

int main() {
    // Independent object (lifetime NOT controlled by Student)
    Course c1("Object-Oriented Programming");
    // Student aggregates Course (has-a relationship)
    {
        Student s1("Ali", &c1);
        s1.showStudent();
    }
    c1.showCourse();
    return 0;
}

```

(These are "HAS-A" relationships where the part can exist independently.)

Here are **three more clear and simple examples of aggregation in C++**, all different from the previous Student–Course example.

(These are "HAS-A" relationships where the part can exist independently.)

Example 2:

In aggregation, the **Room does not own the Chairs**—they are created **outside** and passed into the Room.

C++ Example: Room Aggregates Chairs

```

#include <iostream>
using namespace std;
class Chair {
private:
    int id;

public:
    Chair(int cId) : id(cId) {}

    void show() const {

```

```

        cout << "Chair ID: " << id << endl;
    }
    ~Chair()
    {
        cout << "Destructor of Chair " << endl;
    }
};

class Room {
private:
    Chair* chairs[10]; // aggregation: Room has Chairs
    int count;

public:
    Room() : count(0) {}

    // Room does NOT create Chairs – just stores their pointers
    void addChair(Chair* c) {
        if (count < 10) {
            chairs[count++] = c;
        }
    }

    void showRoom() const {
        cout << "Room contains " << count << " chairs:" << endl;
        for (int i = 0; i < count; i++) {
            chairs[i]->show();
        }
    }
    ~Room()
    {
        cout << "Destructor of Room ()" << endl;
    }
};

int main() {
    // Chairs exist independently (created outside Room)
    Chair c1(101);
    Chair c2(102);
    Chair c3(103);
    {
        Room r1;                // Room object
        r1.addChair(&c1);        // Aggregation
        r1.addChair(&c2);
        r1.addChair(&c3);

        r1.showRoom();
    }
}

```

```
    }  
    c1.show();  
    return 0;  
}
```

✓ Why This Is Aggregation?

- Chairs are **created outside** the Room.
 - Room **only keeps pointers** to Chairs.
 - If the Room is destroyed, the Chairs **still exist**.
 - This is a **weak HAS-A relationship**.
-

👍 Output Example

Room contains 3 chairs:

Chair ID: 101

Chair ID: 102

Chair ID: 103

✅ Example 2: Library *aggregates* Books

📌 Concept

Books are created outside and library only keeps pointers.

```
#include <iostream>  
#include <cstring>  
using namespace std;
```

```
class Book {  
private:  
    char title[50];  
  
public:  
    Book(const char* t) {  
        strcpy(title, t);  
    }  
  
    void showBook() const {  
        cout << "Book: " << title << endl;  
    }  
};
```

```

class Library {
private:
    Book* books[5];
    int count;

public:
    Library() : count(0) {}

    void addBook(Book* b) {    // Aggregation
        books[count++] = b;
    }

    void showLibrary() const {
        cout << "Library collection:\n";
        for(int i = 0; i < count; i++) {
            books[i]->showBook();
        }
    }
};

int main() {
    Book b1("C++ Programming");
    Book b2("Data Structures");

    Library lib;
    lib.addBook(&b1);
    lib.addBook(&b2);

    lib.showLibrary();

    return 0;
}

```

✓ Why Aggregation?

- Library **does not create** Books.
 - Books **exist independently**.
 - Deleting Library does NOT delete books.
-

✅ Example 3: Department *aggregates* Professor

📌 Concept

Professors are hired by a department **but can exist even if department is deleted.**

```
#include <iostream>
```

```
using namespace std;
```

```
class Professor {
```

```
public:
```

```
    void show() const
```

```
    {
```

```
        cout << "I am a professor." << endl;
```

```
    }
```

```
};
```

```
class Department {
```

```
private:
```

```
    Professor* prof;    // aggregation
```

```
public:
```

```
    Department(Professor* p) : prof(p) {}
```

```
    void details() const {
```

```
        cout << "Department details: ";
```

```
        prof->show();
```

```
    }
```

```
};
```

```
int main() {
```

```
    Professor p1;    // Independent object
```

```
    Department cs(&p1);    // Aggregation
```

```
    cs.details();
```

```
    return 0;
```

```
}
```

✓ Why aggregation?

- Department **does not own** Professor.
- Professor exists before and after Department.

✅ **Example 4: Car *aggregates* Engine (Engine can exist independently)**

📌 **Concept**

A **Car has an Engine**, but the **Engine exists independently** and is passed into the Car.

```
#include <iostream>
```

```
using namespace std;
```

```
class Engine {
```

```
public:
```

```
    void start() const {
```

```
        cout << "Engine started!" << endl;
```

```
    }
```

```
};
```

```
class Car {
```

```
private:
```

```
    Engine* engine;    // aggregation
```

```
public:
```

```
    Car(Engine* e) : engine(e) {} // Car does NOT create Engine
```

```
    void drive() const {
```

```
        engine->start();
```

```
        cout << "Car is moving..." << endl;
```

```
    }
```

```
};
```

```
int main() {
```

```
    Engine e1;    // Independent object
```

```
    Car c1(&e1);  // Aggregation
```

```
    c1.drive();
```

```
    return 0;
```

```
}
```

✓ Why is this Aggregation?

- The **Engine lives independently** of Car.
 - When Car is destroyed, Engine still exists.
-

In C++, friend functions are special functions that are **not members** of a class but are allowed to **access its private and protected members**.

Normally, private members of a class cannot be accessed from outside.

But by declaring a function as a **friend**, the class allows it to bypass access restrictions.

Why Use Friend Functions?

- When you want **two classes to work closely together**
- When an **external function must access private data**
- When **operator overloading** requires access to internal data
(e.g., operator+, operator<<, operator>>)

```
#include <iostream>
using namespace std;

class Student {
private:
    int rollNo;
    string name;

public:
    Student(int r, string n) : rollNo(r), name(n) {}

    // Declare friend function
    friend void displayStudent(Student s);
};

// Friend function definition
void displayStudent(Student s) {
    cout << "Roll No: " << s.rollNo << endl;    // Accessing private members
    cout << "Name: " << s.name << endl;
}

int main() {
    Student s1(101, "Ali");
    displayStudent(s1);
    return 0;
}
```

What is a Friend Class?

If class A declares class B as a *friend*, then B can access all private/protected members of A.

Syntax:

```
class A {  
    private:  
        int x;  
  
    public:  
        friend class B; // B is a friend class  
};
```

Now class B can access x directly, even though it is private.

```
#include <iostream>  
using namespace std;  
  
class Student {  
    private:  
        int rollNo;  
        string name;  
  
    public:  
        Student(int r, string n) : rollNo(r), name(n) {}  
  
        friend class Teacher; // Teacher is friend class  
};  
  
class Teacher {  
    public:  
        void showStudentInfo(Student s) {  
            // Direct access to private data of Student  
            cout << "Roll No: " << s.rollNo << endl;  
            cout << "Name: " << s.name << endl;  
        }  
};  
  
int main() {  
    Student s1(101, "Ali");  
    Teacher t1;  
    t1.showStudentInfo(s1);  
  
    return 0;  
}
```

Example of Aggregation:

```
#define _CRT_SECURE_NO_WARNINGS
#pragma warning(disable : 4996)
#include <iostream>
using namespace std;

class Course {
private:
    char courseName[50];

public:
    Course(const char* name)
    {
        strcpy(courseName, name);
    }
    void showCourse() const
    {
        cout << "Course: " << courseName << endl;
    }
    ~Course()
    {
        cout << "Destructor of course" << endl;
    }
};

class Student {
private:
    char name[50];
    Course* course; // Aggregation (pointer to external object)

public:
    // Constructor receives a pointer to Course (not created inside Student)
    Student(const char* n, Course* c) : course(c) {
        strcpy(name, n);

        // or course = c
        // member initialization would not use to call
        // aggregate object constructor as it is
        // pointer not object
        // we are passing the pointer of aggregated
        // object which is created already
    }
    void showStudent() const {
        cout << "Student: " << name << endl;
        course->showCourse();
    }
    ~Student()
    {
        cout << "Destructor of Student" << endl;
    }
};
```

```
int main() {
    // Independent object (lifetime NOT controlled by Student)
    Course c1("Object-Oriented Programming");
    // Student aggregates Course (has-a relationship)
    {
        Student s1("Ali", &c1);
        s1.showStudent();
    }
    c1.showCourse();
    return 0;
}
```

(These are "HAS-A" relationships where the part can exist independently.)

Here are three more clear and simple examples of aggregation in C++, all different from the previous Student–Course example.

(These are "HAS-A" relationships where the part can exist independently.)

Friendship in C++: friendship is granted not taken

- i. Friend functions
- ii. Friend classes

i. Friend functions

```
class X
{
private:
    int a, b;
public:
    void memberFunction()
    {
        cout << "I am void memberFunction() " << endl;
    }
    friend void doSomething();
};
void doSomething()
{
    X o1;
    //o1.a = 10; /// error, doSomething() cant access the private parts of a class

    o1.a = 10; /// after declaring a friend can access the private data
}
// 1. friend function can be declared in private as well as the public part of a class
//2. friendship is only granted not taken.....
```

ii. Friend class

```
class X
{
private:
    int a, b;
public:
    void memberFunction()
    {
        cout << "I am void memberFunction() " << endl;
    }
    friend class Y;
};
```

```
class Y
```

```
{
    int d, e;
public:
    void setD()
    {
        X o1;
        o1.a = 10; // accessing private information of other class
    }
};

int main()
{
    Y o1;
    o1.setD();

    return 0;
}
```

Function overloading : (we have seen constructor overloading)

- i. same name of functions,
- ii. differ with no of operands,
- iii. types of arguments, or
- iv. combination of ii, and iii

Motivation of operator overloading:

- Bjarne Stroustrup tried to make user defined data types works like primitive type

$a + b$; $a * b$;

$c1 + c2$;

Operator overloading:

- **1st attempt:** Want to write functions for complex class, eg add two complex number as we did manually

```
class Complex
{
    double real, imag;

    public:

        Complex add(const Complex &rhs)
        {
            Complex t;
            t.real = this->real + rhs.real;
            t.imag = this->imag + rhs.imag;
            return t;
        }
        Complex subtract(const Complex &rhs);
        Complex multiply(const Complex &rhs);
};

int main()
{
    Complex c1(3, 4);
    Complex c2(3, 4);
    Complex c3 = c1.add(c2); //Complex c4 =c1 + c2 + c3
    //Complex c3 = c1.operator+(c2);
```

}

Issues:

1. For primitive data type like, int, double, we can do $a + b$, and we want to do same for user defined data type, $c1 + c2$,

But we have to write `Complex c3 = c1.add(c2);` // [unnatural]

2. As for primitive data type we able to do cascaded function call

`a4=a1+ a2+ a3,`

we have to do

i) `c4=(c1.add(c2)).add( c3),` // [complex]

OR alternatively

```
t1 = c1.add(c2);
```

```
c4= t1.add(c3);
```

//[unnatural, less readable, less understandable, difficult to maintain] => Need some natural mechanism to handle these issues.

- 2nd attempt:

- o Assume we have already implemented the operator overloading mechanism
- o `c4=c1+ c2+ c3` // [natural, more readable, more understandable, easy to maintain]

+ operator is already overloaded by compiler of use for different data types:

$5 + 6$ => corresponding to this function compiler call `add(int a, int b)`

$3.4 + 4.5$ => corresponding to this function compiler call `add(double a, double b)`

List of operators that can be overloaded in C++

new	delete	new []	delete []					
+	-	*	/	%	^	&		~
!	=	<	>	+=	-=	*=	/=	%=
^=	&=	=	<<	>>	>>=	<<=	==	!=
<=	>=	&&		++	--	,	->*	->
()	[]							

List of operator that can't be overloaded in C++

.	.*	::	? :	#	##
---	----	----	-----	---	----

Rules:

- i. list of operator that can be overloaded:
uniary: +, -, !, ++, --, new, delete
binary, +, *, >>, <<, =, +=, <=,
not allowed to overload :., ::, #, ?:
- ii. Precedence of an operator is not affected due to overloading
- iii. Associativity remains same (eg a + b + c, will evaluate left to right)
- iv. Arity of operator remains same: no of operands should always remain same: a + b; !a;
- v. New operator is not allowed to define and overload eg \$ is not allowed to use and overload for **raisedToPow()**
- vi. Operator overloading of primitive datatypes are not allowed to define: **int operator + (int, int)**
- vii. Maintain the spirit of operator

- 2 ways to write the operator overloading function

1. Member function

2. Non member function

- some time the mixture of both, we write them according to the need.

Syntax of binary operator overloading(eg: +, -, >>, ==, =)

1. Member function:

- a. `type1 operator B_OP (type2 rhs)` // lhs op is received through this

2. Non member function

- a. `type1 operator B_OP (type2 lhs, type3 rhs)` // this is not call though object name, so pass the two argument explicitly

Example:

```
class Complex
{
    double real, imag;

public:
    Complex(double r = 0.0, double i=0.0)
    {
        real = r;
        imag = i;
    }

    Complex operator + (const Complex &rhs)
    {
        Complex t;
        t.real = real + rhs.real;
        t.imag = imag + rhs.imag;
        return t;
    }
    void display()
    {
        cout << real << "+ i " << imag << endl;
    }
};

int main()
{
    Complex c1(3, 4);
    Complex c2(3, 4);
    //Complex c3 = c1.add(c2);

    Complex c3 = c1 + c2;
    //Complex c3 = c1.operator+(c2);

    Complex c4 = c1 + c2 + c3;
    //Complex c3 = (c1.operator+(c2)).operator+(c3);

    c3.display();

    return 0;
}
```

Example 2 : maintain the semantic of operator :

```
void operator + (const Complex &rhs)
{
    //Complex t;
    real = real + rhs.real;
    imag = imag + rhs.imag;
    //return t;
}
```

C1+ c2/// it change the c1 contents

- It is not a good way to write the + operator as it changes the contents of calling object
- Try to mimic the behavior of operator as it is with primitive data type.

Homework:

- Implement operator – ()
- Provide the implementations of compound operator +=, -=
- Provide the implementations of relational operator for complex class (>, <)
- Provide the implementation of these operators (+ and -) as the nonmember function for complex class

Today lecture:

Operator overloading:

- **Recap of last lecture:**
- **Non member functions:**
 - **Non friend function : Use getters and setters**
 - **friend function: Efficient implementation**
- **Assignment operator**

Recap of last lecture:

Rules:

- i. list of operator that can be overloaded:
uniary: +, -, !, ++, --, new, delete
binary, +, *, >>, <<, =, +=, <=,
not allowed to overload : ., ::, #, ?:
- ii. Precedence of an operator is not affected due to overloading
- iii. Associativity remains same (a+b+c : L->R, while a =b =c : R->L)
- iv. Arity of operator remains same: no of operands should always remain same: a + b; !a;
- v. New operator is not allowed to define and overload eg \$ is not allowed to use and overload for **raisedToPow()**
- vi. Operator overloading of primitive datatypes are not allowed to define: **int operator + (int, int)**
- vii. Maintain the spirit of operators

Binary operator overloading: +, -, >>, ==, = etc: 2 ways to write the operator overloading function

1. Member function

2. No member function

- some time the mixture of both, so we write them according to the need.

Syntax of binary operator overloading:

1. Member function:

a. `type1 operator B_OP (type2 rhs) // call through object name, so lhs op is received through this pointer`

2. Non member function

a. `type1 operator B_OP (type2 lhs, type3 rhs) // Not call through object name, so need to pass two arguments explicitly`

Today lecture:

Operator overloading:

- Non member functions:
 1. Non friend function : Use getters and setters
 2. friend function: Efficient implementation

```
#include <iostream>
using namespace std;
class Complex
{
    double real, imag;
public:
    Complex(double r = 0.0, double i = 0.0)
    {
        real = r;
        imag = i;
    }

    Complex operator+ (const Complex& rhs)
    {
        Complex t;
        t.real = real + rhs.real;
        t.imag = imag + rhs.imag;
        return t;
    }

    Complex& operator+ ( double rhs)
    {
        Complex t;
        t.real = real + rhs;
        t.imag = imag ;
        return t;
    }
}
```

```

void display()
{
    cout << real << "+ i " << imag << endl;
}

```

// Non member: friend functions:

```

// friend function enables to achieve efficiency
friend Complex& operator+ (double lhs, const Complex& rhs);

};
Complex& operator+ (double lhs, const Complex& rhs)
{
    Complex temp;
    temp.real = lhs + rhs.real; // due to friendship, we able to access private data
    temp.imag = rhs.imag; // no need to do 4 function call
    return temp;
}
int main()
{
    Complex c1(3, 4);
    Complex c2(3, 4);
    //Complex c3 = c1.add(c2);
    Complex c3 = c2 + 5.6;
    Complex c4 = 5.6 + c2;
    c3.display();
    c4.display();
}

```

Is we write both member and not member function for same takes???

I think, yes we call we object name, it is called the member function and if we call without object name then it is called the nonmember function

Operator overloading:

- Assignment operator:

```
#pragma warning(disable : 4996)
#include<iostream>
using namespace std;
class String
{
    int size;
    char* str;
public :
    String (const char *s = NULL)
    {
        if (s != NULL)
        {
            size = strlen(s);
            str = new char[size + 1];
            strcpy(str, s);
        }
        else
        {
            size = 0;
            str = NULL;
        }
    }

    ~String ()
    {
        if (size != 0)
        {
            delete[] str;
            str = NULL;
            size = 0;
        }
    }

    String (const String & lhs )
    {
        if (lhs.size != 0)
        {
            size = lhs.size;
            str = new char[size + 1];
            strcpy(str, lhs.str);
        }
        else
        {
            size = lhs.size;
            str = NULL;
        }
    }
}
```

```

String & operator =(const String& lhs)
{
    if (size != 0) // resolve the memory leakage issue
    {
        // as we did in destructor
        size = 0;
        delete [] str ;
        str = NULL;
    }
    if (lhs.size != 0) // resolve the dangling pointer issue
    {
        // as we did in copy constructor
        size = lhs.size;
        str = new char[size + 1];
        strcpy(str, lhs.str);
    }
    return *this;
}

void display()
{
    cout << "Str = " << str << endl;
    cout << "Its size is " << size << endl;
}

};

int main()
{
    String s1 = "Hello";
    String s2 = "Pakistan";
    s2 = s1;
    s2.display();
}

```

Next time:

- Will discuss self-assignment check for assignment operator
- Stream insertion and extraction operators

Homework:

- Implement Point class that have same functionalities as our Complex class.
- Implement the assignment operator for Student class
- Implement the assignment operator for Student class that compose the String class object for the name of student.
- Implement operator + for String class that concatenate two string.
- Operator== for comparison two string

Extra material :

```
class Complex
{
    double real, imag;
public:
    Complex(double r = 0.0, double i = 0.0) : real(r), imag(i)
    {}
    Complex operator + (const Complex& rhs)
    {
        Complex temp;
        temp.real = real + rhs.real;
        temp.imag = imag + rhs.imag;
        return temp;
    }
    void operator += (const Complex& rhs)
    {
        real += rhs.real;
        imag += rhs.imag;
    }

    //today lecture
    Complex operator + (double real)
    {
        Complex temp;
        temp.real = real + real;
        temp.imag = imag;
        return temp;
    }

    Complex& operator = (const Complex& rhs) // Complex& ensure that copy
    constructor is not called and case caddig is possible
    {
        real = rhs.real;
        imag = rhs.imag;
        return *this;
    }
    void display()const
    {
        cout << "real : " << real << " +i" << imag << endl;
    }
}
```

```
friend Complex operator + (double r, const Complex& rhs);

};
```

```
Complex operator + (double r, const Complex& rhs)
{
    Complex temp;
    temp.real = r + rhs.real;
    temp.imag = rhs.imag;
    return temp;
}

int main()
{
    Complex c1(4, 5);
    c1.display();
    Complex c2, c3;
    c3 = c2 = c1 ; // case cading is possible,
    c3.display();

    c2 += c3 ;
    c2.display();
    return 0;
}
```

```
#define _CRT_SECURE_NO_WARNINGS
#pragma warning(disable : 4996)
#include <iostream>
using namespace std;
```

```
class String
{
    char* str;
    int size;
public :
    String(const char* s = nullptr)
    {
        if (s != nullptr)
        {
            size = strlen (s);

            str = new char[size + 1];
            strcpy(str,s);
        }
    }
}
```

```

        else
        {
            str = nullptr;
            size = 0;
        }
    }
String(const String& rhs)
{
    if (rhs.str != nullptr)
    {
        size = rhs.size;
        str = new char[size + 1];
        strcpy(str, rhs.str);
    }
    else
    {
        size = 0;
        str = nullptr;
    }
}
~String()
{
    if (size != 0)
    {
        delete[] str;
        str = nullptr;
    }
}
void display()const
{
    cout << "size " << size << endl;
    cout << "str " << str << endl;
}

```

// today lecture

String& operator = (const String& rhs) // & in return helps to avoid copy
 constructor calls and destructors call

```

{
    if (this != &rhs) // handling self assignment,
    {
        if (str != nullptr) // memory leakage issue is fixed (destructor code)
        {
            delete[] str;
            str = nullptr;
            size = 0;
        }
    }
}

```

```

    }
    if (rhs.str != nullptr) // dangling pointer issue (copy constructor code)
// deep copy
    {
        size = rhs.size;
        str = new char[size + 1];
        strcpy(str, rhs.str);
    }
    else
    {
        size = 0;
        str = nullptr;
    }
}
return *this; // help cascading
}

```

```

};
int main()
{
    String s1("hello");
    s1.display();

    String s2("pakistan ");
    s2.display();

    String s3;

    s3 = s2;
    s3.display();

    s1 = s1 = s1;
    s1.display();
    return 0;
}

```

Today lecture:

- Assignment Operator overloading
- Stream insertion and extraction operator

1. Assignment Operator overloading

```
#include <iostream>
using namespace std;
#include <cstring>
#define _CRT_SECURE_NO_WARNINGS
class String
{
    char *str;
    int size;
public:
    String(const char *s = nullptr )
    {
        cout << " String(const char *s = nullptr )" <<endl;
        if (s != nullptr)
        {
            size = strlen (s);
            str = new char [size +1 ];
            strcpy(str,s);
        }
        else
        {
            size = 0;
            str = nullptr;
        }
    }

    String (const String &rhs)
    {
        if (rhs.str != nullptr)
        {
            size = rhs.size;
            str = new char [size +1 ];
            strcpy(str, rhs.str);
        }
        else
        {
            size = 0;
            str = nullptr;
        }
    }
}
```

```

~String()
{
    delete []str;
    str = nullptr;
}

String & operator= (String & rhs)
{
    cout << "Assignment operator" << endl;
    if (this != &rhs) // 4. Self assignment.
    {
        if (str != nullptr)
        {
            // 1. Handle Memory leakage, as in destructor
            delete []str;
            str = nullptr;
            size = 0;
            // 2. Deep copy as did in copyconstructor
            str = new char [rhs.size + 1];
            strcpy(str, rhs.str);
            size = rhs.size;
        }
    }

    //3. Handle cascading or chaining
    return *this;
}

void display()
{
    cout << size << " " << str<<endl;
}

};

int main() {
    String s1("Welcome");
    String s2 ("Pakistan");
    String s3 = s2;
    String s4;
    s1 = s3;
    //s2.display();
    return 0;
}
.....
,,,,,,,,,,,,,,,,,,,,,

```

2. Stream insertion operator <<

Stream Insertion (<<) and extraction (>>) operators

- Stream concept is used for I/O operation in C++.... As a stream/naadi to flow the data from one place to another
- We use them eg
 - o Int i 1, j 3;
 - o cout << i << j
 - o now we want to overload the << for our data types
 - o Complex c1, c2

```
#include <iostream>
using namespace std;
class Complex
{
    double real, imag;
public:
    Complex(double r = 0.0, double i = 0.0) : real(r), imag(i)
    {}
    // first attempt
    void operator << (const Complex& rhs)
    {
        cout << rhs.real << " " << rhs.imag << endl;
    }
    // second attempt
    ostream & operator << (ostream & out)
    {
        out << real << " " << imag << endl;
        return out;
    }
    // third attempt
    friend ostream & operator << (ostream & out, const Complex& rhs);
};

ostream & operator << (ostream &out, const Complex& rhs)
{
    cout << "ostream & operator << (ostream &out, const Complex& rhs)" <<
endl;
    out << rhs.real << " " << rhs.imag << endl;
    return out;
}

int main()
{
    Complex c1(4, 5);
    Complex c2(24, 25);
    //cout << c1; // first attempt error unable to call this function
    // second attempt;
    //c1 << cout;
```

```

// but how can we implement the case cadding...
//c1 << cout; /// << c2; issue. 1 syntax is very strange 2. no
casecading...
/// solution is implement the stream intersion operator as the non
member function

cout << c1 << c2 ;
operator << (operator <<(cout, c1), c2);
return 0;
}

```

3. Implement Stream extraction operator >>

- As the cin >> c1, let hand side is not the complex type object, so need to implement it as the non member function of our class
-

```

#include <iostream>
using namespace std;
class Complex
{
    double real, imag;
public:
    Complex(double r = 0.0, double i = 0.0) : real(r), imag(i)
    {}
    friend ostream & operator << (ostream & out, const Complex& rhs);

    friend istream& operator >> (istream& in, Complex& rhs);
};

ostream & operator << (ostream &out, const Complex& rhs)
{
    out << "ostream & operator << (ostream &out, const Complex& rhs)" <<
endl;
    out << rhs.real << " " << rhs.imag << endl;;
    return out;
}

istream& operator >> (istream& in, Complex& rhs)
{
    cout << "istream & operator >> (istream & out, Complex & rhs)" << endl;
    in >> rhs.real;
    in >> rhs.imag;
    return in;
}

int main()
{
    Complex c3, c4;

    cout << "Please enter the real and imag value of two complex number";
    cin >> c3 >> c4;
}

```

```
    cout << "Here is output" << endl;  
    cout << c3 << c4;  
  
    return 0;  
}
```

Homework:

1. String Class

- Implement the stream insertion operator:
friend ostream& operator << (ostream& out, const String& s);
- Implement the stream extraction operator:
friend istream& operator >> (istream& in, String& s);

2. Student Class

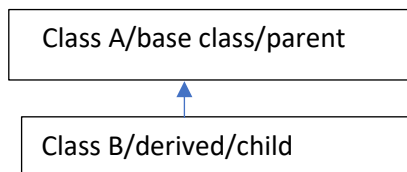
- Implement the assignment operator.
- Implement the stream insertion and extraction operators.

Inheritance: [another way of code REUSIABILITY]

Today Lecture:

1. Inheritance in classes
2. UML notation of inheritance
3. Types of Inheritance in C++
4. Is a relationship
5. Accessing members
6. Allocation of memory
7. Constructors call
8. Base class initiation
9. Initializing members
10. Destructors
11. Order of execution of constructors and destructors

1. Inheritance in classes:
2. UML notation



- If a class B inherits from class A, the B contains all the characteristics (information structure and behavior) of class A.
- A is also called base or parent class whereas the child class is also called the derived or child class
- Beside inherited characteristics, derived class may have its own unique characteristics.

3. Inheritance in C++: there are three type of inheritance in C++:
 - a. Public [we focus on public inheritance as it focus on is a relationship]
 - b. Private
 - c. protected

4. Is a relationship "Is a " relationship is model with the help of public inheritance.

- a. student is a person
- b. Rectangle is a shape
- c. care is a vehicle

a. syntax:

```
class childClass: public baseClass
{
.....
};
```

- base class public members become the derived class public members using public inheritance.

5. Accessing members

- Public members of the base class become public members of derived class
- Private members of the base class are not accessible from the outside of the base class, even in the derived class (to ensure INFORMATION HIDING)

```
class Person
{
    int age;
    char* name;
    int rollNo;
public:
    Person()
    {
        age = 0;
        name = NULL;
        rollNo = 1;
    }
    const char * getName()const
    {
        return name;
    }
    int getAge()const
    {
        return age;
    }
};
class Student:public Person
{
    int semester;
    int rollNo;
public:
    int getSemester()const
    {
        return semester;
    }
    int getRollNo()const
    {
        return rollNo;
    }
    void print()const
    {
        //cout << name << "is in " << semester << "semester" << endl;

        //b. Private members of the base class are not accessible from the outside of the base //class, even in the //derived
        //class (to ensure INFORMATION HIDING)
        cout << getName() << "is in " << semester << "semester" << endl;
        //a.Public members of the base class become public members of derived class, no need to specify the
        //object reference as it become the public member of derived class now
    }
};
int main()
{
    Student obj;
    obj.getAge(); // a.Public members of the base class become public members of derived class
    obj.print();
    return 0;
}
```

6. Allocation in memory:

Base mem1
Base mem2
...
Derived mem1
Derived mem2

Derived class object

- Every object of derived class has an anonymous object of base class
- As it is anonymous object, so we don't use the dot operator to access the public member of base class.

7. Constructor class sequence:

- The anonymous object of base class must be initialized using constructor of base class
- When a derived class object is created the constructor of base is executed before the constructor of derived class
- Normally the default constructor of base class is executed before the constructor of derived class

Base mem1
Base mem2
...
Derived mem1
Derived mem2

Base class constructor initialized the anonymous object
...

Derived class constructor initialized the derived class object
...

8. Order of construction:

```
class Parent
{
public:
    Parent()
    {
        cout << "Parent()" << endl;
    }
};
class Child : public Parent
{
public:
    Child()
    {
        cout << "Child()" << endl;
    }
};
int main()
{
    Child obj;
    return 0;
}
```

OUTPUT: Parent()
Child()

9. Need of default constructor:

Caution: if base class has no default constructor then compiler write it.

But if in base class have the overloaded constructor then the compiler is not able to write the default constructor and produce error. Otherwise use the base class or member initialization list to call to specific constructor of base class.

10. Base class initializer

```
class Parent
{
public:
    Parent(int i)
    {
        cout << "Parent(int i)" << endl;
    }

};
class Child : public Parent
{
    int member;
public:
    Child():Parent(3), member(0)
    {
        cout << "Child()" << endl;
    }

};
int main()
{
    Child obj;
    return 0;
}
```

Output:

```
Parent(int i)
child()
```

11. Can we initialize the base class data members?

```
class Person
{
public:
    int age;
    char* name;
    int rollNo;

    Person()
    {
        age = 0;
        name = NULL;
        rollNo = 1;
    }

};
```

```

class Student :public Person
{
    int semester;
public:
    Student(int a):age(a) //error, as it is assignment now
                        // Initialization is done by default constructor
                        // of base class
    {

    }
};

int main()
{
    Student obj(1);
    return 0;
}

```

12. Order of construction and destruction:

```

class Parent
{
public:
    Parent()
    {
        cout << "Parent()" << endl;
    }
    ~Parent()
    {
        cout << "~Parent()" << endl;
    }
};

class Child : public Parent
{
public:
    Child()
    {
        cout << "Child()" << endl;
    }
    ~Child()
    {
        cout << "~Child()" << endl;
    }
};

int main()
{
    Child obj;
    return 0;
}

```

Output:
Parent()
Child()

`~Child()`
`~Parent()`

;;;;;;;;;; **Single code**, covered all the concept that discussed above

// Online C++ compiler to run C++ program online

```
#include <iostream>
#include<cstring>
using namespace std;
class Person
{
    char * name;
    int age;
public:

    Person(const char *n= nullptr, int a = 0)
    {
        cout << "Person(const char *n= nullptr, int a = 0)"<<endl;
        if (n != nullptr)
        {
            name = new char [strlen(n) + 1];
            strcpy(name, n);
            age = a;
        }
        else
        {
            name = nullptr;
            age = a;
        }
    }

    const char * getName()const
    {
        return name;
    }
    int getAge()const
    {
        return age;
    }
    ~Person()
    {
        if (name != nullptr)
            {delete [] name;
            name = nullptr;}
        cout << "~Person()" <<endl;
    }
};

class Student : public Person
{
    int rollNo;
    int semester;
public:
    Student (const char *n = nullptr, int a = 0, int r = 0, int s = 0):Person(n, a),
    rollNo(r), semester (s)//, age(a)
```

```

{
    cout << "Student (char *n = nullptr, int a = 0, int r = 0, int s = 0)" <<endl;

    //rollNo = r;
    //semester = s;
}
int getRollNo() const
{
    return rollNo;
}
int getSemester() const
{
    return semester;
}
~ Student ()
{
    cout << "~ Student ()" <<endl;
}
void display()
{
    if (getName() != nullptr)
    {
        cout << this->getName() <<endl; // as base class object is anonymous so it is
called directly
    }
    else
    {
        cout << "name = null" <<endl;
    }

    cout << getAge()<<endl; // as age is private, not accessed directly

    cout << getRollNo()<<endl;
    cout << getSemester()<<endl;
}

};
int main() {
    Student s1 ("Ali", 19, 1, 3);
    cout << s1. getAge(); // as due to inheritance getAge() becomes the part of Student class,
so it can be accessed through s1
    cout <<endl;
    cout << s1.getSemester() <<endl;
    s1. display();
    return 0;
}

```

;;;;;;;;;;;;;

Homework Assignment – Inheritance & Dynamic Memory

Q1. Implement a Teacher class derived from Person

- Derive a class Teacher from the given Person class.
- Add data members:
 - char* subject;
 - double salary;
- Implement:
 - parameterized constructor using member initialization list
 - deep copy for subject
 - destructor
 - display() function

Q2. Modify the Student class

Add a new dynamically allocated data member:

char* department;

Now update:

- constructor
- destructor
- deep copy in copy constructor
- display function

Q3. Write a program that demonstrates constructor / destructor calling order

Create objects of Student and Teacher and show clearly (via print statements) the order in which:

- base constructor
 - derived constructor
 - derived destructor
 - base destructor
- are executed.

Q4. Add a copy constructor to the Student class

Write your own:

Student(const Student& rhs);

It must:

- call the Person copy constructor
- deep copy rollNo, semester, and new dynamic members (if any)

Test it using:

Student s1("Ali", 19, 1, 3);

Student s2 = s1;

Lecture contents:

- Protected access Specifier in inheritance
- Implicit and explicit use of IS A relationship in C++

Protected access Specifier in inheritance:

```
#define _CRT_SECURE_NO_WARNINGS
```

```
#pragma warning(disable : 4996)
```

```
#include <iostream>
```

```
#include <cstring>
```

```
using namespace std;
```

```
class Date
```

```
{
```

```
private:
```

```
    int day, month, year;
```

```
        // utility function.
```

```
    bool isLeapYear(int y)
```

```
{
```

```
    return (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
```

LEAP YEAR:

Summary:

- The year is divisible by 4 **BUT** not by 100. (e.g., 2024) **OR**
- The year is divisible by 400. (e.g., 2000)

```
}
```

```
public:
```

```
    Date(int d = 1, int m=1, int y = 2025) : day(d), month(m), year(y) {}
```

```
    int getYear()
```

```
{
```

```
        return year;
```

```
}
```

```
    void printDate() {
```

```
        cout << day << "/" << month << "/" << year << endl;
```

```
}
```

```
};
```

```
class SpecialDate : public Date
```

```
{
```

```
private:
```

```
    char eventName [100];    // e.g., "Eid", "Birthday"
```

```
    bool isHolidayFlag;
```

```
public:
```

```
    SpecialDate(int d, int m, int y, const char *event, bool holiday) : Date(d, m, y), isHolidayFlag(holiday)
```

```
{
```

```
        strcpy (eventName, event);
```

```
}
```

```
    void showDetails() {
```

```
        printDate();
```

```

cout << "Event: " << eventName << endl;
cout << "Holiday? " << (isHolidayFlag ? "Yes" : "No") << endl;
cout << "Leap Year? " << (isLeapYear(getYear()) ? "Yes" : "No") << endl; } // NOT ACCESSABLE

};
int main()
{
    Date aDate;
    //aDate.isLeapYear(); // not accessible

    SpecialDate eid(17, 6, 2025, "Eid-ul-Adha", true);
    //eid.isLeapYear(); // not accessible
    eid.showDetails();

    return 0;
}

```

Solution:

1. **rewrite isLeapYear() in the derived class**

This would allow the derived class to have its own version, but it results in redundant code, which is not a good programming practice.

2. Making isLeapYear() public **does allow access**, but it:

- a. Breaks encapsulation
- b. Exposes internal logic unnecessarily
- c. Cannot be done if the base class is a precompiled library
- d. May lead to long-term design problems
- e. This is why programmers prefer **protected** over **public** in such cases:
→ Protected allows access in derived classes **without exposing it to the outside world**.
- f. again, as some time only the executable code(library) is given so we can't access it

3. **Protected Access Specifier: it is the intermediate type of access specification between private and public => make the accessible the protected member of base class in derived class**

- a. Protected members can not be accessed outside of the class
- b. Protected members of base become protected members of derived class in public inheritance

// Online C++ compiler to run C++ program online

```
#include <iostream>
```

```
#include <cstring>
```

```
using namespace std;
```

```
class Date
```

```
{
```

```
private:
```

```
    int day, month, year;
```

```
protected:// now it is only accessible in derived class but not accessible outside of base and derived
```

```
class
```

```
    bool isLeapYear(int y) {
```

```
        return (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
```

```
    }
```

```
public:
```

```
    Date(int d=1, int m=1, int y=2025) : day(d), month(m), year(y) {}
```

```
    void printDate() {    cout << day << "/" << month << "/" << year << endl;}
```

```
    int getYear()
```

```
    {
```

```
        return year;
```

```
    }
```

```
};
```

```
class SpecialDate : public Date
```

```
{
```

```
private:
```

```
    char eventName [100];    // e.g., "Christmas", "Birthday"
```

```
    bool isHolidayFlag;
```

```
public:
```

```
    SpecialDate(int d, int m, int y, const char *event, bool holiday)
```

```
        : Date(d, m, y), isHolidayFlag(holiday)
```

```
    {
```

```
        strcpy(eventName, event);
```

```
    }
```

```
void showDetails()
```

```
{
```

```
    printDate();
```

```
    cout << "Event: " << eventName << endl;
```

```
    cout << "Holiday? " << (isHolidayFlag ? "Yes" : "No") << endl;
```

```
    cout << "LeapYear? " << (isLeapYear(getYear()) ? "Yes" : "No") << endl;
```

```
    // isLeapYear() // now accessible due to protected access specifier
```

```
}
```

```
};
```

```
int main()
```

```
{
```

```
        Date aDate;
//        aDate.isLeapYear(); // not accessible

    SpecialDate eid(17, 6, 2025, "Eid-ul-Adha", true);
//        eid.isLeapYear(); // not accessible
        eid.showDetails();
    return 0;
}
```

Only make the accessibility in the protected member of base class in derived class

Disadvantage: protected access specifier makes the member available as protected member in derived class using public inheritance.

Yes, it is violated of information hiding but it is only allowed in class hierarchy

Class hierarchy represents a family and in family it is allowed in real word close family members can access the private room of another family members. => protected access specifier is given for this purpose.

Is a relationship in C++

```
#include <iostream>
using namespace std;
class Person
{
    int age;
public:
    Person(int a)
    {
        age = a;
    }
    int getAge()const
    {
        return age;
    }
};
class Student : public Person
{
    int rollNo;
public:
    Student(int a, int r):Person(a), rollNo(r)
    {}
    int getRollNo()const
    {
        return rollNo;
    }
};

void play (const Person & p)
{
    cout <<p.getAge() <<"Playing"    ;
    //cout << p.getName() << " is Playing";
}
int main()
{
    Student s(18,3);
    s.getAge(); // due to public inheritance
    s.getRollNo();

    Person* p;
    p = &s; // student is a kind of person
           // we can put student of object
           // to Person pointer
           // pictorially p point to anonymous base class object

    p->getAge();
    //p->getRoll();
    // 1 p is only point to base class anonymous object
    // 2 p static data type of Person, so compiler try
    //to find getRoll() in Person class
    // so get error

    // similarly we can do with reference variable
    // explicit conversion use of reference variable
```

```

    Person& refp = s; // is a kind of relationship in C++
    refp.getAge();
    //refp.getRollNo();
// implicit use of reference// implicit conversion
    play(s);
    Person p1(35);
    play(p1);

    return 1;
}

```

Homework:

Q1. Modify the Code (Hands-On Programming Task)

A class Book contains:

```

class Book {
private:
    string title;
    int pages;
    bool isLongBook(int p) {
        return p > 300;
    }
public:
    Book(string t, int p): title(t), pages(p) {}
    void printBook();
};

```

A derived class Novel needs to check whether its page count qualifies as a long novel.

Task:

1. Modify the base class so that `isLongBook()` is accessible only in the derived class, not to the outside world.
2. Create a derived class Novel that:
 - Inherits from Book
 - Contains an attribute genre
 - Prints whether the novel is long using `isLongBook()`
3. Write a `main()` function to test your code.

.....

Q2. IS-A Exercise (Programming)

You are given:

```
class Animal {  
public:  
    void eat() { cout << "Eating\n"; }  
};  
  
class Dog : public Animal {  
public:  
    void bark() { cout << "Barking\n"; }  
};
```

Tasks:

- Create an object of Dog and show which functions it can access.
- Create a Animal* pointer pointing to a Dog object.

- Which functions can you call using the pointer? Why?
- c. Create a function:

```
void perform(Animal &a);
```

Pass both an Animal object and a Dog object to it and explain the output.

- Explain how this example demonstrates **implicit upcasting** and the **IS-A** relationship.

Implementation of Copy constructor and assignment operator in inheritance:

```
// Online C++ compiler to run C++ program online
#include <iostream>
#include<cstring>
using namespace std;

class Person
{
    char * name;
public:
    Person (const char * n= "unknown")
    {
        if (n != nullptr)
        {
            name = new char [strlen(n) + 1];
            strcpy(name, n);
        }
    }

    Person (const Person & p)
    {
        name = new char [strlen(p.name) + 1];
        strcpy(name, p.name);
    }
    ~Person()
    {
        delete [] name;
        name = nullptr;
    }
    const char *getName ()const
    {
        return name;
    }
    Person & operator= (const Person & p)
    {
        if (this != &p)
        {
            delete []name;
            // name = nullptr;

            name = new char [strlen(p.name) + 1];
            strcpy(name, p.name);
        }
    }
}
```

```

        }
        return *this;
    }

};

class Student: public Person
{
    char * dept;
public:
    Student (const char * n= "unknown", const char * d= "Unknown"): Person
(n)
    {
        if (d != nullptr)
        {
            dept = new char [strlen(d) + 1];
            strcpy(dept, d);
        }
    }

    Student (const Student & s) : Person(s)
    {
        dept = new char [strlen(s.dept) + 1];
        strcpy(dept, s.dept);
    }
    ~Student()
    {
        delete [] dept;
        dept = nullptr;
    }

    Student & operator= (const Student & s)
    {
        if (this != &s)
        {
            Person ::operator = (s);
            //static_cast <Person &> (*this) = s;
            delete []dept;
            // dept = nullptr;

```

```

        dept = new char [strlen(s.dept) + 1];
        strcpy(dept, s.dept);
    }
    return *this;
}
void print ()
{
    cout << "Name: " << getName() <<endl;;
    cout << "Department: " << dept <<endl;

}

};
int main() {
    Student s1 ("Ali", "CS");
    {
        Student s2 = s1;
        s2. print();
        Student s3;
        s3 = s1;
        s3.print();
    }
    s1.print ();
    return 0;
}

```

Homework:

Q1. Implement a New Class Hierarchy, Create two classes:

Class 1: Book

- Data members: char* title, int pages
- Implement:
 - Constructor
 - Copy constructor
 - Assignment operator
 - Destructor

Class 2: EBook (inherits from Book)

- Additional data member: char* format (e.g., "PDF", "EPUB")
- Implement:
 - Constructor
 - Copy constructor
 - Assignment operator (must call Book::operator=)
 - Destructor

Then write a main() function that:

- Creates EBook objects
- Copies them using both copy constructor and assignment operator
- Prints all data members to verify deep copying

Q2. Implement a New Class Hierarchy (Function Overriding Without virtual)

You are required to implement a simple class hierarchy to understand how function overriding behaves when the base class does not use the virtual keyword. Create the following two classes:

```
class Shape {
public:
    void draw() {
        cout << "Drawing a generic shape" << endl;
    }
};

class Circle : public Shape {
public:
    void draw() {    // same name, but base class function is NOT virtual
        cout << "Drawing a circle" << endl;
    }
};
```

Tasks

- a. Create an object of Shape and an object of Circle. Call draw() on both.
- b. Create a Shape* pointer that points to a Circle object.
Call draw() using the pointer and record the output.
- c. Explain in 3–4 lines why the Circle version of draw() is NOT called when invoked through a Shape* pointer.
- d. Modify only the Shape class by making draw() a virtual function.
Run the same tests again and explain the difference in behavior.

```

#define _CRT_SECURE_NO_WARNINGS
#pragma warning(disable : 4996)
#include <iostream>
using namespace std;
class Shape
{
    protected:
        char sType;
    public:
        void draw()
        {
            cout << "ShapeDraw()" << endl;
        }
        int calcArea()
        {
            cout << "ShapeCalcArea()" << endl;
            return 0;
        }
};

class Rectangle: public Shape
{
    public:
        Rectangle()
        {
            sType = 'R';
        }
        void draw()
        {
            cout << "RectangleDraw()" << endl;
        }
        int calcArea()
        {
            cout << "RectangleCalcArea()" << endl;
            return 1;
        }
};

class Triangle : public Shape
{
    public:
        Triangle()
        {
            sType = 'T';
        }
        void draw()
        {
            cout << "TriangleDraw()" << endl;
        }
        int calcArea()
        {
            cout << "TriangleCalcArea()" << endl;
            return 1;
        }
};

```

```

    }
};

class Line : public Shape
{
    public:
    Line()
    {
        sType = 'L';
    }
    void draw()
    {
        cout << "LineDraw()" << endl;
    }
    int calcArea()
    {
        cout << "LineCalcArea()" << endl;
        return 0;
    }
};

void drawShape( Shape **sp, int size) /// editor
{
    for (int i = 0; i < size; i++)
    {
        sp[0]->draw();
    }
}

int main()
{
    const int SIZE = 10;
    Shape *p[SIZE];

    p[0] = new Line;
    p[1] = new Triangle;
    p[2] = new Rectangle;
    p[3] = new Line;
    p[4] = new Rectangle;
    p[5] = new Triangle;
    p[6] = new Triangle;
    p[7] = new Rectangle;
    p[8] = new Line;
    p[9] = new Rectangle;

    drawShape(p, SIZE);

    return 0;
}

```

Output:

```

ShapeDraw()
ShapeDraw()
ShapeDraw()
ShapeDraw()
ShapeDraw()
ShapeDraw()
ShapeDraw()
ShapeDraw()
ShapeDraw()
ShapeDraw()

```

Reason: static type that cause confusion in the overriding.

Solutions:

1. Down casting using static_cast

```
void drawShape( Shape **sp, int size) /// editor
{
    for (int i = 0; i < size; i++)
    {
        if (sp[i]->getType() == 'T')
        {
            static_cast<Triangle*>(sp[i])->draw();
        }
        else if (sp[i]->getType() == 'R')
        {
            static_cast<Rectangle*>(sp[i])->draw();
        }
        else //if (sp[0]->getType() == 'R')
        {
            static_cast<Line*>(sp[i])->draw();
        }
    }
}

void CalcAreaShapes(Shape** sp, int size) /// editor
{
    for (int i = 0; i < size; i++)
    {
        if (sp[i]->getType() == 'T')
        {
            static_cast<Triangle*>(sp[i])->draw();
        }
        else if (sp[i]->getType() == 'R')
        {
            static_cast<Rectangle*>(sp[i])->draw();
        }
        else //if (sp[0]->getType() == 'R')
        {
            static_cast<Line*>(sp[i])->draw();
        }
    }
}
/// case use switch statement
```

Issue.

- a. Need to remember classes/types
- b. Complicated code

Solution:

Polymorphism

- In the Object-Oriented (OO) model, polymorphism means that different objects can respond differently to the same message (stimulus). As a result, the sender of a message does not need to know the exact class of the receiving object.
- In C++, polymorphism is implemented using virtual functions.
- The target of a virtual function call is determined at run time, based on the type of object being referenced, not the type of pointer or reference.
- In C++, a function is declared virtual using the keyword virtual, for example:
virtual void draw();

Overriding function as virtual function

```
//////////
class Shape
{
    protected:
        char sType;
    public:
        char getType()
        {
            return sType;
        }
        virtual void draw()
        {
            cout << "ShapeDraw()" << endl;
        }
        virtual int calcArea()
        {
            cout << "ShapeCalcArea()" << endl;
            return 0;
        }
};

//////////
void CalcAreaShapes(Shape** sp, int size) /// editor
{
    for (int i = 0; i < size; i++)
    {
        sp[i]->calcArea();
    }
}

void drawShape( Shape **sp, int size) /// editor
{
    for (int i = 0; i < size; i++)
    {
        sp[i]->draw();
    }
}

...
```

Benefits of Polymorphism

Polymorphism simplifies code by allowing the **same message (function call)** to exhibit **different behaviors** depending on the **actual object that receives the message**. This enables flexible and extensible designs without modifying existing code.

How Polymorphism Works

Static vs. Dynamic Binding

- **Static Binding (Early Binding):**
The target function for a call is **determined at compile time**.
 - **Dynamic Binding (Late Binding):**
The target function for a call is **determined at run time**, based on the type of object being referenced.
-

Static Binding (Early Binding): Examples

```
Line line;  
line.draw();    // Line::draw() is called  
Shape* sp = new Line();  
sp->draw();    // Shape::draw() is called if draw() is NOT virtual
```

Dynamic Binding (Late Binding): Example

```
Shape* sp = new Line();  
sp->draw();    // Line::draw() is called if draw() IS virtual
```

Rule of Thumb (Best Practices in C++)

1. If a class manages resources, **define a destructor**.
2. Define a **copy constructor**.
3. Define a **copy assignment operator**.
(Together known as the Rule of Three)
4. If a base class function is **non-virtual**, avoid overriding it in derived classes, as this can lead to confusion.
Instead, declare the base class function as **virtual** if polymorphic behavior is intended.
5. Always declare the **destructor of a base class as virtual** when using inheritance.

Homework:

Coding Problem 1: Fix the Polymorphism Bug (Static Binding)

Problem Statement

The following code compiles and runs, but it does **not** produce polymorphic behavior.

```
#include <iostream>
```

```
using namespace std;
```

```
class Shape
```

```
{
```

```
public:
```

```
    void draw()
```

```
    {
```

```
        cout << "ShapeDraw()" << endl;
```

```
    }
```

```
};
```

```
class Circle : public Shape
```

```
{
```

```
public:
```

```
    void draw()
```

```
    {
```

```
        cout << "CircleDraw()" << endl;
```

```
    }
```

```
};
```

```
int main()
```

```
{
```

```
    Shape* sp = new Circle();
```

```
    sp->draw();
```

```
    return 0;
```

```
}
```

Tasks

1. Modify the code so that **dynamic binding** occurs.
2. Do **not** change the function call in main().
3. The output must be: CircleDraw()

Coding Problem 2: Shape Editor Using Polymorphism

Problem Statement

Create a **Shape Editor** using polymorphism.

Requirements

1. Create a base class Shape with:
 1. virtual void draw();
 2. virtual int calcArea();
 - 3.
4. Create derived classes:
 - o Rectangle
 - o Triangle
 - o Line
5. Override both functions in each derived class.
6. In main():
 - o Create an array of Shape*
 - o Store different derived objects
 - o Call draw() and calcArea() using a loop

Constraints

- No if, switch, getType()
No static_cast or dynamic_cast
- Only polymorphism is allowed.

Coding Problem 3: Remove Downcasting from Legacy Code

Given Code (Bad Design)

```
void drawShapes(Shape** sp, int size)
{
    for (int i = 0; i < size; i++)
    {
        if (sp[i]->getType() == 'R')
            static_cast<Rectangle*>(sp[i])->draw();
        else if (sp[i]->getType() == 'T')
            static_cast<Triangle*>(sp[i])->draw();
        else
            static_cast<Line*>(sp[i])->draw();
    }
}
```

Tasks

1. Redesign the classes to **eliminate downcasting**.
2. Rewrite drawShapes() using **only one line** inside the loop.
3. The correct derived draw() must be called at runtime.

Coding Problem 4: Area Calculator Using Runtime Polymorphism

Problem Statement

Write a function:

```
int totalArea(Shape** sp, int size);
```

Requirements

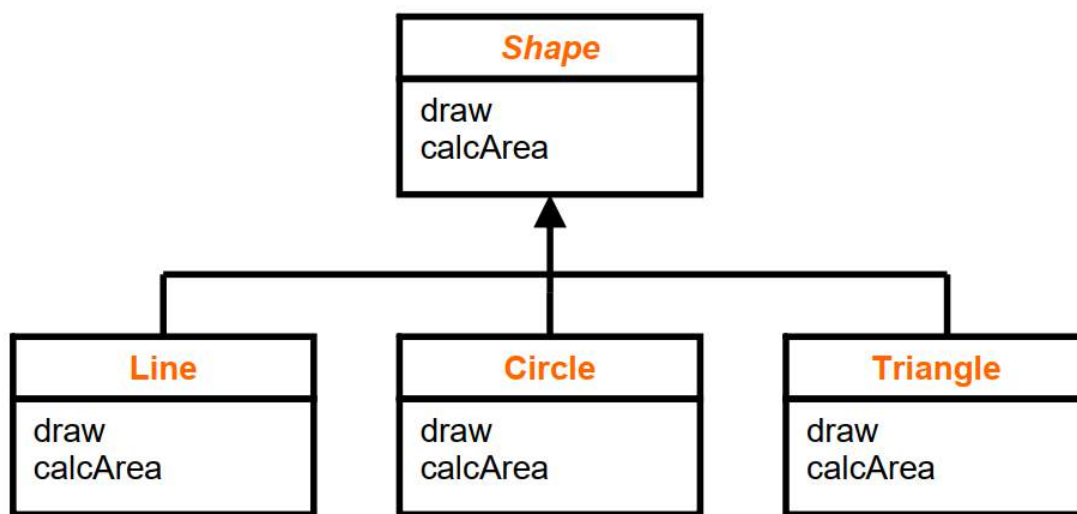
1. Use polymorphism to calculate total area.
2. The function must work for:
 - Rectangles
 - Triangles
 - Lines (area = 0)
3. No type checking allowed.
4. Adding a new shape must **not require modifying** this function.

Today lecture:

- Abstract vs Concrete classes
- Use of virtual and pure virtual function
- virtual destructor
- virtual table=> under the hood

Recap last lecture:

The main concept of Polymorphism is that the same method can behave differently according to the object with respect of which it has been called.



The `draw` and `calcArea` methods here are showing polymorphic behavior as they exist in base as well as in derived classes, the method that will be called is decided at runtime according to the nature of the calling object, for example in the case of above class hierarchy if we call `draw` method with respect of Line class object, `draw` of Line class will be called and if we call it with reference of Circle or Triangle class, `draw` method of Circle or Triangle class will be called.

We achieve this type of polymorphism using virtual functions.

The advantage of this approach is that sender simply pass method call and appropriate method is called automatically, as we saw in previous lecture in which we simply used for loop to draw all kinds of shapes using shape pointers array.

We also saw the drawbacks of achieving same functionality without virtual functions using switch statement that resulted in delocalized and complex code. Then we saw solution of this problem using virtual functions.

Abstract Class:

```
#include <iostream>

using namespace std;

class Shape
{
    public:

        virtual void draw() = 0; // pure virtual function make Shape as abstract class
                                // interface for imp of polymorphism
        virtual double calcArea()// provide interface (polymorphism) + implementation (code
reusability)
        {
            cout << "Shape calcArea\n";
            return 0.0;
        }

};

class Line:public Shape
{
    public:
        virtual void draw()
        {
            cout << "line\n";
        }

};

class Rectangle: public Shape
{
    public:
        void draw()
        {
            cout << "Rectangle" << endl;
        }
        double calcArea()
        {
            cout << "RectangleCalcArea()" << endl;
            return 1.0;
        }
        // move(), redraw(), and delete()etc and many more functions needs to
implement
```

```

};

class Triangle : public Shape
{
Public:
    void draw()
    {
        cout << "TriangleDraw()" << endl;
    }
    double calcArea()
    {
        cout << "TriangleCalcArea()" << endl;
        return 1.0;
    }
};

int main()
{
    Shape* sp;
    Shape p; // error as Shape is an abstract class
    sp = new Line();
    sp->draw();
    sp->calcArea();
    return 0;
}

```

Virtual Destructor:

```
class Shape
{
    public:
        virtual void draw() = 0;
        virtual double calcArea()
        {
            cout << "Shapecalare\n";
            return 0.0;
        }
        ~Shape()
        {
            cout << "~Shape\n";
        }
};

class Line:public Shape
{
    public:
        virtual void draw()
        {
            cout << "line\n";
        }
        ~Line()
        {
            cout << "~Line \n";
        }
};

int main()

{
    Shape* sp;
    sp = new Line();

    delete sp;
    return 0;
}
```

It cause memoy leakage.

Solution : virtual destructor

Output:

~shape

```

class Shape
{
    public:
        virtual void draw() = 0;
        virtual double calcArea()
        {
            cout << "Shapecalare\n";
            return 0.0;
        }
        Virtual ~Shape()
        {
            cout << "~Shape\n";
        }
};

class Line:public Shape
{
    public:
        virtual void draw()
        {
            cout << "line\n";
        }
        virtual ~Line()
        {
            cout << "~Line \n";
        }
};

int main()
{
    Shape* sp;
    sp = new Line();

    delete sp;
    return 0;
}

```

Output:
~Line()
~shape

Virtual Function Table:

```

class shape
{
public:
    virtual void draw() = 0;
    virtual double calcArea()
    {
        return 0.0;
    }
};

```

```

class Line : public Shape
{
public:
    virtual void draw()
    {
        cout << "Line";
    }
};

```

main()

```

{
    Shape *sp = new Line();

```

```

    sp -> draw();

```

```

    // *(*sp+0)

```

```

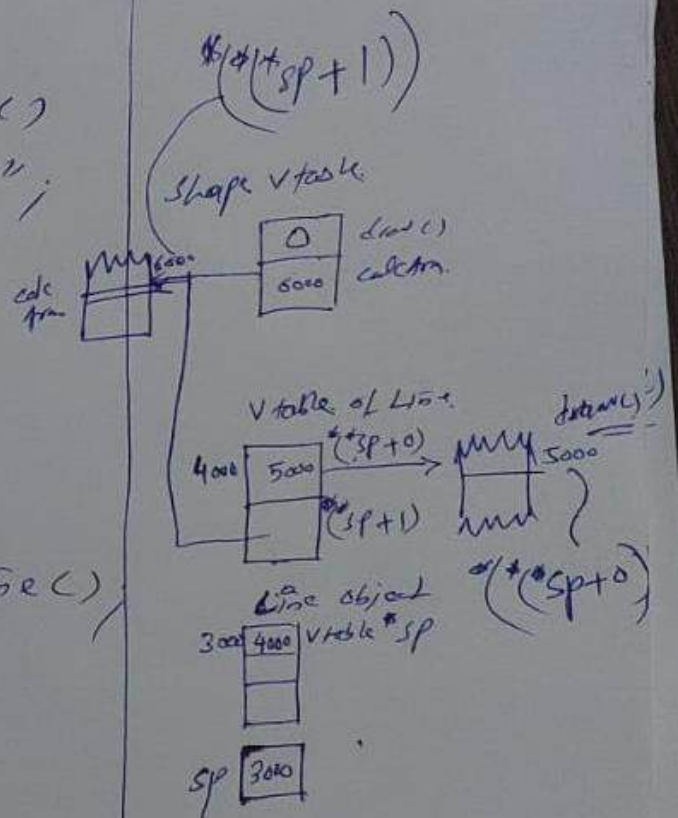
    sp -> calcArea();

```

```

    // *(*sp+1)

```



Home work:

Question1 : Code Completion (Abstract Class)

Complete the following code so that it demonstrates runtime polymorphism correctly:

```
class Shape {  
public:  
    virtual void draw() = 0;  
    virtual double calcArea() = 0;  
    // destructor missing  
};
```

```
class Circle : public Shape {  
    double radius;  
public:  
    Circle(double r);  
    void draw();  
    double calcArea();  
};
```

👉 Tasks:

1. Complete the Circle class implementation.
2. Add a virtual destructor in Shape.
3. Write a main() function that uses a Shape* to call draw() and calcArea().

Question 2: Programming Task (Mini Design)

Design a small system using polymorphism:

Requirements:

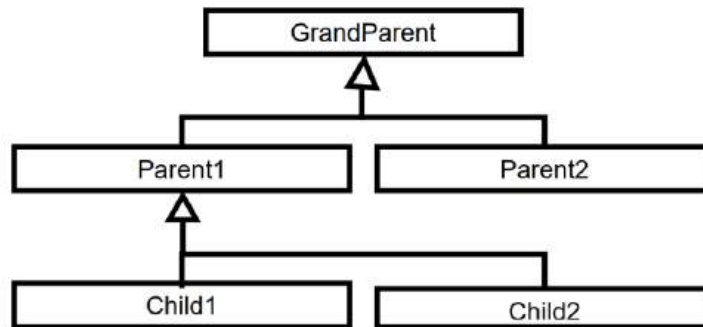
- Create an abstract base class Shape
- Include at least:
 - One pure virtual function
 - One virtual function with implementation
 - A virtual destructor
- Derive at least three classes (e.g., Line, Rectangle, Triangle)
- Store objects in a Shape pointer array
- Use a loop to call draw() and calcArea() for all objects

👉 Do NOT use switch-case or if-else for type checking

1. Multilevel Inheritance/ Hierarchy of Inheritance

We represent the classes involved in inheritance relation in tree like hierarchy

a. Example



b. here is the concept of direct base class and indirect base class

Direct Base Class

A direct base class is explicitly listed in a derived class's header with a colon (:)

```
class Child1:public Parent1 {    // Here Parent1 is Direct Base Class of Child1
...
};
```

Indirect Base Class

An indirect base class is not explicitly listed in a derived class's header with a colon (:)

It is inherited from two or more levels up the hierarchy of inheritance

```
class GrandParent {
...
};
class Parent1: public GrandParent {
...
};
class Child1:public Parent1 {    // Here GrandParent is InDirect Base Class of Child1
...
};
```

c. Base Initialization in class hierarchy:

As we have seen earlier in the case of copy constructor we have to call base class constructor from the initialization list of child class because base class object is created first and then derived class object is created. Now we see this concept in detail in the perspective of class hierarchy,

- The child can only call constructor of its direct base class to perform its initialization using its constructor *initialization list*.
- The child cannot call the constructor of any of its indirect base classes to perform their initialization using its constructor *initialization list*

It is explained in example code given below,

d. Example:

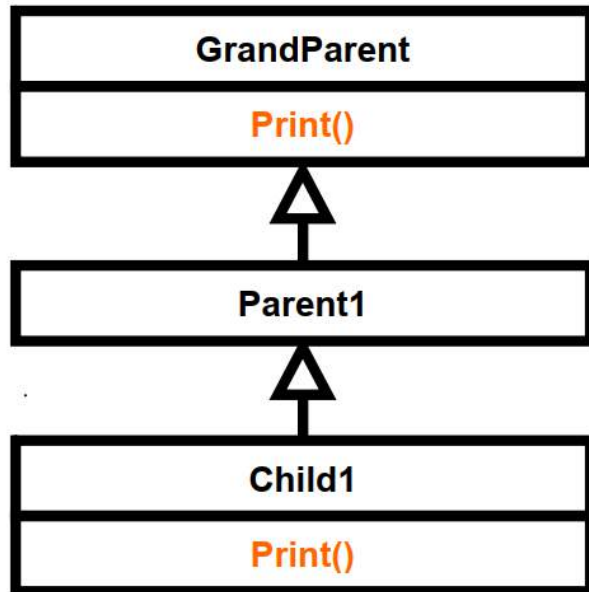
```
#include <iostream>
using namespace std;
class GrandParent {
    int gpData;
public:
    GrandParent() : gpData(0) { cout << "GrandParent()\n"; }
    GrandParent(int i) : gpData(i) { cout << "GrandParent(int i)\n"; }
    void Print() const { cout << "GrandParent::Print()\n"; }
    //virtual void Print() const { cout << "GrandParent::Print()\n"; }
};
class Parent1 : public GrandParent {
    int pData;
public:
    Parent1() : GrandParent(), pData(0) { cout << "Parent1()\n"; }
};
class Child1 : public Parent1 {
public:
    Child1() : Parent1() { cout << "Child1()\n"; }
    //Child1(int i) : GrandParent(i) //Error: Child1 can not call its
    //indirect base class GrandParent Constructor from its
    constructor
    //initialization list.
    void Print() const
    {
        cout << "Child1::Print()\n";
    }
};
int main()
{
    Child1 c1;
    c1.Print();
    GrandParent *gp;
    gp = &c1;
    gp->Print();
}
```

output

```
GrandParent()
Parent1()
Child1()
Child1::Print()
GrandParent::Print()
```

e. **Overriding in case of class hierarchy:**

In class hierarchy Child class can override the function of any of its Parent class (direct or indirect) as shown in the diagram below (we see this concept in detail in types of inheritance given below)



```
void GrandParent::Print() {
cout << "GrandParent::Print"<< endl;
}
void Child1::Print() {
cout << "Child1::Print" << endl;
}
int main(){
Child1 obj
obj.Print();
obj.Parent1::Print();
obj.GrandParent::Print();
return 0;
}
```

2. Types of Inheritance

There are three types of inheritance

- Public
- Protected
- Private

We can use these keywords (public, private or protected) to specify the type of inheritance

a. Public Inheritance

```
class Child: public Parent {...};
```

Member access in	
Base Class	Derived Class
Public	Public
Protected	Protected
Private	Hidden

b. Protected Inheritance

```
class Child: protected Parent {...};
```

Member access in	
Base Class	Derived Class
Public	Protected
Protected	Protected
Private	Hidden

c. Private Inheritance

```
class Child: private Parent {...};
```

Member access in	
Base Class	Derived Class
Public	Private
Protected	Private
Private	Hidden

If the user does not specifies the type of inheritance then the default type is private inheritance,
class Child: private Parent {...} is equivalent to,
class Child: Parent {...}

We have seen public inheritance in previous lecture, now we see private and protected inheritance in detail,

a. Private Inheritance:

What Private Inheritance Prevents

Feature	Public Inheritance	Private Inheritance
Is-a relationship	✔ Yes	✗ No
Polymorphism	✔ Yes	✗ No
Base pointer to derived	✔ Allowed	✗ Not allowed
Interface inheritance	✔ Yes	✗ No
Implementation reuse	✔ Yes	✔ Yes

```
#define _CRT_SECURE_NO_WARNINGS
#pragma warning(disable : 4996)
#include <iostream>
using namespace std;

const int MAX_SIZE = 100;
/* =====
   Base Class: Collection
   ===== */
class Collection {
private:
    int data[MAX_SIZE];
    int size;

public:
    Collection() : size(0) {}

    void AddElement(int x) {
        if (size < MAX_SIZE) {
            data[size++] = x;
        }
    }
}
```

```

        else {
            cout << "Collection is full\n";
        }
    }

    bool SearchElement(int x) {
        for (int i = 0; i < size; i++) {
            if (data[i] == x)
                return true;
        }
        return false;
    }

    bool SearchElementAgain(int x) { // redundant but kept as given
        return SearchElement(x);
    }

    bool DeleteElement(int x) {
        for (int i = 0; i < size; i++) {
            if (data[i] == x) {
                // Shift elements left
                for (int j = i; j < size - 1; j++) {
                    data[j] = data[j + 1];
                }
                size--;
                return true;
            }
        }
        return false;
    }

protected:
    void PrintElements() const {
        for (int i = 0; i < size; i++) {
            cout << data[i] << " ";
        }
        cout << endl;
    }
};

```

```

/* =====
Derived Class: Set
(Private Inheritance)
===== */
class Set : private Collection {
public:
    void AddMember(int i) {
        if (!IsMember(i))
            AddElement(i); // inherited privately
    }

    bool IsMember(int i) {
        return SearchElement(i);
    }

    bool DeleteMember(int i) {
        return DeleteElement(i);
    }

    void PrintSet() {
        PrintElements(); // protected access
    }
};

/* =====
Main Function
===== */
int main() {
    Set s;

    s.AddMember(10);
    s.AddMember(20);
    s.AddMember(10); // duplicate ignored
    s.AddMember(30);

    cout << "Set elements: ";
    s.PrintSet();

    cout << "Is 20 a member? " << s.IsMember(20) << endl;
    cout << "Is 40 a member? " << s.IsMember(40) << endl;

    s.DeleteMember(20);
    cout << "After deleting 20: ";
    s.PrintSet();
}

```

```

    return 0;
}

```

Properties of private Inheritance:

1. In Private Inheritance only member functions and friend classes or functions of a derived class can convert pointer or reference of derived object to that of parent object

```

class Parent{ };
class Child : private Parent{ };

```

```

int main(){ Child cobj;
Parent *pptr = & cobj; //Error
return 0; }

```

2. Passing the child object to the reference of Parent during parent function call is possible and similar copy constructor of parent class can be invoked from the base initialization list of the child class copy constructor as was in the case of public inheritance child class object has an anonymous object of parent class. As was in the case of public inheritance the default constructor and copy constructor of parent class are called when constructing object of derived class.

```

class Parent {
public:
    Parent() {
        cout << "Parent Constructor\n";
    }
    Parent(const Parent& prhs) {
        cout << "Parent Copy Constructor\n";
    }
    void doSomething(const Parent& p)
    {
        cout << "void doSomething(const Parent& p) \n";
    }
};
class Child : private Parent {
public:
    Child()
    {
        cout << "Child Constructor";
        doSomething(*this);
    }
    Child(const Child& crhs) : Parent(crhs) {
        cout << "Child Copy Constructor";
    }
};
int main() {

```

```

Child cobj1; // default constructor will be invoked
Child cobj2 = cobj1; // copy constructor will be invoked
//Child cobj2(cobj1); // another way of calling copy constructor
return 0;
}

```

3. In private inheritance the derived class that is more than one level down the hierarchy cannot access the member functions of grand parent class as public and protected members functions of derived class become private members of privately derived class for all practical purposes. For example see the code below here Child class is derived class that is more than one level down the hierarchy and hence can not access the member functions of **GrandParent** class.

```

#include <iostream>
using namespace std;

```

```

class GrandParent
{
    public:
        void doSomething(const GrandParent& p)
        {
            cout << "void doSomething(const Parent& p) \n";
        }
        void doSomething()
        {
            cout << "void doSomething(const Parent& p) \n";
        }
        GrandParent() {
            cout << "Default Constructor\n";
        }
        GrandParent(const GrandParent& prhs) {
            cout << "Parent Copy Constructor\n";
        }
};

class Parent:private GrandParent {
public:
    Parent() {
        doSomething(*this);
        doSomething();

        cout << "Parent Constructor\n";
    }
    Parent(const Parent& prhs): GrandParent(prhs) {
        cout << "Parent Copy Constructor\n";
    }
}

```

```

    }

};
class Child : private Parent {
public:
    Child()
    {
        cout << "Child Constructor";
        //doSomething(*this); // error
        //doSomething(); // error
    }
    Child(const Child& crhs) : Parent(crhs) {
        cout << "Child Copy Constructor";
    }
};

int main() {
    Child cobj1; // default constructor will be invoked
    Child cobj2 = cobj1; // copy constructor will be invoked
    //Child cobj2(cobj1); // another way of calling copy
    constructor
    return 0;
}

```

Output:

```

Default Constructor
void doSomething(const Parent&
p)
void doSomething(const Parent&
p)
Parent Constructor
Child ConstructorParent Copy
Constructor

```

4. In private inheritance the derived class that is more than one level down the hierarchy cannot convert the pointer or reference to that of GrandParent, for example in code given below Child class can not convert its reference to its GrandParent reference. The reason is same that in private inheritance we implement specialization and we restrict all features of base class to privately derived class only and its friend classes or functions.

Protected Inheritance: Child class can access/call the member functions of grand parent class

```
class GrandParent
{
    public:
        void doSomething(const GrandParent& p)
        {
            cout << "void doSomething(const Parent& p) \n";
        }
        void doSomething()
        {
            cout << "void doSomething(const Parent& p) \n";
        }
        GrandParent() {
            cout << "Default Constructor\n";
        }
        GrandParent(const GrandParent& prhs) {
            cout << "Parent Copy Constructor\n";
        }
};

class Parent:private GrandParent {
public:
    Parent() {
        doSomething(*this);
        doSomething();

        cout << "Parent Constructor\n";
    }
    Parent(const Parent& prhs): GrandParent(prhs) {
        cout << "Parent Copy Constructor\n";
    }
};

class Child : private Parent {
public:
    Child()
    {
        cout << "Child Constructor";
        doSomething(*this); // OK
        doSomething(); // OK
    }
    Child(const Child& crhs) : Parent(crhs) {
        cout << "Child Copy Constructor";
    }
}
```

```
};  
int main() {  
    Child cobj1; // default constructor will be invoked  
    Child cobj2 = cobj1; // copy constructor will be invoked  
    //Child cobj2(cobj1); // another way of calling copy  
    constructor  
    return 0;  
}
```

Output:

Default Constructor

void doSomething(const Parent& p)

void doSomething(const Parent& p)

Parent Constructor

Child Constructorvoid doSomething(const Parent& p)

void doSomething(const Parent& p)

Parent Copy Constructor

Parent Copy Constructor

Child Copy Constructor

Multiple Inheritance

Multiple inheritance is a feature of C++ in which a **derived class inherits from more than one base class**. This allows the derived class to combine behaviors and data from multiple parent classes.

EXAMPLE 1:

```
#include <iostream>
using namespace std;

class Teacher {
public:
    void teach() {
        cout << "Teaching a class\n";
    }
};

class Student {
public:
    void study() {
        cout << "Studying\n";
    }
};

class TeachingAssistant : public Teacher, public Student {
public:
    void assist() {
        cout << "Assisting in teaching\n";
    }
};

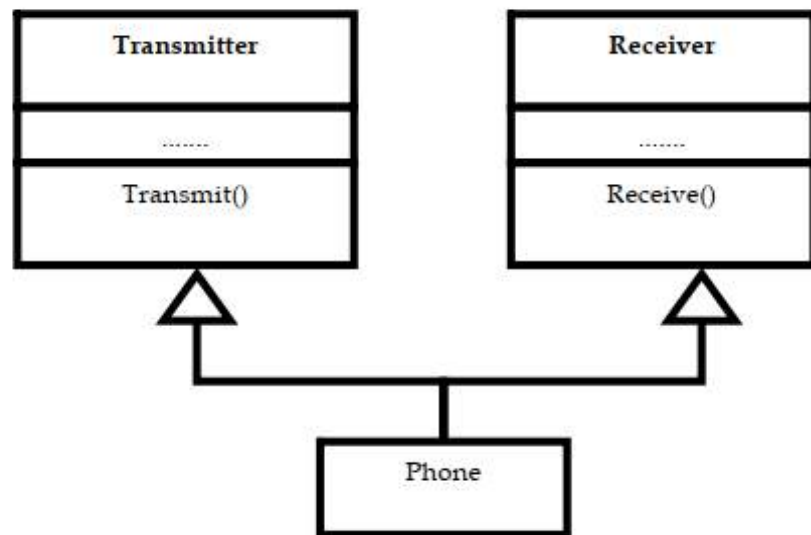
int main() {
    TeachingAssistant ta;

    ta.teach();    // from Teacher
    ta.study();    // from Student
    ta.assist();   // from TeachingAssistant

    Teacher* t;
    t = &ta;
    t->teach();
    //t->study(); // error
    Student* s;
    s = &ta;
    s->study();
    //s->teach(); // error
    return 0;
}
```

EXAMPLE2:

We have seen multiple inheritance in start of OOP now we see its implementation in C++, A class in C++ can inherit from more than one classes like Phone class can inherit from Transmitter or Receiver



```
#include <iostream>
using namespace std;

class Transmitter
{
public:
    virtual void transmit()
    {
        cout << "void tranmit()\n";
    }
};

class Reciver
{
public:
    virtual void receive()
    {
        cout << "void receive()\n";
    }
};

class Phone : public Transmitter, public Reciver
{
public:
    Phone()
    {
        cout << "Phone() \n";
    }
};
```

```

    }
    virtual void transmit()
    {
        cout << "phone ::void tranmit()\n";
    }
    virtual void receive()
    {
        cout << "phone ::void receive()\n";
    }
};

int main() {
    Phone o1;
    o1.transmit();
    o1.receive();

    //Due to Public Inheritance, when multiple class are publically inheritance, the
    object of derived clas cna replace with object
    // of all the base class
    Transmitter * pt;
    Reciver* pr;

    pt = &o1;
    pr = &o1;

    pt->transmit();
    //pt->receive();
    //pr->transmit();
    pr->receive();

    return 0;
}

```

////////

Example 3: class Mermaid: private Woman, private Fish

// As Mermaid class is privately inherited from Woman and Fish so any class derived from Mermaid class will Not have access to public and protected members of Woman and Fish classes

{ ... }

2. Problems in Multiple Inheritance / Ambiguity Problem in Multiple Inheritance

CASE: 1

EXAMPLE 1:

If two base classes contain functions or data members with the same name, the compiler gets confused. Ambiguous Example

```
#include <iostream>
```

```
using namespace std;
```

```
class A {
```

```
public:
```

```
    void show() {
```

```
        cout << "A::show()\n";
```

```
    }
```

```
};
```

```
class B {
```

```
public:
```

```
    void show() {
```

```
        cout << "B::show()\n";
```

```
    }
```

```
};
```

```
class C : public A, public B
```

```
{
```

```
public :
```

```
    //void show() // solution 2
```

```
    //{
```

```
        // cout << "C::show()\n";
```

```
    //}
```

```
};
```

```
int main() {
```

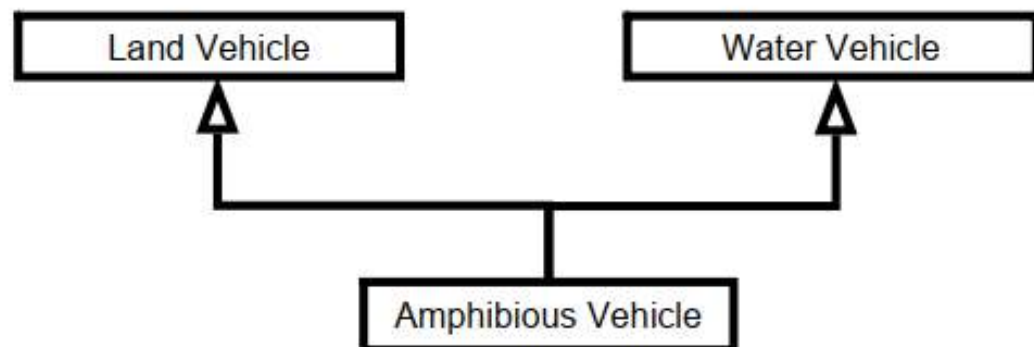
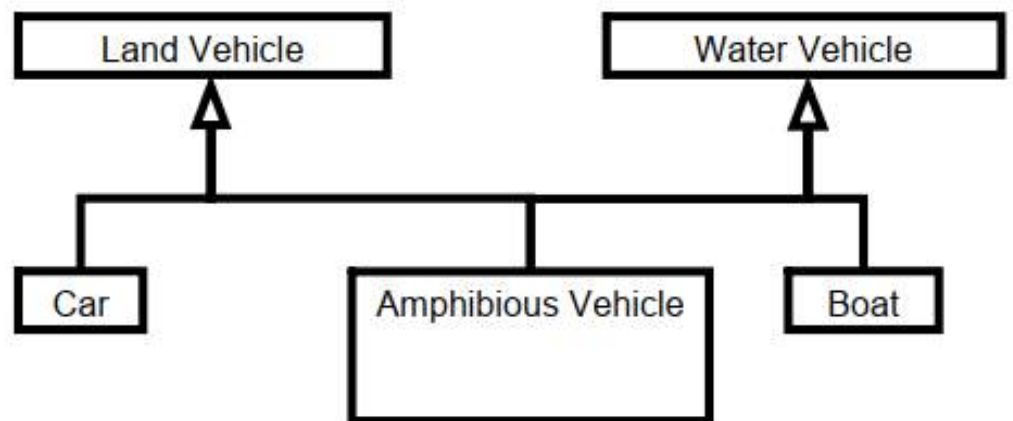
```
    C obj;
```

```
// obj.show(); // ERROR: ambiguous  
//Solution :1. Scope Resolution Operator
```

```
obj.A::show();  
obj.B::show();  
//      2. overriding in derived class  
obj.show();
```

```
}
```

Example 2:



```

class LandVehicle {
    double lWeight;
public:
    LandVehicle()
    {
        lWeight = 100;
    }

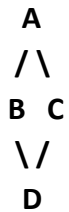
    double GetMaxLoad()
    {
        return lWeight;
    }
};
class WaterVehicle {
    double wWeight;
public:
    WaterVehicle()
    {
        wWeight = 200;
    }
    double GetMaxLoad()
    {
        return wWeight;
    }
};
class AmphibiousVehicle : public LandVehicle, public WaterVehicle {
};
int main() {
    AmphibiousVehicle obj;
    //obj.GetMaxLoad(); // Error
    // Solution 1: The solution of this problem is that Programmer must explicitly
specify the class
    //name when calling ambiguous function
    cout << obj.LandVehicle::GetMaxLoad();
    cout << obj.WaterVehicle::GetMaxLoad();
//solution 2: overriding the function
    return 0;
}

```

2. Diamond Problem (Major Issue)

Occurs when two base classes inherit from the same base class, and a derived class inherits from both.

✗ Diamond Structure



Problem:

D gets two copies of A, causing ambiguity and duplication.

```
class A {
protected:
    int data;
public:
    void show() {
        cout << "A::show()\n";
    }
};

class B: public A
{

};

class C : public A
{

};

class D : public B, public C
{
public:
    D()
    {
        // data = 20; // error duplication
        C::data = 30;
        //B::data = 20;
```

```

    }
};

int main() {
    D obj;
    // obj.show(); // ✗ ERROR: ambiguous
    //Solution :1. Scope Resolution Operator

    obj.B::show();
    //obj.C::show();

    //      2. overriding in derivied class
    //obj.show();

}
// b. data members

```

◆ Solution: Virtual Inheritance

```

class A {
public:
    int x;
};

class B : virtual public A { };
class C : virtual public A { };

class D : public B, public C { };

int main() {
    D obj;
    obj.x = 10; // ✓ only one copy of A
}

```

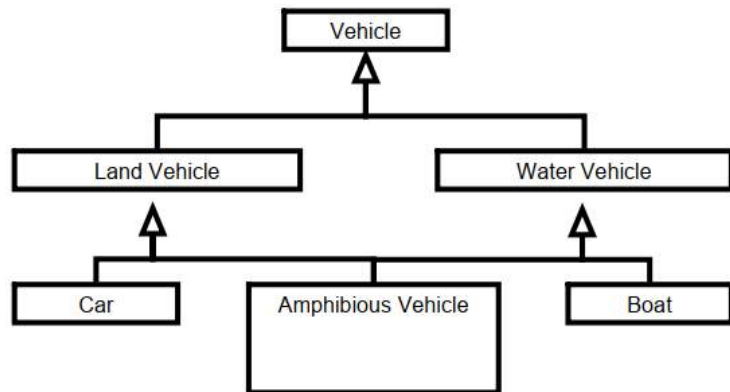
◆ Summary Table

Aspect	Description
Definition	Inherit from more than one base class

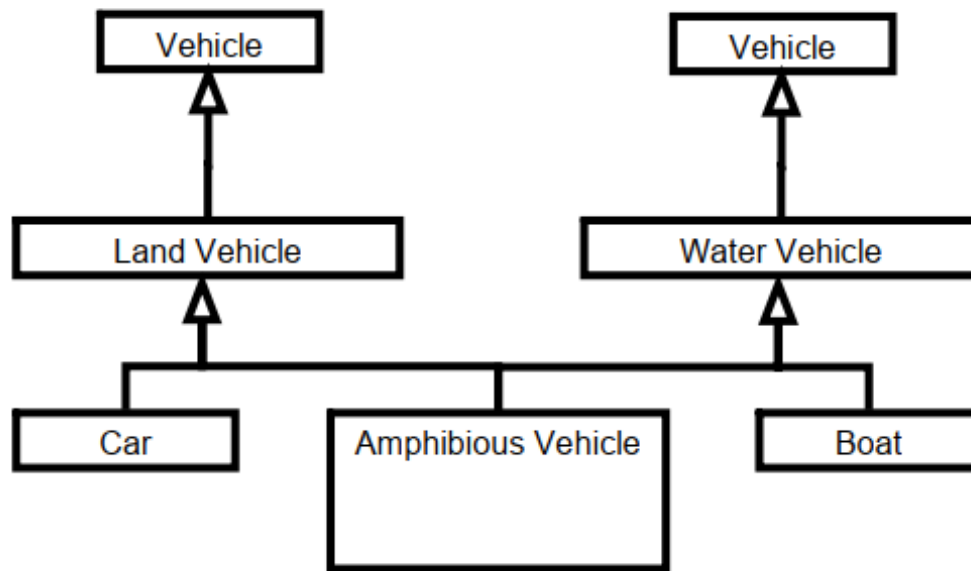
Aspect	Description
Benefit	Combines multiple behaviors
Problem	Ambiguity, diamond problem
Solution	Scope resolution, virtual inheritance
Best Practice	Use for interfaces

Example2:

The ambiguous call problem can arise when dealing with multiple level of multiple inheritance



OR we can view it as



```
#include <iostream>
using namespace std;

class Vehicle
{
protected:
    double weight;
public:
    Vehicle()
    {
        weight = 100;
    }
    double getMaxLoad()
    {
        return weight;
    }
};

class LandVehicle:public Vehicle {

};

class WaterVehicle: public Vehicle {

};

class AmphibiousVehicle : public LandVehicle, public WaterVehicle {
```

```

};
int main() {
    AmphibiousVehicle obj;
    //cout << obj.getMaxLoad(); //
    // Error : The error is due to the reason that object of AmphibiousVehicle class
    has two implicit
    //Vehicle class objects one with respect to LandVehicle and one with respect to
    // WaterVehicle class,
    // Solution 1: The solution of this problem is that Programmer must explicitly
    specify the class
    //name when calling ambiguous function
    cout << obj.Vehicle ::getMaxLoad();
    cout << obj.LandVehicle::getMaxLoad();
    cout << obj.WaterVehicle::getMaxLoad();
    return 0;
}

```

4. DataMembers issue in multiple with multi level inheritance:

```

#include <iostream>
using namespace std;

class Vehicle
{
protected:
    double weight;
public:

    double getMaxLoad()
    {
        return weight;
    }
};

class LandVehicle:public Vehicle {

};

class WaterVehicle: public Vehicle {

};

class AmphibiousVehicle : public LandVehicle, public WaterVehicle {

```

```

public:
    AmphibiousVehicle()
    {
        weight = 100 // error due to two copies of weights
        LandVehicle::weight = 100;
        WaterVehicle::weight = 100;
    }
};

int main() {
    AmphibiousVehicle obj;
    //cout << obj.getMaxLoad(); //
    // Error : The error is due to the reason that object of AmphibiousVehicle class
has two implicit
    //Vehicle class objects one with respect to LandVehicle and one with respect to
    // WaterVehicle class,
    // Solution 1: The solution of this problem is that Programmer must explicitly
specify the class
    //name when calling ambiguous function
    cout << obj.Vehicle ::getMaxLoad();
    cout << obj.LandVehicle::getMaxLoad();
    cout << obj.WaterVehicle::getMaxLoad();
    return 0;
}

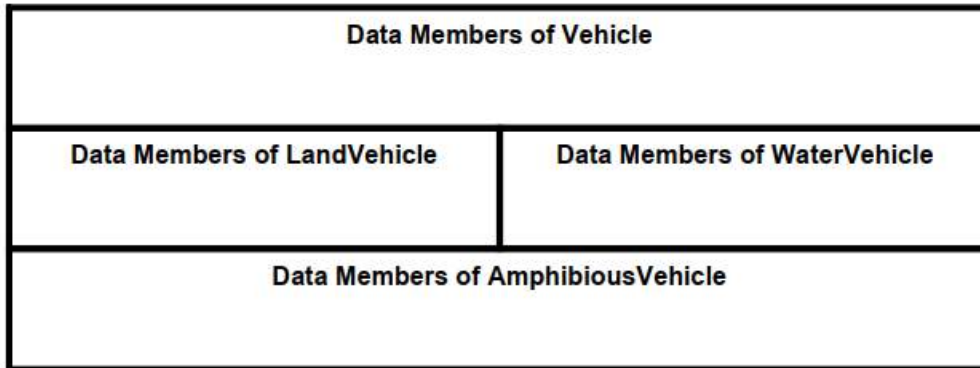
```

Data Members of Vehicle	Data Members of Vehicle
Data Members of LandVehicle	Data Members of WaterVehicle
Data Members of AmphibiousVehicle	

Virtual Inheritance

The solution to avoid this problem is virtual inheritance so that in multiple inheritance only one copy of base class is generated as shown below instead of two separate copies.

Memory View: In virtual inheritance there is exactly one copy of the anonymous base class object.



```

class Vehicle
{
protected:
    double weight;
public:

    double getMaxLoad()
    {
        return weight;
    }
};

class LandVehicle:public virtual Vehicle {

};
class WaterVehicle: public virtual Vehicle {

};
class AmphibiousVehicle : public LandVehicle, public WaterVehicle {

public:
    AmphibiousVehicle()
    {
        //LandVehicle::weight = 100;
        //WaterVehicle::weight = 100;
        weight = 100;
    }
};
int main() {
    AmphibiousVehicle obj;
    cout << obj.getMaxLoad();
    return 0;
}

```

When to use Virtual Inheritance?

Virtual inheritance must be used when necessary. It can be used in the situations when programmer wants to use two distinct data members inherited from base class rather than one.

